LLM 学习

大语言模型的核心机制、训练方法与推理系统

Yang kunpeng

目录

第一章 QKV 注意力机制 **2**

1.1 Multi-Head Attention（MHA） 2

1.2 Multi-Query Attention（MQA） 5

1.3 Grouped-Query Attention（GQA） 8

1.4 KV Cache：自回归推理中的 K/V 缓存 12

1.5 Multi-head Latent Attention（MLA） 18

1.6 QKV 注意力机制中的关键注意事项 23

第二章 Transformer 中的位置编码 **29**

2.1 绝对位置编码：Sinusoidal 与 Learned Position Embedding 29

2.2 相对位置编码：Shaw、Transformer-XL、T5 与 DeBERTa 32

2.3 RoPE：旋转位置编码 36

2.4 ALiBi：线性位置偏置 40

2.5 RoPE 的长上下文扩展：PI、NTK-aware Scaling 与 YaRN 42

2.6 不同位置编码方法的比较 46

第三章 Normalization 与残差连接 **48**

3.1 Residual Connection：残差连接 49

3.2 LayerNorm：层归一化 51

3.3 RMSNorm：均方根归一化 54

3.4 Pre-Norm 与 Post-Norm 56

3.5 Python 实现：LayerNorm、RMSNorm、Pre-Norm 与 Post-Norm . . . 60

3.6 Normalization 与残差结构的统一理解 64

第四章 前馈网络（Feed Forward Network，FFN） **67**

4.1 Transformer FFN：逐位置前馈网络 67

4.2 Python 实现：手写 SwiGLU FFN 77

4.3 不同 FFN 与激活函数的比较 79

4.4 FFN 在现代 Transformer Block 中的位置 80

第五章 Mixture of Experts：稀疏专家模型与现代 MoE LLM 82
5.1 从 Dense FFN 到 Mixture of Experts 83
5.2 Router、Gate 与 Top-K Routing 84
5.3 Expert FFN 与 Sparse Activation 86
5.4 Load Balancing、Capacity 与 Routing Stability 89
5.5 MoE 架构的发展与代表模型 92
5.6 Expert Parallelism 与分布式 MoE 94
5.7 Python 实现：最小 Sparse MoE 96
5.8 Dense 与 MoE 的比较及常见误区 101

第六章 Transformer 模型类型与现代 LLM 架构演进 103
6.1 从 Transformer Block 到完整模型 104
6.2 三种经典 Transformer 架构 105
6.3 现代 LLM 架构的扩展 111
6.4 Python 实现：三种 Transformer 的最小结构 117
6.5 三种模型类型的比较 120

第七章 LLM 推理与文本生成：Decoding、Sampling 与性能指标 123
7.1 从 Logits 到最终生成文本 123
7.2 Decoding 与 Sampling 的基本分类 124
7.3 Python：完整 Sampling 与 Decode Loop 130
7.4 Prefill 与 Decode 134
7.5 LLM 推理性能指标 136
7.6 生成策略与推理性能的统一理解 138

第八章 语言模型训练：从 Next Token Prediction 到预训练 140
8.1 语言模型目标与损失函数 140
8.2 大规模预训练：数据、优化与计算规模 147
8.3 训练流程与规模规律的统一理解 157

第九章 LLM 微调与对齐：SFT、PEFT、RLHF 与偏好优化 159
9.1 从预训练到 Post-Training 159
9.2 SFT：从 Instruction Data 到训练 Loss 161
9.3 Full Fine-Tuning 与 PEFT 164
9.4 从 SFT 到 Preference Alignment 173

9.5 Post-Training 方法的比较 179
9.6 LLM Post-Training 的完整工程流程 180

第十章 模型量化：从 FP32 到 INT4 高效推理 183
10.1 为什么 LLM 需要量化 184
10.2 Quantization 数学：Scale、Zero Point 与误差 187
10.3 PTQ 与 QAT 191
10.4 GPTQ 与 AWQ：低比特权重量化 192
10.5 Quantized Linear 与真实推理 Kernel 196
10.6 Python 实现：基础 INT8 Quantization 198
10.7 量化方法的比较与常见误区 202

第十一章 FlashAttention：从标准 Attention 到 IO-Aware Attention 206
11.1 标准 Attention 的计算与显存问题 207
11.2 IO-Aware Attention 与 Online Softmax 209
11.3 FlashAttention-2：并行策略与性能改进 213
11.4 Python：标准 Attention 与 Tiled Online Attention 214
11.5 PyTorch SDPA 与工程实现 217
11.6 FlashAttention 的适用边界与总结 219

第十二章 vLLM 与高性能 LLM 推理 220
12.1 LLM 推理的两个阶段：Prefill 与 Decode 221
12.2 KV Cache：推理显存的核心状态 223
12.3 PagedAttention：分页式 KV Cache 管理 224
12.4 请求调度与生成优化 225
12.5 LLM 推理的分布式并行 228
12.6 一次请求的完整 Serving 生命周期 229
12.7 Python：简化 Paged KV Cache 230
12.8 推理优化的统一性能框架 235

LLM 数学和代码实现

从 QKV 到部署

–作者: Yang kunpeng

第 1 章

QKV 注意力机制

QKV 注意力机制是 Transformer 中建立 token 之间信息交互关系的核心计算结构。设序列输入为 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$, 其中 B 表示 batch size，L 表示序列长度，d_{model} 表示隐藏维度。$\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 均由输入 $\boldsymbol{X}$ 经过线性投影得到：

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^{\mathrm{Q}}, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^{\mathrm{K}}, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^{\mathrm{V}}.$$

Self-Attention 的基础计算采用缩放点积注意力：

$$\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}.$$

MHA、MQA、GQA 和 MLA 的关键差异在于 Query、Key 与 Value 的组织或压缩方式。MHA 为每个 Query Head 配置独立的 K/V；MQA 让所有 Query Head 共享同一组 K/V；GQA 位于二者之间，让一组 Query Head 共享一个 KV Head。在此基础上，本章单独讨论自回归推理中的 KV Cache，解释为什么历史 K/V 可以复用、缓存怎样增长，以及 MQA / GQA 为什么能够显著降低缓存压力。随后，MLA 进一步从“减少 KV Head 数量”转向“压缩 KV Representation”，使用低维 Latent 表示降低 KV Cache。最后，本章统一讨论缩放、Mask、Tensor Shape 与实现时需要注意的细节。

1.1 Multi-Head Attention（MHA）

Self-Attention 与 Q、K、V 在 Self-Attention 中，$\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 都来自同一个输入 $\boldsymbol{X}$。对于第 i 个注意力头，可以写成

$$\boldsymbol{Q}_i = \boldsymbol{X}\boldsymbol{W}_i^{\mathrm{Q}} \in \mathbb{R}^{L\times d_k}, \qquad \boldsymbol{K}_i = \boldsymbol{X}\boldsymbol{W}_i^{\mathrm{K}} \in \mathbb{R}^{L\times d_k}, \qquad \boldsymbol{V}_i = \boldsymbol{X}\boldsymbol{W}_i^{\mathrm{V}} \in \mathbb{R}^{L\times d_k}.$$

这里省略 batch 维度，主要关注单个样本内部的序列计算。

单个注意力头首先计算 Query 与 Key 的点积：$\boldsymbol{Q}_i\boldsymbol{K}_i^{\mathsf{T}} \in \mathbb{R}^{L\times L}$. 因此，$\boldsymbol{Q}_i\boldsymbol{K}_i^{\mathsf{T}}$ 得到的是一个 token 与 token 之间的两两注意力分数矩阵。随后对该矩阵进行缩放和 Softmax，并利用得到的权重对 Value 进行加权：

$$\text{head}_i = \text{softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}_i^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}_i.$$

将 $\boldsymbol{Q}_i$、$\boldsymbol{K}_i$、$\boldsymbol{V}_i$ 的投影过程代入后，可以写为

$$\text{head}_i = \text{softmax}\left(\frac{\boldsymbol{X}\boldsymbol{W}_i^{\text{Q}}(\boldsymbol{X}\boldsymbol{W}_i^{\text{K}})^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{X}\boldsymbol{W}_i^{\text{V}}.$$

因此，一个 Attention Head 的核心数据流可以概括为

$$\boldsymbol{X} \longrightarrow (\boldsymbol{Q}_i, \boldsymbol{K}_i, \boldsymbol{V}_i) \longrightarrow \boldsymbol{Q}_i\boldsymbol{K}_i^{\mathsf{T}} \longrightarrow \frac{\boldsymbol{Q}_i\boldsymbol{K}_i^{\mathsf{T}}}{\sqrt{d_k}} \longrightarrow \text{softmax} \longrightarrow \boldsymbol{V}_i \longrightarrow \text{head}_i\,.$$

MHA 的计算过程 如果只使用一个 Attention Head，模型只在一个表示子空间中计算 token 之间的关系。Multi-Head Attention 使用多个 Head，每个 Head 使用各自的投影矩阵

$$\boldsymbol{W}_i^{\text{Q}}, \qquad \boldsymbol{W}_i^{\text{K}}, \qquad \boldsymbol{W}_i^{\text{V}},$$

从而可以在不同表示子空间中计算注意力关系。

假设一共有 h 个注意力头，则得到 $\text{head}_1, \text{head}_2, \ldots, \text{head}_h$. 将它们在 Head 维度上拼接：

$$\boldsymbol{H} = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_h\right),$$

再经过输出投影矩阵 $\boldsymbol{W}^{\text{O}}$：

$$\text{MHA}(\boldsymbol{X}) = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_h\right)\boldsymbol{W}^{\text{O}}.$$

为了理解实际实现中的 Tensor Shape，考虑 $d_{\text{model}} = 512, \qquad h = 8$. 则每个 Head 的维度为 $d_k = d_v = \frac{512}{8} = 64$. 对应的主要 Shape 如表所示。

计算阶段	Tensor Shape
输入 $\boldsymbol{X}$	$[B, L, 512]$
线性投影后的 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$	$[B, L, 512]$
拆分为 8 个 Head	$[B, 8, L, 64]$
Attention Score $\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}$	$[B, 8, L, L]$
每个 Head 的输出	$[B, 8, L, 64]$
拼接 Head 后	$[B, L, 512]$

这里最重要的变化发生在 $[B, L, 512]$ 与 $[B, 8, L, 64]$ 之间。模型并没有改变总隐藏维度，只是把最后一个维度拆成了 Head 数量和单个 Head 的维度。

MHA 的特点 MHA 的结构可以简化表示为

$$\boldsymbol{Q}_1 \to (\boldsymbol{K}_1, \boldsymbol{V}_1), \quad \boldsymbol{Q}_2 \to (\boldsymbol{K}_2, \boldsymbol{V}_2), \quad \ldots, \quad \boldsymbol{Q}_h \to (\boldsymbol{K}_h, \boldsymbol{V}_h).$$

每个 Query Head 都有独立的 Key Head 和 Value Head，因此 $h_q = h_{\text{kv}}$.

这种结构保留了多组独立的 K/V 表示，但也意味着在自回归推理阶段需要缓存每一个 KV Head 对应的历史 K/V。后面的 MQA 和 GQA 正是通过改变 K/V Head 的数量以及共享方式，对这一结构进行调整。

Python 实现：完整 MHA 类 下面给出与上述数学过程对应的完整 PyTorch 实现。代码首先计算 Query、Key、Value，然后拆分 Head，计算缩放点积注意力，最后重新合并各个 Head 并进行输出投影。

```python
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class MHA(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()

        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
```

```
        self.d_head = d_model // num_heads

        self.wq = nn.Linear(d_model, d_model, bias=False)
        self.wk = nn.Linear(d_model, d_model, bias=False)
        self.wv = nn.Linear(d_model, d_model, bias=False)
        self.wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, mask=None):
        B, L, _ = x.shape

        Q = self.wq(x)
        K = self.wk(x)
        V = self.wv(x)

        Q = Q.view(B, L, self.num_heads, self.d_head).transpose(1, 2)
        K = K.view(B, L, self.num_heads, self.d_head).transpose(1, 2)
        V = V.view(B, L, self.num_heads, self.d_head).transpose(1, 2)

        scores = Q @ K.transpose(-2, -1)
        scores /= math.sqrt(self.d_head)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))

        attn = F.softmax(scores, dim=-1)
        out = attn @ V

        out = out.transpose(1, 2).contiguous()
        out = out.view(B, L, self.d_model)

        return self.wo(out)
```

1.2 Multi-Query Attention（MQA）

从 MHA 到 MQA MHA 中，每个 Query Head 都有自己独立的 Key 和 Value：$\boldsymbol{Q}_i$，$\boldsymbol{K}_i$，$\boldsymbol{V}_i$. 如果一共有 h 个 Query Head，也就对应 h 组 Key Head 和 Value Head。

MQA 对这一结构进行了直接修改：仍然保留多个 Query Head，但所有 Query Head 共享同一组 Key 和 Value。即 $\boldsymbol{Q}_i = \boldsymbol{X}\boldsymbol{W}_i^{\mathrm{Q}}$，而 Key 和 Value 只有一组：

$$\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^{\mathrm{K}}, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^{\mathrm{V}}.$$

多个 Query Head，共享同一组 Key 和 Value。

如果一共有 h 个 Query Head，则它们的关系可以写成

$$\begin{aligned}
\mathrm{head}_1 &= \mathrm{Attention}(\boldsymbol{Q}_1, \boldsymbol{K}, \boldsymbol{V}),\\
\mathrm{head}_2 &= \mathrm{Attention}(\boldsymbol{Q}_2, \boldsymbol{K}, \boldsymbol{V}),\\
&\vdots\\
\mathrm{head}_h &= \mathrm{Attention}(\boldsymbol{Q}_h, \boldsymbol{K}, \boldsymbol{V}).
\end{aligned}$$

MQA 的计算与 Tensor Shape 对于第 i 个 Query Head，MQA 的注意力计算为

$$\mathrm{head}_i = \mathrm{softmax}\left(\frac{\boldsymbol{Q}_i\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}.$$

这里与 MHA 相比，缩放点积注意力本身没有发生变化；改变的是 $\boldsymbol{K}$ 和 $\boldsymbol{V}$ 不再带有与 Query 一一对应的独立 Head。

所有 Head 计算结束后，仍然执行拼接和输出投影：

$$\mathrm{MQA}(\boldsymbol{X}) = \mathrm{Concat}\left(\mathrm{head}_1, \ldots, \mathrm{head}_h\right)\boldsymbol{W}^{\mathrm{O}}.$$

继续使用 $d_{\text{model}} = 512, \quad h = 8, \quad d_k = 64$ 的例子。MHA 与 MQA 最直观的区别可以通过 Tensor Shape 观察。

机制	Query	Key	Value
MHA	$[B, 8, L, 64]$	$[B, 8, L, 64]$	$[B, 8, L, 64]$
MQA	$[B, 8, L, 64]$	$[B, 1, L, 64]$	$[B, 1, L, 64]$

因此，MQA 可以从 Shape 上概括为“多 Query、单 KV”。Query 仍然保留 8 个 Head，而 Key 和 Value 都只有一个 Head。

MQA 为什么适合 LLM 推理 在自回归推理中，历史 token 的 Key 和 Value 会被缓存并在后续生成步骤中继续使用。MHA 为每一个 Query Head 保留独立的 K/V，而 MQA 将所有 Query Head 的 K/V 合并为一组共享表示。

根据这一结构，MQA 可以显著减少：

- KV Cache 的存储量；
- 自回归推理时读取 K/V 的显存带宽；
- 长上下文推理的显存开销。

因此，MQA 特别适合大语言模型的自回归推理。它把 K/V 的共享推到了最彻底的形式：无论有多少个 Query Head，都只有一组 $\boldsymbol{K}$ 和 $\boldsymbol{V}$。

Python 实现：完整 MQA 类 下面给出完整 MQA 类。与 MHA 相比，最关键的结构变化体现在 $\boldsymbol{W}^{\mathrm{K}}$ 和 $\boldsymbol{W}^{\mathrm{V}}$ 的输出维度：它们只产生一个 d_{head} 维的 Key/Value Head。代码随后使用 `unsqueeze(1)` 增加 Head 维度，使 K/V 的 Shape 变为 $[B, 1, L, d_{\text{head}}]$，并在矩阵乘法中与多个 Query Head 共享。

```python
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class MQA(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()

        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_head = d_model // num_heads

        self.wq = nn.Linear(d_model, d_model, bias=False)
        self.wk = nn.Linear(d_model, self.d_head, bias=False)
        self.wv = nn.Linear(d_model, self.d_head, bias=False)
        self.wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, mask=None):
        B, L, _ = x.shape

        Q = self.wq(x)
        K = self.wk(x)
        V = self.wv(x)
```

```
        Q = Q.view(B, L, self.num_heads, self.d_head).transpose(1, 2)

        K = K.unsqueeze(1)
        V = V.unsqueeze(1)

        scores = Q @ K.transpose(-2, -1)
        scores /= math.sqrt(self.d_head)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))

        attn = F.softmax(scores, dim=-1)
        out = attn @ V

        out = out.transpose(1, 2).contiguous()
        out = out.view(B, L, self.d_model)

        return self.wo(out)
```

1.3 Grouped-Query Attention（GQA）

GQA：MHA 与 MQA 之间的折中 MHA 和 MQA 可以看作 KV Head 数量的两个边界情况。MHA 中 $h_{\text{kv}} = h_q$, 即每个 Query Head 都有自己的 K/V；MQA 中 $h_{\text{kv}} = 1$, 即所有 Query Head 共享同一组 K/V。

GQA 位于二者之间。它保留 h_q 个 Query Head，但只使用 h_{kv} 个 Key Head 和 h_{kv} 个 Value Head，并满足 $h_{\text{kv}} < h_q$, 以及 $h_q \bmod h_{\text{kv}} = 0$. 定义每个 KV Head 对应的 Query Head 数量为 $g = \frac{h_q}{h_{\text{kv}}}$.

多个 Query Head 分组共享少量的 Key 和 Value。

对于第 i 个 Query Head，设其对应的 KV Head 为第 j 个，则

$$\boldsymbol{Q}_i = \boldsymbol{X}\boldsymbol{W}_i^{\text{Q}}, \qquad \boldsymbol{K}_j = \boldsymbol{X}\boldsymbol{W}_j^{\text{K}}, \qquad \boldsymbol{V}_j = \boldsymbol{X}\boldsymbol{W}_j^{\text{V}}.$$

Query Head 与 KV Head 的分组对应关系可以写为

$$j = \left\lfloor \frac{i-1}{g} \right\rfloor + 1.$$

GQA 的计算过程 第 i 个 Query Head 使用其所属分组对应的 $\boldsymbol{K}_j$ 和 $\boldsymbol{V}_j$ 进行注意力计算：

$$\text{head}_i = \text{softmax}\left(\frac{\boldsymbol{Q}_i \boldsymbol{K}_j^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}_j.$$

全部 Query Head 的输出仍然在最后进行拼接：

$$\text{GQA}(\boldsymbol{X}) = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_{h_q}\right)\boldsymbol{W}^{\text{O}}.$$

考虑 $h_q = 8,\quad h_{\text{kv}} = 2$. 此时 $g = \frac{8}{2} = 4$. 因此，前 4 个 Query Head 共享第一组 K/V，后 4 个 Query Head 共享第二组 K/V：

$$\begin{aligned}
\boldsymbol{Q}_1, \boldsymbol{Q}_2, \boldsymbol{Q}_3, \boldsymbol{Q}_4 &\longrightarrow \boldsymbol{K}_1, \boldsymbol{V}_1, \\
\boldsymbol{Q}_5, \boldsymbol{Q}_6, \boldsymbol{Q}_7, \boldsymbol{Q}_8 &\longrightarrow \boldsymbol{K}_2, \boldsymbol{V}_2.
\end{aligned}$$

对应的注意力计算为

$$\begin{aligned}
\text{head}_1 &= \text{Attention}(\boldsymbol{Q}_1, \boldsymbol{K}_1, \boldsymbol{V}_1), \\
\text{head}_2 &= \text{Attention}(\boldsymbol{Q}_2, \boldsymbol{K}_1, \boldsymbol{V}_1), \\
\text{head}_3 &= \text{Attention}(\boldsymbol{Q}_3, \boldsymbol{K}_1, \boldsymbol{V}_1), \\
\text{head}_4 &= \text{Attention}(\boldsymbol{Q}_4, \boldsymbol{K}_1, \boldsymbol{V}_1), \\
\text{head}_5 &= \text{Attention}(\boldsymbol{Q}_5, \boldsymbol{K}_2, \boldsymbol{V}_2), \\
\text{head}_6 &= \text{Attention}(\boldsymbol{Q}_6, \boldsymbol{K}_2, \boldsymbol{V}_2), \\
\text{head}_7 &= \text{Attention}(\boldsymbol{Q}_7, \boldsymbol{K}_2, \boldsymbol{V}_2), \\
\text{head}_8 &= \text{Attention}(\boldsymbol{Q}_8, \boldsymbol{K}_2, \boldsymbol{V}_2).
\end{aligned}$$

继续令 $d_{\text{model}} = 512,\quad h_q = 8,\quad h_{\text{kv}} = 2$, 并设 $d_k = d_v = \frac{512}{8} = 64$. 则 GQA 中的主要 Tensor Shape 为

Tensor	Shape
Q	$[B, 8, L, 64]$
K	$[B, 2, L, 64]$
V	$[B, 2, L, 64]$
单个 Head 输出	$[B, L, 64]$（忽略 Head 维度时）
所有 Head 拼接	$[B, L, 512]$

GQA 在保持多组 K/V 表示的同时，减少了 KV Head 数量，因此可以减少 KV Cache 的存储量、自回归推理读取 K/V 的显存带宽以及长上下文推理的显存开销。它可以看作 MHA 与 MQA 之间的折中方案。

从 GQA 统一理解 MHA、MQA 从 KV Head 数量 h_{kv} 的角度，可以把三种机制统一起来：

$$
\begin{aligned}
h_{\text{kv}} = h_q &\Rightarrow \text{MHA},\\
1 < h_{\text{kv}} < h_q &\Rightarrow \text{GQA},\\
h_{\text{kv}} = 1 &\Rightarrow \text{MQA}.
\end{aligned}
$$

机制	Query Head 数	KV Head 数	K/V 共享方式
MHA	h_q	h_q	每个 Query Head 使用独立 K/V
GQA	h_q	h_{kv}	一组 Query Head 共享一组 K/V
MQA	h_q	1	所有 Query Head 共享同一组 K/V

从这个统一视角看，三种机制真正发生变化的不是 Attention 的基本公式，而是 Query Head 与 KV Head 的对应方式以及 KV Head 的数量。

Python 实现：完整 GQA 类 下面给出完整 GQA 类。代码首先分别投影出 h_q 个 Query Head 和 h_{kv} 个 K/V Head，然后计算 $g = \frac{h_q}{h_{\text{kv}}}$，并使用 `repeat_interleave` 将每个 K/V Head 映射到对应的一组 Query Head，使后续批量矩阵乘法可以继续使用统一的注意力计算形式。

```python
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class GQA(nn.Module):
    def __init__(self, d_model, num_query_heads, num_kv_heads):
        super().__init__()

        assert d_model % num_query_heads == 0
        assert num_query_heads % num_kv_heads == 0

        self.d_model = d_model
        self.num_query_heads = num_query_heads
        self.num_kv_heads = num_kv_heads

        self.d_head = d_model // num_query_heads
        self.group_size = num_query_heads // num_kv_heads

        self.wq = nn.Linear(
            d_model,
            num_query_heads * self.d_head,
            bias=False,
        )
        self.wk = nn.Linear(
            d_model,
            num_kv_heads * self.d_head,
            bias=False,
        )
        self.wv = nn.Linear(
            d_model,
            num_kv_heads * self.d_head,
            bias=False,
        )
        self.wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, mask=None):
        B, L, _ = x.shape

```

```
        Q = self.wq(x)
        K = self.wk(x)
        V = self.wv(x)

        Q = Q.view(
            B, L, self.num_query_heads, self.d_head
        ).transpose(1, 2)

        K = K.view(
            B, L, self.num_kv_heads, self.d_head
        ).transpose(1, 2)

        V = V.view(
            B, L, self.num_kv_heads, self.d_head
        ).transpose(1, 2)

        K = K.repeat_interleave(self.group_size, dim=1)
        V = V.repeat_interleave(self.group_size, dim=1)

        scores = Q @ K.transpose(-2, -1)
        scores /= math.sqrt(self.d_head)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))

        attn = F.softmax(scores, dim=-1)
        out = attn @ V

        out = out.transpose(1, 2).contiguous()
        out = out.view(B, L, self.d_model)

        return self.wo(out)
```

1.4 KV Cache：自回归推理中的 K/V 缓存

自回归生成为什么会重复计算 K/V 在自回归生成中，模型一次只生成一个新 token。假设 Prompt 已经包含 $x_1, x_2, \ldots, x_t$，当前需要生成位置 $t+1$ 的 token。

如果每一步都把完整历史序列重新送入 Attention，则第 $t+1$ 步会重新计算历史位置对应的 $K_1, K_2, \ldots, K_t$ 和 $V_1, V_2, \ldots, V_t$.

但这些历史 token 的隐藏状态已经确定，在普通自回归推理中，它们对应的 Key 和 Value 也不会因为生成了一个新 token 而重新改变。因此，重复计算历史 K/V 会造成大量冗余计算。

以连续生成三个 token 为例，不使用 Cache 时可以概念化为

$$\begin{aligned}
&\text{Step 1:} \quad x_{1:t} \longrightarrow K_{1:t}, V_{1:t},\\
&\text{Step 2:} \quad x_{1:t+1} \longrightarrow K_{1:t+1}, V_{1:t+1},\\
&\text{Step 3:} \quad x_{1:t+2} \longrightarrow K_{1:t+2}, V_{1:t+2}.
\end{aligned}$$

其中 $K_{1:t}, V_{1:t}$ 在后续步骤中被反复计算。

KV Cache 的基本思想就是把已经计算过的历史 Key 和 Value 保存下来：

$$K_{\text{cache}} = K_{1:t}, \qquad V_{\text{cache}} = V_{1:t}.$$

当新 token x_{t+1} 到来时，只计算新的

$$Q_{t+1}, \qquad K_{t+1}, \qquad V_{t+1}.$$

然后把新的 K/V 追加到缓存：

$$K_{1:t+1} = [K_{1:t}; K_{t+1}],$$

$$V_{1:t+1} = [V_{1:t}; V_{t+1}].$$

当前 Query 只需要与完整历史 Key Cache 计算：

$$S_{t+1} = \frac{Q_{t+1} K_{1:t+1}^{\mathsf{T}}}{\sqrt{d_k}},$$

再利用 Attention Weight 对 Value Cache 加权：

$$O_{t+1} = \text{softmax}(S_{t+1}) V_{1:t+1}.$$

KV Cache 缓存的是历史 Key 和 Value，而不是历史 Query。增量 Decode 时，每个旧位置的 Query 已经完成了自己的输出计算，后续步骤真正需要重复读取的是历史 K/V。

KV Cache 的显存占用与 MHA、GQA、MQA KV Cache 用额外显存换取历史 K/V 的重复计算减少。

设：B = Batch Size, L = 当前缓存序列长度, h_{kv} = KV Head 数, d_{head} = 单个 KV Head 维度, 每个缓存元素占 b bytes。

单层 Key Cache 的理论元素量为 $BLh_{\text{kv}}d_{\text{head}}$.

Value Cache 相同，因此单层 KV Cache 近似为

$$M_{\text{KV,layer}} \approx 2BLh_{\text{kv}}d_{\text{head}}b.$$

如果模型共有 N 层，则

$$M_{\text{KV,total}} \approx 2NBLh_{\text{kv}}d_{\text{head}}b.$$

这说明 KV Cache 的占用近似随模型层数 N、Batch Size B、上下文长度 L、KV Head 数量 h_{kv} 和单个 Head 的维度 d_{head} 线性增长。

其中 $L\uparrow$ 会直接导致长上下文推理的 KV Cache 显存持续增加。

MQA 与 GQA 的重要推理优势正来自减少 h_{kv}。

机制	KV Head 数	单层 Cache 近似规模	主要特点
MHA	$h_{\text{kv}} = h_q$	$2BLh_qd_{\text{head}}b$	每个 Query Head 独立 K/V
GQA	$1 < h_{\text{kv}} < h_q$	$2BLh_{\text{kv}}d_{\text{head}}b$	Query Head 分组共享 K/V
MQA	$h_{\text{kv}} = 1$	$2BLd_{\text{head}}b$	所有 Query Head 共享 K/V

在相同 B, L, d_{head} 下，如果 MHA 使用 $h_q = 32$ 个 KV Head，而 GQA 使用 $h_{\text{kv}} = 8$，则仅从 K/V Head 数量看，理论 KV Cache 约缩小为原来的

$$\frac{8}{32} = \frac{1}{4}.$$

MQA 使用 $h_{\text{kv}} = 1$ 时，对应比例进一步变为 $\frac{1}{32}$.

因此，可以从 KV Cache 角度理解

这条演进路线：

减少 KV Head 数量通常会降低 KV Cache 的存储与读取开销。

下一节的 MLA 会进一步改变问题本身：它不只减少 KV Head 数量，而是尝试压缩需要缓存的 KV Representation。

KV Cache 的核心权衡是 Compute–Memory Trade-off：通过缓存历史 K/V 减少重复计算，但缓存本身会随 Batch Size、Context Length、Layer 数和 KV Head 数增长。

Python 实现：简单 KV Cache 简化的 Multi-Head Self-Attention 实现提供两条路径：

- `forward_full()`：对完整序列使用 Causal Mask；
- `forward_step()`：每次只输入一个新 token，并追加 KV Cache。

其中，`forward_full()` 用于展示完整序列的 Causal Attention，`forward_step()` 用于展示单 Token Decode 时如何维护和追加 KV Cache。

```python
class SimpleKVCacheAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()

        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_head = d_model // num_heads

        self.wq = nn.Linear(d_model, d_model, bias=False)
        self.wk = nn.Linear(d_model, d_model, bias=False)
        self.wv = nn.Linear(d_model, d_model, bias=False)
        self.wo = nn.Linear(d_model, d_model, bias=False)

    def _split_heads(self, x):
        B, L, _ = x.shape
        x = x.view(
            B, L, self.num_heads, self.d_head
        )
        return x.transpose(1, 2)

    def _merge_heads(self, x):
        B, H, L, D = x.shape
        x = x.transpose(1, 2).contiguous()
```

```
        return x.view(B, L, H * D)

    def forward_full(self, x):
        B, L, _ = x.shape

        Q = self._split_heads(self.wq(x))
        K = self._split_heads(self.wk(x))
        V = self._split_heads(self.wv(x))

        scores = Q @ K.transpose(-2, -1)
        scores /= math.sqrt(self.d_head)

        mask = torch.tril(
            torch.ones(
                L, L,
                dtype=torch.bool,
                device=x.device,
            )
        )
        scores = scores.masked_fill(
            ~mask,
            float("-inf"),
        )

        attn = F.softmax(scores, dim=-1)
        out = attn @ V
        out = self._merge_heads(out)

        return self.wo(out)

    def forward_step(self, x_t, cache=None):
        # x_t: [B, 1, d_model]
        Q = self._split_heads(self.wq(x_t))
        K_new = self._split_heads(self.wk(x_t))
        V_new = self._split_heads(self.wv(x_t))

        if cache is None:
            K_cache = K_new
            V_cache = V_new
        else:
```

```
            K_cache = torch.cat(
                [cache["k"], K_new],
                dim=2,
            )
            V_cache = torch.cat(
                [cache["v"], V_new],
                dim=2,
            )

        scores = Q @ K_cache.transpose(-2, -1)
        scores /= math.sqrt(self.d_head)

        # Cache 中只包含历史和当前 token,
        # 因此单步 Decode 不会看到未来位置。
        attn = F.softmax(scores, dim=-1)
        out = attn @ V_cache
        out = self._merge_heads(out)
        out = self.wo(out)

        new_cache = {
            "k": K_cache,
            "v": V_cache,
        }

        return out, new_cache
```

在这个实现中，Cache Shape 可以写成 $[B, H, L, D]$。

每生成一个新 token，第三个维度从 t 增长为 $t + 1$.

生产级推理框架还需要处理连续批处理、不同请求的序列长度、Cache Page / Block 管理、Prefix Reuse、量化以及设备间调度，但最基础的计算逻辑仍然是

Cache Historical K/V + Append New K/V + Current Q attends to all cached K/V.

1.5 Multi-head Latent Attention（MLA）

从 MHA、MQA、GQA 到 MLA 前面的 MHA、MQA 和 GQA 可以主要从 KV Head 数量理解。MHA 中 $h_{\text{kv}} = h_q$, 每个 Query Head 都拥有独立的 K/V；MQA 中 $h_{\text{kv}} = 1$, 所有 Query Head 共享同一组 K/V；GQA 则使用 $1 < h_{\text{kv}} < h_q$ 作为二者之间的折中。

这三种机制的共同思路都是改变 KV Head 的数量或共享关系。Multi-head Latent Attention（MLA）进一步提出另一个问题：

除了减少 KV Head 数量之外，能否直接压缩 Key/Value 的表示本身，只缓存一个更低维的 Latent Representation？

设位置 t 的隐藏状态为 $\boldsymbol{h}_t \in \mathbb{R}^{d_{\text{model}}}$. MLA 首先通过 Down Projection 将隐藏状态压缩为低维 KV Latent：

$$\boldsymbol{c}_t^{\text{KV}} = \boldsymbol{W}^{\text{DKV}} \boldsymbol{h}_t,$$

其中

$$\boldsymbol{c}_t^{\text{KV}} \in \mathbb{R}^{d_c}, \qquad d_c \ll h_q d_{\text{head}}.$$

再通过 Up Projection 得到用于注意力计算的 Content Key 与 Value：

$$\boldsymbol{k}_t^{\text{C}} = \boldsymbol{W}^{\text{UK}} \boldsymbol{c}_t^{\text{KV}}, \qquad \boldsymbol{v}_t = \boldsymbol{W}^{\text{UV}} \boldsymbol{c}_t^{\text{KV}}.$$

因此，MLA 的核心数据流可以概括为

$$\boldsymbol{h}_t \longrightarrow \boldsymbol{c}_t^{\text{KV}} \longrightarrow (\boldsymbol{k}_t^{\text{C}}, \boldsymbol{v}_t).$$

Query 侧也可以使用低秩投影。先压缩为

$$\boldsymbol{c}_t^{\text{Q}} = \boldsymbol{W}^{\text{DQ}} \boldsymbol{h}_t,$$

再恢复多头 Query：

$$\boldsymbol{q}_t^{\text{C}} = \boldsymbol{W}^{\text{UQ}} \boldsymbol{c}_t^{\text{Q}}.$$

这里的 Down Projection 与 Up Projection 组成低秩表示路径。与 MQA / GQA 主要减少 KV Head 数量不同，MLA 的核心变化发生在 KV Representation 本身：

MLA 的计算与 Tensor Shape 为了说明 Tensor Shape，考虑

$$d_{\text{model}} = 512, \qquad h_q = 8, \qquad d_{\text{head}} = 64,$$

并设 Query Latent 与 KV Latent 的维度均为 $d_q = d_c = 128$。

输入为 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times 512}$. Query 先被压缩为

$$\boldsymbol{C}^{\mathrm{Q}} = \boldsymbol{X}(\boldsymbol{W}^{\mathrm{DQ}})^{\mathsf{T}} \in \mathbb{R}^{B\times L\times 128},$$

再恢复为多头 Query：

$$\boldsymbol{Q}^{\mathrm{C}} \in \mathbb{R}^{B\times 8\times L\times 64}.$$

KV 侧先得到

$$\boldsymbol{C}^{\mathrm{KV}} = \boldsymbol{X}(\boldsymbol{W}^{\mathrm{DKV}})^{\mathsf{T}} \in \mathbb{R}^{B\times L\times 128},$$

训练时可以通过 Up Projection 得到多头 Content Key 与 Value：

$$\boldsymbol{K}^{\mathrm{C}}, \boldsymbol{V} \in \mathbb{R}^{B\times 8\times L\times 64}.$$

对应的主要 Tensor Shape 如表所示。

Tensor	Shape
输入 $\boldsymbol{X}$	$[B, L, 512]$
Query Latent $\boldsymbol{C}^{\mathrm{Q}}$	$[B, L, 128]$
KV Latent $\boldsymbol{C}^{\mathrm{KV}}$	$[B, L, 128]$
恢复后的 $\boldsymbol{Q}^{\mathrm{C}}$	$[B, 8, L, 64]$
恢复后的 $\boldsymbol{K}^{\mathrm{C}}$	$[B, 8, L, 64]$
恢复后的 $\boldsymbol{V}$	$[B, 8, L, 64]$
Attention Score	$[B, 8, L, L]$
输出	$[B, L, 512]$

如果只观察训练阶段的普通矩阵计算，恢复后的 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 仍然可以进入标准缩放点积注意力：

$$\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}.$$

MLA 真正重要的区别体现在推理阶段：历史 token 不必以完整多头 K/V 的形式进入 KV Cache，而可以优先保存低维的 $\boldsymbol{C}^{\mathrm{KV}}$。

DeepSeek-style MLA 还会将 Content 与 RoPE 位置部分进行解耦。可以概念化地写成

$$\boldsymbol{q}_t = [\boldsymbol{q}_t^{\mathrm{C}}; \boldsymbol{q}_t^{\mathrm{R}}], \qquad \boldsymbol{k}_t = [\boldsymbol{k}_t^{\mathrm{C}}; \boldsymbol{k}_t^{\mathrm{R}}],$$

其中上标 C 表示 Content Part，上标 R 表示携带 RoPE 位置信息的部分。于是 Attention Score 可以分解为

$$\boldsymbol{q}_t^{\mathsf{T}} \boldsymbol{k}_j = (\boldsymbol{q}_t^{\mathrm{C}})^{\mathsf{T}} \boldsymbol{k}_j^{\mathrm{C}} + (\boldsymbol{q}_t^{\mathrm{R}})^{\mathsf{T}} \boldsymbol{k}_j^{\mathrm{R}}.$$

这种 Decoupled RoPE 使低秩 Content 表示与位置相关表示能够分开组织。位置编码的完整数学原理在位置编码章节中单独讨论，本节只保留它与 MLA 的结构关系。

MLA 为什么适合 LLM 推理 在自回归推理中，历史 token 的 K/V 会在后续生成步骤中被反复使用，因此需要进入 KV Cache。若忽略 batch、layer 和数据类型，只观察每个 token 每层需要缓存的 K/V 元素数量，则 MHA 大致与 $2h_q d_{\text{head}}$ 成正比，GQA 大致与 $2h_{\text{kv}} d_{\text{head}}$ 成正比，MQA 大致与 $2d_{\text{head}}$ 成正比。

MLA 则不再把“缓存多少个完整 KV Head”作为唯一优化方向，而是缓存压缩后的 KV Latent。概念上可以写成

$$\text{MLA Cache} \propto d_c + d_R,$$

其中 d_R 表示需要单独缓存的 RoPE Key 分量维度。

MLA 推理优化中的另一个关键思想是 Matrix Absorption。由

$$\boldsymbol{k}_j^{\mathrm{C}} = \boldsymbol{W}^{\mathrm{UK}} \boldsymbol{c}_j^{\mathrm{KV}}$$

可得

$$(\boldsymbol{q}_t^{\mathrm{C}})^{\mathsf{T}} \boldsymbol{k}_j^{\mathrm{C}} = (\boldsymbol{q}_t^{\mathrm{C}})^{\mathsf{T}} \boldsymbol{W}^{\mathrm{UK}} \boldsymbol{c}_j^{\mathrm{KV}}.$$

利用矩阵结合律，可以改写为

$$(\boldsymbol{q}_t^{\mathrm{C}})^{\mathsf{T}} \boldsymbol{W}^{\mathrm{UK}} \boldsymbol{c}_j^{\mathrm{KV}} = \left((\boldsymbol{W}^{\mathrm{UK}})^{\mathsf{T}} \boldsymbol{q}_t^{\mathrm{C}}\right)^{\mathsf{T}} \boldsymbol{c}_j^{\mathrm{KV}}.$$

这意味着在合适的推理实现中，可以先把 $\boldsymbol{W}^{\mathrm{UK}}$ 吸收到 Query 一侧，再直接让变换后的 Query 与缓存的 KV Latent 计算，而不必在每一个 Decode Step 显式恢复所有历史 token 的完整 Content Key。

Value 侧也具有类似的线性重组关系：

$$\sum_j a_{tj}\boldsymbol{v}_j = \sum_j a_{tj}\boldsymbol{W}^{\mathrm{UV}}\boldsymbol{c}_j^{\mathrm{KV}} = \boldsymbol{W}^{\mathrm{UV}}\left(\sum_j a_{tj}\boldsymbol{c}_j^{\mathrm{KV}}\right).$$

因此，MLA 的优势不仅来自“低维缓存”，还来自线性投影可以与注意力计算重新组合，从而减少完整 K/V 的显式物化与内存访问。

MQA / GQA 主要通过减少 KV Head 数量降低 KV Cache；MLA 则进一步通过低秩 Latent 压缩 KV Representation，并利用 Matrix Absorption 优化 Decode 阶段的内存访问。

从 MLA 统一理解 MHA、MQA、GQA MHA、MQA、GQA 和 MLA 都服务于多头注意力，但它们对 KV 的组织方式不同。

机制	Query 结构	KV 组织方式	KV Cache 优化思路
MHA	多个 Query Head	每个 Query Head 使用独立 K/V	不共享完整 K/V
MQA	多个 Query Head	所有 Query Head 共享一组 K/V	最大化 KV Head 共享
GQA	多个 Query Head	一组 Query Head 共享一组 K/V	在多组 KV 与缓存开销之间折中
MLA	多头 Query，可带低秩 Query 路径	使用压缩 KV Latent 表示，并可恢复多头 Content K/V	直接压缩 KV Representation

因此，前三种机制可以继续用 KV Head 数量 h_{kv} 统一描述，而 MLA 不适合只用一个 KV Head 数量来概括。更合理的演进视角是

对应

Python 实现：完整 MLA 类 下面的简化 MLA 实现采用与 MHA、MQA、GQA 一致的接口，重点展示 Query / KV 的 Low-Rank Projection 与标准 Attention；Decoupled RoPE 可与位置编码章节中的 RoPE 组合，不在类内部重复实现。

代码首先计算 Query Latent 与 KV Latent，再通过 Up Projection 恢复多头 Query、Key、Value。训练式前向中为了直观展示 Attention Shape，会显式恢复 Key/Value；

真正的高效增量推理可以缓存 kv_latent，并结合前面的 Matrix Absorption 避免反复恢复全部历史 Key/Value。

```
class MLA(nn.Module):
    def __init__(self, d_model, num_heads, q_rank, kv_rank):
        super().__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_head = d_model // num_heads

        self.q_down = nn.Linear(d_model, q_rank, bias=False)
        self.q_up = nn.Linear(q_rank, d_model, bias=False)

        self.kv_down = nn.Linear(d_model, kv_rank, bias=False)
        self.k_up = nn.Linear(kv_rank, d_model, bias=False)
        self.v_up = nn.Linear(kv_rank, d_model, bias=False)

        self.wo = nn.Linear(d_model, d_model, bias=False)

    def compress_kv(self, x):
        return self.kv_down(x)

    def forward(self, x, mask=None):
        B, L, _ = x.shape

        q_latent = self.q_down(x)
        kv_latent = self.kv_down(x)

        Q = self.q_up(q_latent)
        K = self.k_up(kv_latent)
        V = self.v_up(kv_latent)

        Q = Q.view(B, L, self.num_heads, self.d_head).transpose(1, 2)
        K = K.view(B, L, self.num_heads, self.d_head).transpose(1, 2)
        V = V.view(B, L, self.num_heads, self.d_head).transpose(1, 2)

        scores = Q @ K.transpose(-2, -1)
        scores = scores / math.sqrt(self.d_head)
```

```
38
39          if mask is not None:
40              scores = scores.masked_fill(mask == 0, float("-inf"))
41
42          attn = F.softmax(scores, dim=-1)
43          out = attn @ V
44
45          out = out.transpose(1, 2).contiguous()
46          out = out.view(B, L, self.d_model)
47
48          return self.wo(out)
```

与前面三种实现一样，该类仍保持

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{model}}]$$

的输入输出接口。

同时可以单独调用 `compress_kv(x)` 观察 MLA 的压缩表示：

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_c].$$

1.6 QKV 注意力机制中的关键注意事项

Scaled Dot-Product Attention MHA、MQA、GQA 和 MLA 的 Attention 主体最终都建立在缩放点积注意力之上：

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}.$$

其中的关键问题是：为什么 $\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}$ 之后需要除以 $\sqrt{d_k}$?

考虑单个 Query 向量 $\boldsymbol{q}$ 和 Key 向量 $\boldsymbol{k}$ 的点积：

$$\boldsymbol{q} \cdot \boldsymbol{k} = \sum_{i=1}^{d_k} q_i k_i.$$

假设各维近似独立，并且均值为 0、方差为 1，则有 $\text{Var}(\boldsymbol{q} \cdot \boldsymbol{k}) \approx d_k$, 从而 $\text{Std}(\boldsymbol{q} \cdot \boldsymbol{k}) \approx \sqrt{d_k}$. 因此，当 d_k 增大时，点积的数值幅度也会随之增大。

较大的 Attention Score 会让 Softmax 变得非常尖锐。例如

$$[10, 20, 30] \xrightarrow{\text{softmax}} [\approx 0, \approx 0, \approx 1].$$

当 Softmax 输出接近 0 或 1 时，梯度会变小。Softmax 导数中包含 $p_i(1-p_i)$。当 $p_i \to 0$ 或 $p_i \to 1$ 时，$p_i(1-p_i) \to 0$。因此，如果不控制点积的尺度，训练会更容易进入梯度较小的区域。

Transformer 使用 $\frac{\boldsymbol{QK}^\top}{\sqrt{d_k}}$ 进行缩放。因为原始点积的标准差近似为 $\sqrt{d_k}$，缩放之后有

$$\text{Std}\left(\frac{\boldsymbol{QK}^\top}{\sqrt{d_k}}\right) \approx 1.$$

这样可以稳定 Attention Score 的尺度，避免 Softmax 过早进入饱和区域。

例如，当 $d_k = 64$ 时，有 $\sqrt{d_k} = 8$. 如果原始 Attention Score 为 $[8, 16, 24]$, 直接 Softmax 会非常接近 $[0, 0, 1]$. 缩放后得到 $\frac{[8,16,24]}{8} = [1, 2, 3]$, 再经过 Softmax：

$$[1, 2, 3] \xrightarrow{\text{softmax}} [0.09, 0.24, 0.67].$$

缩放后的分布更加平滑，梯度也更加稳定。

Mask、Tensor Shape 与实现细节 在实际代码中，注意力计算通常按照以下顺序进行：

$$QK^\top \longrightarrow \text{Scaling} \longrightarrow \text{Mask} \longrightarrow \text{softmax} \longrightarrow \times V.$$

如果存在 Mask，应当在 Softmax 之前作用于 Attention Score。代码中使用

```
scores = scores.masked_fill(mask == 0, float("-inf"))
```

把不允许参与注意力计算的位置替换为负无穷。经过 Softmax 后，这些位置的权重趋近于 0。

对于自回归注意力，可以使用下三角矩阵构造 causal mask：

```
def create_causal_mask(seq_len):
    return torch.tril(
        torch.ones(seq_len, seq_len, dtype=torch.bool)
    )
```

这样可以使当前位置只能使用当前及之前位置的信息，而不能使用其后的 token。

另一个需要持续跟踪的问题是 Tensor Shape。对于标准多头形式，主要 Shape 变化为

$$[B, L, d_{\text{model}}] \longrightarrow [B, H, L, d_{\text{head}}] \longrightarrow [B, H, L, L]$$
$$\longrightarrow [B, H, L, d_{\text{head}}] \longrightarrow [B, L, d_{\text{model}}].$$

其中，$[B, H, L, L]$ 是 Attention Score 的 Shape，两个 L 分别对应 Query 位置与 Key 位置。

MHA、GQA、MQA 在计算前的 K/V Head 维度分别为

$$\begin{aligned}
\text{MHA:} \quad & \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{B \times h_q \times L \times d_k}, \\
\text{GQA:} \quad & \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{B \times h_{\text{kv}} \times L \times d_k}, \\
\text{MQA:} \quad & \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{B \times 1 \times L \times d_k}.
\end{aligned}$$

MLA 则多出一个更值得持续跟踪的 Latent Shape：

$$\boldsymbol{C}^{\text{KV}} \in \mathbb{R}^{B \times L \times d_c}.$$

训练式计算中可以从 $\boldsymbol{C}^{\text{KV}}$ 恢复多头 K/V；增量推理中则可以优先缓存 $\boldsymbol{C}^{\text{KV}}$ 与必要的位置分量，而不需要把所有历史 token 都以完整多头 K/V 的形式保存。

在 Head 合并阶段，代码通常先执行

```
out = out.transpose(1, 2).contiguous()
out = out.view(B, L, self.d_model)
```

其维度变化为

$$[B, H, L, D] \longrightarrow [B, L, H, D] \longrightarrow [B, L, H \times D].$$

`transpose` 用来交换序列维与 Head 维，`contiguous` 使转置后的 Tensor 在内存布局上连续，随后再使用 `view` 合并 Head 维和单 Head 维度。

四种注意力机制的统一计算视角 MHA、GQA、MQA 和 MLA 的 Attention 主体都可以继续理解为

$$\text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\text{T}}}{\sqrt{d_k}}\right)\boldsymbol{V},$$

但四者优化 K/V 的方式并不完全相同。

MHA、GQA 和 MQA 主要改变 KV Head 的数量以及 Query Head 与 KV Head 的对应关系：

$$\begin{aligned}\text{MHA:}\quad & \boldsymbol{Q}_i, \boldsymbol{K}_i, \boldsymbol{V}_i,\\ \text{GQA:}\quad & \boldsymbol{Q}_i, \boldsymbol{K}_j, \boldsymbol{V}_j,\\ \text{MQA:}\quad & \boldsymbol{Q}_i, \boldsymbol{K}, \boldsymbol{V}.\end{aligned}$$

MLA 则先构造

$$\boldsymbol{c}_t^{\text{KV}} = \boldsymbol{W}^{\text{DKV}} \boldsymbol{h}_t,$$

再从压缩表示得到 Attention 所需的 Content K/V。因此它进一步改变了 KV 的表示方式。

机制	KV 数量 / 表示	Query 与 KV 的关系	核心优化方向
MHA	h_q 组 K/V	每个 Query Head 使用独立 K/V	保留完整多头 K/V
MQA	1 组 K/V	所有 Query Head 共享 K/V	最大化 KV Head 共享
GQA	h_{kv} 组 K/V	Query Head 分组共享 K/V	Head Sharing 折中
MLA	压缩 KV Latent	从 Latent 构造 / 等价使用多头 K/V	Low-Rank KV Compression

因此可以把注意力推理优化理解为两步：

Python 实现：完整 Attention 测试代码 将前面四个完整类放在同一个 Python 文件中后，可以使用下面的测试代码统一验证 MHA、MQA、GQA 和 MLA。四种机制内部的 K/V 组织方式不同，但它们最终都保持 $[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{model}}]$ 的输入输出接口。

MLA 还额外验证压缩后的 KV Latent：

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_c].$$

```
def create_causal_mask(seq_len):
    return torch.tril(
        torch.ones(seq_len, seq_len, dtype=torch.bool)
    )

def test_attention():
    torch.manual_seed(42)

    B = 2
    L = 8
    d_model = 512

    x = torch.randn(B, L, d_model)
    mask = create_causal_mask(L)

    models = {
        "MHA": MHA(
            d_model=512,
            num_heads=8,
        ),
        "MQA": MQA(
            d_model=512,
            num_heads=8,
        ),
        "GQA": GQA(
            d_model=512,
            num_query_heads=8,
            num_kv_heads=2,
        ),
        "MLA": MLA(
            d_model=512,
            num_heads=8,
            q_rank=128,
            kv_rank=128,
        ),
    }

    for name, model in models.items():
        output = model(x, mask)
```

```
40
41          print(f"{name}")
42          print(f"input  shape: {x.shape}")
43          print(f"output shape: {output.shape}")
44
45          if name == "MLA":
46              kv_latent = model.compress_kv(x)
47              print(f"KV latent:   {kv_latent.shape}")
48              assert kv_latent.shape == (B, L, 128)
49
50          print()
51          assert output.shape == x.shape
52
53      print("所有测试通过")
54
55  if __name__ == "__main__":
56      test_attention()
```

第 2 章

Transformer 中的位置编码

Transformer 的 Self-Attention 可以直接建立任意两个 token 之间的依赖关系，但它本身并不包含递归或卷积结构，因此不会天然感知 token 在序列中的先后顺序。若只给模型输入 token embedding，而不引入额外的位置信息，则注意力机制更关注“有哪些 token”，却难以区分“这些 token 按什么顺序出现”。

因此，Transformer 需要显式地向输入或 Attention 计算中注入位置信息。位置编码的发展大致经历了以下路线：

位置编码方法包括绝对位置、相对位置、RoPE / ALiBi，以及面向长上下文的扩展方法。

为了统一符号，设输入序列为 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$，其中 B 表示 batch size，L 表示序列长度，d_{model} 表示隐藏维度。对于多头注意力，第 h 个 Head 的 Query、Key 可以表示为 $\boldsymbol{Q}, \boldsymbol{K} \in \mathbb{R}^{B\times H\times L\times d_{\text{head}}}$.

2.1 绝对位置编码：Sinusoidal 与 Learned Position Embedding

主要思想 绝对位置编码直接回答一个问题：

当前 token 位于序列中的第几个位置。

其基本做法是为位置 $0, 1, 2, \ldots, L-1$ 分别构造一个位置向量，再将位置向量与 token embedding 结合。最常见的两种方法是：

- 固定的正弦余弦位置编码（Sinusoidal Position Encoding）；
- 可学习绝对位置嵌入（Learned Positional Embedding）。

两者在输入端的形式都可以写成 $\boldsymbol{H} = \boldsymbol{X} + \boldsymbol{P}$，其中 $\boldsymbol{P}$ 是与序列位置对应的位置表示。区别在于：Sinusoidal 的 $\boldsymbol{P}$ 由确定性函数计算得到，而 Learned Position Embedding 的 $\boldsymbol{P}$ 是模型参数。

Sinusoidal Position Encoding 的数学表示 原始 Transformer 使用固定的正弦余弦位置编码。对于位置 pos 和隐藏维度索引 i，定义

$$\mathrm{PE}(\mathrm{pos}, 2i) = \sin\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right),$$

$$\mathrm{PE}(\mathrm{pos}, 2i+1) = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right).$$

也可以定义不同维度对应的角频率 $\theta_i = 10000^{-2i/d_{\mathrm{model}}}$, 于是上式可以写成

$$\mathrm{PE}(\mathrm{pos}, 2i) = \sin(\mathrm{pos}\,\theta_i), \qquad \mathrm{PE}(\mathrm{pos}, 2i+1) = \cos(\mathrm{pos}\,\theta_i).$$

最终输入为

$$\boldsymbol{h}_{\mathrm{pos}} = \boldsymbol{x}_{\mathrm{pos}} + \mathbf{PE}_{\mathrm{pos}}.$$

这种设计让不同维度具有不同频率：低维位置变化较快，高维位置变化较慢，因此一个位置可以由多组不同尺度的周期信号共同表示。

Sinusoidal 还有一个重要性质。根据三角函数公式，位置 $\mathrm{pos}+k$ 的编码可以由位置 pos 的正弦和余弦值通过线性组合得到。例如

$$\sin((\mathrm{pos}+k)\theta) = \sin(\mathrm{pos}\,\theta)\cos(k\theta) + \cos(\mathrm{pos}\,\theta)\sin(k\theta).$$

因此，固定的正弦余弦编码不仅提供绝对位置，也使模型有机会从这些表示中学习相对位移关系。

Learned Position Embedding 的数学表示 可学习绝对位置嵌入直接定义一个可训练参数矩阵 $\boldsymbol{P} \in \mathbb{R}^{L_{\max} \times d_{\mathrm{model}}}$, 其中第 i 行 $\boldsymbol{P}_i$ 表示位置 i 的 embedding。对位置 i 的 token，有 $\boldsymbol{h}_i = \boldsymbol{x}_i + \boldsymbol{P}_i$.

这种方法与普通 token embedding 的思路完全一致，只是索引从 token id 变为 position id。它的优势是位置表示可以由数据直接学习；缺点是最大位置通常受参数矩阵 $L_{\max}$ 限制。如果模型只训练并定义到位置 $L_{\max}-1$，则更长位置没有天然对应的参数。

从结构上看，两种绝对位置编码都将 Token Embedding 与 Position Embedding 相加，主要改变 Transformer 的输入，而不直接修改 Attention Score 的计算公式。

Python 实现：Sinusoidal 与 Learned Position Embedding 下面给出两种绝对位置编码的 PyTorch 实现。代码输出 Shape 与输入一致，均保持 $[B, L, d_{\mathrm{model}}] \longrightarrow [B, L, d_{\mathrm{model}}]$.

```python
import math
import torch
import torch.nn as nn

class SinusoidalPositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=4096):
        super().__init__()

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(
            max_len, dtype=torch.float
        ).unsqueeze(1)

        div_term = torch.exp(
            torch.arange(0, d_model, 2).float()
            * (-math.log(10000.0) / d_model)
        )

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.register_buffer("pe", pe.unsqueeze(0))

    def forward(self, x):
        # x: [B, L, d_model]
        L = x.size(1)
        return x + self.pe[:, :L]

class LearnedPositionalEmbedding(nn.Module):
    def __init__(self, d_model, max_len=4096):
        super().__init__()
        self.position_embedding = nn.Embedding(
            max_len, d_model
        )

    def forward(self, x):
        # x: [B, L, d_model]
        L = x.size(1)
        position_ids = torch.arange(
```

```
            L, device=x.device
        ).unsqueeze(0)

        position_embedding = self.position_embedding(
            position_ids
        )
        return x + position_embedding
```

Sinusoidal Position Encoding 是原始 Transformer 的代表方案；可学习绝对位置嵌入则被 GPT-1、GPT-2、BERT、OPT 等模型采用。二者结构简单，但都以“位置编号本身”为主要建模对象，因此后续研究逐渐转向 token 之间的相对距离。

2.2 相对位置编码：Shaw、Transformer-XL、T5 与 DeBERTa

主要思想 对于 Attention 而言，很多语义关系并不取决于某个 token 是“第 100 个 token”，而更取决于两个 token 之间相隔多少位置。因此，相对位置编码把重点从绝对位置 i 转向两个位置之间的距离 $r_{ij} = i - j$.

例如，若 Query 位于位置 $i = 10$，Key 位于位置 $j = 8$，则 $r_{ij} = 2$. 这表示 Key 位于 Query 前两个位置。

不同相对位置方法的区别主要在于：相对距离 $i - j$ 最终以什么形式进入 Attention。Shaw 方法把相对位置表示成向量；Transformer-XL 对内容和位置项进一步分解；T5 将相对位置压缩成 Bucket 并直接作为 Attention Bias；DeBERTa 则显式区分 Content-to-Content、Content-to-Position 和 Position-to-Content。

Shaw Relative Position Encoding 标准缩放点积注意力的分数为

$$e_{ij} = \frac{\boldsymbol{q}_i^{\top} \boldsymbol{k}_j}{\sqrt{d_k}}.$$

Shaw Relative Position Encoding 为相对距离 $i - j$ 学习一个向量 $\boldsymbol{a}_{ij}^{\mathrm{K}}$，并将其加入 Key 侧：

$$e_{ij} = \frac{\boldsymbol{q}_i^{\top} (\boldsymbol{k}_j + \boldsymbol{a}_{ij}^{\mathrm{K}})}{\sqrt{d_k}}, \qquad \boldsymbol{a}_{ij}^{\mathrm{K}} = \boldsymbol{r}_{i-j}.$$

展开后得到

$$e_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j + \boldsymbol{q}_i^\top \boldsymbol{r}_{i-j}}{\sqrt{d_k}}.$$

其中 $\boldsymbol{q}_i^\top \boldsymbol{k}_j$ 表示 token 内容之间的相关性，而 $\boldsymbol{q}_i^\top \boldsymbol{r}_{i-j}$ 表示 Query 内容与相对位置之间的相关性。

实际实现通常会限制最大相对距离 K，将 $i-j$ 裁剪到 $[-K, K]$，从而只需要学习 $2K+1$ 个相对位置向量。

Transformer-XL 的相对位置分解 Transformer-XL 对相对位置进一步进行分解。一个常见的简化表示为

$$A_{ij} = \boldsymbol{q}_i^\top \boldsymbol{k}_j + \boldsymbol{q}_i^\top \boldsymbol{r}_{i-j} + \boldsymbol{u}^\top \boldsymbol{k}_j + \boldsymbol{v}^\top \boldsymbol{r}_{i-j},$$

其中 $\boldsymbol{u}$ 和 $\boldsymbol{v}$ 是可学习的全局偏置向量。

四个部分分别可以理解为：

- $\boldsymbol{q}_i^\top \boldsymbol{k}_j$：Content-to-Content；
- $\boldsymbol{q}_i^\top \boldsymbol{r}_{i-j}$：Content-to-Position；
- $\boldsymbol{u}^\top \boldsymbol{k}_j$：全局内容偏置；
- $\boldsymbol{v}^\top \boldsymbol{r}_{i-j}$：全局位置偏置。

Transformer-XL 的重要意义在于，它不再简单地把位置向量加入输入，而是把位置关系直接纳入 Attention 的打分结构。这种思路也为后续更复杂的相对位置模型提供了基础。

T5 Relative Position Bias T5 进一步简化了相对位置表示。它不再为每个相对位置构造完整向量，而是直接向 Attention Score 中加入一个标量 Bias：

$$A_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}} + b(i-j).$$

如果每一个相对距离都使用独立参数，长序列会需要大量位置参数。T5 因此使用 Relative Position Bucket：

$$i - j \longrightarrow \text{bucket}(i-j).$$

最终写成

$$A_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}} + b_{\text{bucket}(i-j)}.$$

Bucket 通常对短距离进行细粒度区分，对长距离进行近似对数级压缩。直观上，距离 1 与距离 2 可能代表明显不同的局部关系，而距离 500 与距离 501 往往没有必要使用两个完全独立的位置参数。

DeBERTa 的解耦位置表示 DeBERTa 将内容向量和位置向量分别编码，并显式计算不同类型的交互。其 Attention Score 可以概括为

$$A_{ij} = \boldsymbol{q}_i^{\mathrm{C}}(\boldsymbol{k}_j^{\mathrm{C}})^{\mathsf{T}} + \boldsymbol{q}_i^{\mathrm{C}}(\boldsymbol{k}_{i-j}^{\mathrm{P}})^{\mathsf{T}} + \boldsymbol{q}_{i-j}^{\mathrm{P}}(\boldsymbol{k}_j^{\mathrm{C}})^{\mathsf{T}}.$$

其中：

- $\boldsymbol{q}_i^{\mathrm{C}}(\boldsymbol{k}_j^{\mathrm{C}})^{\mathsf{T}}$ 为 Content-to-Content；
- $\boldsymbol{q}_i^{\mathrm{C}}(\boldsymbol{k}_{i-j}^{\mathrm{P}})^{\mathsf{T}}$ 为 Content-to-Position；
- $\boldsymbol{q}_{i-j}^{\mathrm{P}}(\boldsymbol{k}_j^{\mathrm{C}})^{\mathsf{T}}$ 为 Position-to-Content。

这种方法的核心是：

token 的内容表示与位置表示不必在输入端直接相加，而可以在 Attention 中以不同交互项分别建模。

Python 实现：相对位置向量与 T5 Relative Bias 下面首先实现 Shaw 风格的相对位置向量。对于长度为 L 的序列，输出相对位置 embedding 的 Shape 为 $[L, L, d_{\text{head}}]$.

```python
import torch
import torch.nn as nn

class RelativePositionEmbedding(nn.Module):
    def __init__(self, max_relative_position, head_dim):
        super().__init__()
        self.max_relative_position = max_relative_position
        self.embedding = nn.Embedding(
            2 * max_relative_position + 1,
            head_dim,
        )

    def forward(self, seq_len, device=None):
        position = torch.arange(seq_len, device=device)

        relative_position = (
            position[:, None] - position[None, :]
```

```
        )

        relative_position = torch.clamp(
            relative_position,
            -self.max_relative_position,
            self.max_relative_position,
        )

        relative_position += self.max_relative_position
        return self.embedding(relative_position)
```

若 Q 的 Shape 为 $[B, H, L, D]$，则可以通过

```
relative_score = torch.einsum(
    "bhid,ijd->bhij",
    Q,
    relative_embedding,
)

content_score = Q @ K.transpose(-2, -1)
scores = content_score + relative_score
scores = scores / (Q.size(-1) ** 0.5)
```

得到 Shaw 风格的 Attention Score。

下面给出一个简化的 T5 Relative Position Bias。为了突出核心机制，代码省略 T5 原始实现中更完整的双向处理和对数 Bucket 细节，只保留“相对距离 → Bucket → Head-specific Bias”的主流程。

```
class T5RelativePositionBias(nn.Module):
    def __init__(self, num_heads, num_buckets=32):
        super().__init__()
        self.num_heads = num_heads
        self.num_buckets = num_buckets
        self.bias = nn.Embedding(num_buckets, num_heads)

    def relative_position_bucket(self, relative_position):
        # Simplified bucket mapping for explanation.
        distance = relative_position.abs()
        bucket = torch.clamp(
```

```
            distance,
            max=self.num_buckets - 1,
        )
        return bucket

    def forward(self, seq_len, device=None):
        position = torch.arange(seq_len, device=device)
        relative_position = (
            position[:, None] - position[None, :]
        )

        bucket = self.relative_position_bucket(
            relative_position
        )

        # [L, L, H] -> [H, L, L]
        bias = self.bias(bucket).permute(2, 0, 1)
        return bias
```

使用时只需要将 Bias 加到 Softmax 之前的 Attention Score：

```
scores = Q @ K.transpose(-2, -1)
scores = scores / (Q.size(-1) ** 0.5)
scores = scores + position_bias.unsqueeze(0)
```

因此，相对位置编码与绝对位置编码之间最重要的结构差异是：绝对位置编码通常在输入端执行 $\boldsymbol{X} + \boldsymbol{P}$，而相对位置编码通常直接修改 token 与 token 之间的 Attention 打分。

2.3 RoPE：旋转位置编码

主要思想 RoPE（Rotary Position Embedding）是现代大语言模型中最常见的位置编码方案之一。它不把位置向量加到输入 $\boldsymbol{X}$ 上，也不直接给 Attention Score 增加离散 Bias，而是对 Query 和 Key 的每两个维度执行与位置相关的二维旋转。

对 $\boldsymbol{Q}$、$\boldsymbol{K}$ 注入绝对位置旋转，使二者做内积后自然得到相对位置关系。

考虑二维向量

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

二维旋转矩阵为

$$\boldsymbol{R}(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}.$$

如果 token 位于位置 m，则可以使用角度 $m\theta$ 对向量进行旋转：$\boldsymbol{x}'_m = \boldsymbol{R}(m\theta)\boldsymbol{x}$.

在实际高维向量中，RoPE 将每两个维度组成一对，并为不同维度对设置不同频率。

RoPE 的数学表示 设 Head 维度为 d，定义第 i 个二维子空间的频率

$$\theta_i = 10000^{-2i/d}, \qquad 0 \le i < \frac{d}{2}.$$

对于位置 m，第 i 个二维子空间的旋转角为 $m\theta_i$.

因此 Query 和 Key 分别变为

$$\boldsymbol{q}'_m = \boldsymbol{R}_m \boldsymbol{q}_m, \qquad \boldsymbol{k}'_n = \boldsymbol{R}_n \boldsymbol{k}_n.$$

它们的内积为

$$(\boldsymbol{q}'_m)^{\mathsf{T}} \boldsymbol{k}'_n = \boldsymbol{q}_m^{\mathsf{T}} \boldsymbol{R}_m^{\mathsf{T}} \boldsymbol{R}_n \boldsymbol{k}_n.$$

旋转矩阵满足 $\boldsymbol{R}_m^{\mathsf{T}} \boldsymbol{R}_n = \boldsymbol{R}_{n-m}$, 因此

$$(\boldsymbol{q}'_m)^{\mathsf{T}} \boldsymbol{k}'_n = \boldsymbol{q}_m^{\mathsf{T}} \boldsymbol{R}_{n-m} \boldsymbol{k}_n.$$

可以看到，虽然位置 m 和 n 分别用于产生绝对旋转角，但最终内积只通过 $n-m$ 体现二者的位置关系。这是 RoPE 最关键的数学性质。

如果用逐维形式表示，对于一对元素 (x_{2i}, x_{2i+1})，位置 m 的旋转结果为

$$\begin{aligned} x'_{2i} &= x_{2i}\cos(m\theta_i) - x_{2i+1}\sin(m\theta_i), \\ x'_{2i+1} &= x_{2i}\sin(m\theta_i) + x_{2i+1}\cos(m\theta_i). \end{aligned}$$

因此 RoPE 不改变 Tensor Shape，只改变 $\boldsymbol{Q}$ 和 $\boldsymbol{K}$ 在每个二维子空间中的方向：

$$[B, H, L, D] \longrightarrow [B, H, L, D].$$

Python 实现：RoPE 下面实现 RoPE 的三个核心步骤：构造频率、计算每个位置的 cos / sin、对 $\boldsymbol{Q}$ 和 $\boldsymbol{K}$ 执行旋转。

```python
import torch

def rotate_half(x):
    # (..., x0, x1, x2, x3, ...)
    # -> (..., -x1, x0, -x3, x2, ...)
    x_even = x[..., 0::2]
    x_odd = x[..., 1::2]

    rotated = torch.stack(
        (-x_odd, x_even), dim=-1
    )
    return rotated.flatten(-2)

def build_rope_cache(
    seq_len,
    head_dim,
    base=10000.0,
    device=None,
):
    assert head_dim % 2 == 0

    inv_freq = 1.0 / (
        base
        ** (
            torch.arange(
                0, head_dim, 2, device=device
            ).float()
            / head_dim
        )
    )

    position = torch.arange(
        seq_len, device=device
```

```
    ).float()

    freqs = torch.outer(position, inv_freq)
    freqs = torch.repeat_interleave(freqs, 2, dim=-1)

    return freqs.cos(), freqs.sin()

def apply_rope(x, cos, sin):
    # x:   [B, H, L, D]
    # cos: [L, D]
    # sin: [L, D]
    cos = cos.unsqueeze(0).unsqueeze(0)
    sin = sin.unsqueeze(0).unsqueeze(0)

    return x * cos + rotate_half(x) * sin
```

在 Attention 中使用：

```
cos, sin = build_rope_cache(
    seq_len=Q.size(2),
    head_dim=Q.size(3),
    device=Q.device,
)

Q = apply_rope(Q, cos, sin)
K = apply_rope(K, cos, sin)

scores = Q @ K.transpose(-2, -1)
scores = scores / (Q.size(-1) ** 0.5)
```

RoPE 被 LLaMA、LLaMA 2/3、Qwen、Mistral、Gemma、GPT-NeoX 等大量现代模型或模型家族采用。它兼具较低的实现复杂度和良好的相对位置性质，因此成为当前 LLM 中非常重要的位置编码路线。

2.4 ALiBi：线性位置偏置

主要思想 ALiBi（Attention with Linear Biases）采用了与 RoPE 不同的思路。它不创建位置 embedding，也不旋转 $\boldsymbol{Q}$、$\boldsymbol{K}$，而是直接根据 Query 与 Key 之间的距离，对 Attention Score 加入线性惩罚。

标准 Attention Score 为

$$A_{ij} = \frac{\boldsymbol{q}_i^{\mathsf{T}} \boldsymbol{k}_j}{\sqrt{d_k}}.$$

ALiBi 将其改为

$$A_{ij} = \frac{\boldsymbol{q}_i^{\mathsf{T}} \boldsymbol{k}_j}{\sqrt{d_k}} - m_h \, d(i,j),$$

其中 m_h 是第 h 个 Attention Head 的 slope，$d(i,j)$ 表示位置 i 与位置 j 的距离。

对于 causal language model，由于只允许关注当前位置及之前的位置，常使用

$$d(i,j) = i - j, \qquad j \le i.$$

于是距离越远，加入的负偏置越大。

数学表示与多 Head 斜率 对于第 h 个 Head，可以写成

$$A_{ij}^{(h)} = \frac{(\boldsymbol{q}_i^{(h)})^{\mathsf{T}} \boldsymbol{k}_j^{(h)}}{\sqrt{d_k}} - m_h (i - j).$$

不同 Head 使用不同的 m_h。较大的 slope 会更强地惩罚远距离 token，使该 Head 更偏向局部上下文；较小的 slope 对长距离的惩罚更弱，因此可以保留更宽的注意力范围。

从计算位置看，ALiBi 与 T5 Relative Position Bias 都直接修改 Attention Score，但二者的 Bias 构造方式不同：T5 的 Bias 来自可学习的相对位置 Bucket，而 ALiBi 使用确定性的线性距离函数和每个 Head 的 slope。

Python 实现：ALiBi ALiBi Bias 示例用于说明机制；实际实现应使用论文给定的 slope 生成规则，并结合 causal mask 进行广播。

```python
import torch

def build_alibi_bias(
    num_heads,
    seq_len,
    device=None,
):
    # A compact slope schedule for demonstration.
    slopes = torch.pow(
        2.0,
        -torch.arange(
            1, num_heads + 1, device=device
        ).float()
        / num_heads,
    )

    position = torch.arange(seq_len, device=device)

    # distance[i, j] = i - j for causal attention
    distance = position[:, None] - position[None, :]
    distance = distance.clamp(min=0).float()

    # [H, L, L]
    bias = -slopes[:, None, None] * distance[None, :, :]
    return bias
```

加入 Attention Score：

```python
scores = Q @ K.transpose(-2, -1)
scores = scores / (Q.size(-1) ** 0.5)

alibi_bias = build_alibi_bias(
    num_heads=Q.size(1),
    seq_len=Q.size(2),
    device=Q.device,
)

scores = scores + alibi_bias.unsqueeze(0)
```

从结构上，可以将 RoPE 与 ALiBi 对比为

$$\begin{aligned}\text{RoPE:}&\quad Q, K \text{ 发生位置相关旋转},\\ \text{ALiBi:}&\quad Q, K \text{ 保持不变，Attention Score 加距离偏置}.\end{aligned}$$

ALiBi 的实现非常简单，并具有一定的长度外推能力。BLOOM、MPT 等模型或模型家族采用过这一类线性偏置方案。

2.5 RoPE 的长上下文扩展：PI、NTK-aware Scaling 与 YaRN

为什么标准 RoPE 需要扩展 假设一个模型训练时的最大上下文长度为 $L_{\text{train}} = 4096$. 标准 RoPE 在训练阶段主要观察位置 $0, 1, \ldots, 4095$ 对应的旋转角。如果推理时直接扩展到 $L_{\text{target}} = 32768$, 模型会遇到大量训练阶段从未观察过的角度与相位关系，导致位置分布发生变化。

因此，长上下文 RoPE 方法的核心问题不是重新发明一种完全不同的位置编码，而是：

如何改变位置或频率映射，使模型能够在更长序列中继续使用 RoPE。

典型方法包括 Position Interpolation、NTK-aware RoPE Scaling 和 YaRN。

Position Interpolation Position Interpolation（PI）的核心思想是把更长的位置区间压缩回训练时的位置范围。

设目标上下文长度与训练长度之比为

$$s = \frac{L_{\text{target}}}{L_{\text{train}}}.$$

原始位置 pos 被缩放为

$$\text{pos}' = \frac{\text{pos}}{s} = \text{pos}\frac{L_{\text{train}}}{L_{\text{target}}}.$$

例如

$$L_{\text{train}} = 4096, \qquad L_{\text{target}} = 32768,$$

则 $s = 8$. 位置 32000 被映射为 $\text{pos}' = \frac{32000}{8} = 4000$. 这样可以让更长上下文对应的旋转角仍然落在接近原始训练范围的位置区间内。

PI 的优点是简单直接，但所有频率都使用同一位置缩放，也可能压缩短距离位置之间的分辨率。

NTK-aware RoPE Scaling 标准 RoPE 的频率可以写成

$$\theta_i = \text{base}^{-2i/d}.$$

Position Interpolation 主要修改位置 pos；NTK-aware Scaling 则更关注修改频率分布或等效的 RoPE base，使不同维度的旋转速度发生变化。

一个用于说明思想的形式是

$$\theta_i' = (\text{base}')^{-2i/d}, \qquad \text{base}' > \text{base}.$$

当 base 增大后，部分维度的频率降低，对应旋转周期变长，因此能够覆盖更长的位置范围。

需要注意，实际 NTK-aware RoPE Scaling 通常使用比简单 $\text{base}' = \text{base} \times s$ 更精细的公式。这里的重要思想是：

不统一压缩所有位置，而是重新分配 RoPE 在不同维度上的频率尺度。

YaRN YaRN 进一步考虑到 RoPE 的不同维度具有不同频率，因此没有必要让所有频率都采用完全相同的插值或外推方式。

可以将其核心思想抽象表示为

$$\theta_i' = (1 - \gamma_i)\theta_i^{\text{interp}} + \gamma_i \theta_i^{\text{extrap}},$$

其中：

- θ_i^{interp} 表示插值后的频率；
- θ_i^{extrap} 表示保留或外推的频率；
- γ_i 控制第 i 个频率维度采用哪一种策略。

直观上，高频部分更关系到局部位置分辨率，低频部分更关系到长距离变化，因此 YaRN 可以在不同频率区间使用不同程度的缩放，并在中间区域进行平滑过渡。实际 YaRN 还会结合 attention scaling 等机制，以减少扩展上下文后 Attention 分布的变化。

因此，三种方法可以从修改对象上理解为

$$\begin{aligned} \text{PI:} &\quad \text{主要缩放 position,} \\ \text{NTK-aware:} &\quad \text{主要调整 RoPE frequency / base,} \\ \text{YaRN:} &\quad \text{按频率区间组合 interpolation 与 extrapolation.} \end{aligned}$$

Python 实现：PI 与 NTK-aware RoPE 下面在前面的 RoPE cache 基础上实现 Position Interpolation。关键变化只有一处：在计算频率之前将 position 除以 scaling factor。

```python
def build_pi_rope_cache(
    seq_len,
    head_dim,
    scaling_factor,
    base=10000.0,
    device=None,
):
    inv_freq = 1.0 / (
        base
        ** (
            torch.arange(
                0, head_dim, 2, device=device
            ).float()
            / head_dim
        )
    )

    position = torch.arange(
        seq_len, device=device
    ).float()

    # Position Interpolation
    position = position / scaling_factor

    freqs = torch.outer(position, inv_freq)
    freqs = torch.repeat_interleave(freqs, 2, dim=-1)

    return freqs.cos(), freqs.sin()
```

这里用增大 base 的方式简化演示 NTK-aware 思想。实际模型应使用对应方案的精确 scaling 公式，不能将该示例视为通用标准。

```
def build_ntk_rope_cache(
    seq_len,
    head_dim,
    scaling_factor,
    base=10000.0,
    device=None,
):
    # Simplified educational form.
    new_base = base * scaling_factor

    inv_freq = 1.0 / (
        new_base
        ** (
            torch.arange(
                0, head_dim, 2, device=device
            ).float()
            / head_dim
        )
    )

    position = torch.arange(
        seq_len, device=device
    ).float()

    freqs = torch.outer(position, inv_freq)
    freqs = torch.repeat_interleave(freqs, 2, dim=-1)

    return freqs.cos(), freqs.sin()
```

PI、NTK-aware Scaling 和 YaRN 严格来说都不是与 RoPE 平行的“全新位置编码范式”，而是建立在 RoPE 上的长上下文扩展方法。理解时应先掌握标准 RoPE，再进一步分析它们分别改变了 position、frequency 以及不同频率区间的插值策略。

2.6 不同位置编码方法的比较

从作用位置理解不同方法 不同位置编码方法最直观的区别，是位置信息在 Transformer 的哪个阶段被注入。

绝对位置编码通常作用于输入：

$$\boldsymbol{X} \longrightarrow \boldsymbol{X} + \boldsymbol{P}.$$

Shaw、Transformer-XL、T5、DeBERTa 等相对位置方法主要作用于 Attention 的打分过程：

$$\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}} \longrightarrow \boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}} + \text{Relative Position Term}.$$

RoPE 作用于 Query 和 Key：

$$\boldsymbol{Q}, \boldsymbol{K} \longrightarrow \boldsymbol{R}(\boldsymbol{Q}), \boldsymbol{R}(\boldsymbol{K}) \longrightarrow \boldsymbol{R}(\boldsymbol{Q})\boldsymbol{R}(\boldsymbol{K})^{\mathsf{T}}.$$

ALiBi 直接作用于 Attention Score：

$$\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}} \longrightarrow \boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}} + \text{Linear Distance Bias}.$$

长上下文 RoPE Scaling 则作用于 RoPE 的 position 或 frequency 映射：

$$(\text{pos}, \theta) \longrightarrow (\text{pos}', \theta').$$

因此，可以把位置编码理解为四个主要注入位置：

$$\text{Input} \quad / \quad \text{Q,K} \quad / \quad \text{Attention Score} \quad / \quad \text{RoPE Frequency Mapping}.$$

主要方法对比

方法	作用位置	核心数学形式	代表模型或模型家族
Sinusoidal	Input	$\mathrm{PE}(\mathrm{pos}, 2i) = \sin(\mathrm{pos}\,\theta_i)$, $\mathrm{PE}(\mathrm{pos}, 2i+1) = \cos(\mathrm{pos}\,\theta_i)$	Transformer
Learned Absolute	Input	$h_i = x_i + P_i$	GPT-1、GPT-2、BERT、OPT
Shaw Relative	Attention Score	$\boldsymbol{q}_i^\top(\boldsymbol{k}_j + \boldsymbol{r}_{i-j})$	Relative Transformer 类方法
Transformer-XL	Attention Score	$\boldsymbol{q}_i^\top \boldsymbol{k}_j + \boldsymbol{q}_i^\top \boldsymbol{r}_{i-j} + \boldsymbol{u}^\top \boldsymbol{k}_j + \boldsymbol{v}^\top \boldsymbol{r}_{i-j}$	Transformer-XL、XLNet
T5 Relative Bias	Attention Score	$\boldsymbol{q}_i^\top \boldsymbol{k}_j + b_{\mathrm{bucket}(i-j)}$	T5、Flan-T5
DeBERTa	Attention Score	C2C + C2P + P2C	DeBERTa 系列
RoPE	Query / Key	$\boldsymbol{q}'_m = \boldsymbol{R}_m \boldsymbol{q}_m$, $\boldsymbol{k}'_n = \boldsymbol{R}_n \boldsymbol{k}_n$	LLaMA、Qwen、Mistral、Gemma、GPT-NeoX 等
ALiBi	Attention Score	$\boldsymbol{q}_i^\top \boldsymbol{k}_j - m_h(i-j)$	BLOOM、MPT 等
Position Interpolation	RoPE position	$\mathrm{pos}' = \mathrm{pos}/s$	长上下文 RoPE 扩展方案
NTK-aware Scaling	RoPE frequency	$\theta'_i = (\mathrm{base}')^{-2i/d}$	多种 RoPE 长上下文实现
YaRN	RoPE frequency	不同频率区间混合插值与外推	长上下文 LLM 实现

第 3 章

Normalization 与残差连接

在前面的注意力机制与位置编码中，重点关注的是 token 之间如何建立依赖关系，以及模型如何获得序列位置信息。但一个 Transformer Block 并不只有 Attention。为了让深层网络能够稳定训练，Transformer 还需要两个非常重要的结构：Normalization 与 Residual Connection。

Residual Connection 解决的核心问题是：当网络不断加深时，如何让原始表示与梯度都能够沿着一条较短的路径继续传播。Normalization 解决的核心问题则是：如何控制隐藏状态的数值尺度，使不同层之间的激活分布更加稳定。

设某一层输入为 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$，其中 B 表示 batch size，L 表示序列长度，d_{model} 表示隐藏维度。一个典型的 Transformer 子层可以抽象为

$$\boldsymbol{F} : \mathbb{R}^{B\times L\times d_{\text{model}}} \longrightarrow \mathbb{R}^{B\times L\times d_{\text{model}}}.$$

因为输入与输出 Shape 一致，所以可以直接构造残差连接 $\boldsymbol{Y} = \boldsymbol{X} + \boldsymbol{F}(\boldsymbol{X})$.

Normalization 则通常沿隐藏维度对每个 token 的表示进行归一化。现代 LLM 中最常见的两类方法是 LayerNorm 与 RMSNorm；而从归一化放置位置来看，又可以分为 Pre-Norm 与 Post-Norm。

核心问题包括：

- Residual Connection 为什么能够帮助深层网络训练；
- LayerNorm 如何计算均值、方差并进行仿射变换；
- RMSNorm 如何只使用均方根控制向量尺度；
- Pre-Norm 与 Post-Norm 在 Transformer Block 中的结构差异；
- 为什么现代 LLM 经常采用 RMSNorm；
- 为什么深层 Decoder-only LLM 经常采用 Pre-Norm；
- 如何使用 PyTorch 手写 LayerNorm、RMSNorm 以及 Pre-Norm / Post-Norm 残差结构。

3.1 Residual Connection：残差连接

主要思想 假设某个子层需要学习从输入 $\boldsymbol{x}$ 到目标表示 $\boldsymbol{y}$ 的映射。普通前馈结构直接学习 $\boldsymbol{y} = \boldsymbol{F}(\boldsymbol{x})$. Residual Connection 则将输入 $\boldsymbol{x}$ 直接加到子层输出上：$\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x})$.

这里可以把 $\boldsymbol{F}(\boldsymbol{x})$ 理解为当前子层需要在原始表示基础上学习的 “修正量”。因此，子层不必重新构造完整输出，而只需要学习 $\boldsymbol{F}(\boldsymbol{x}) = \boldsymbol{y} - \boldsymbol{x}$.

保留一条恒等映射路径，让每个子层只负责学习输入表示需要增加的变化。

对于 Transformer，Attention 和 FFN 通常都保持隐藏维度 d_{model} 不变，因此可以直接相加：

$$\boldsymbol{X} \in \mathbb{R}^{B\times L\times d_{\text{model}}}, \qquad \boldsymbol{F}(\boldsymbol{X}) \in \mathbb{R}^{B\times L\times d_{\text{model}}},$$

从而

$$\boldsymbol{Y} = \boldsymbol{X} + \boldsymbol{F}(\boldsymbol{X}) \in \mathbb{R}^{B\times L\times d_{\text{model}}}.$$

残差连接与梯度传播 Residual Connection 的重要意义不仅体现在前向传播，也体现在反向传播。

设 $\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x})$, 损失函数为 $\mathcal{L}$。根据链式法则，

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{y}} \left(\boldsymbol{I} + \frac{\partial \boldsymbol{F}(\boldsymbol{x})}{\partial \boldsymbol{x}} \right),$$

其中 $\boldsymbol{I}$ 表示恒等映射对应的 Jacobian。

这一公式中最重要的是 $\boldsymbol{I}$. 即使 $\boldsymbol{F}(\boldsymbol{x})$ 对输入的局部梯度较小，梯度仍然存在一条通过恒等路径传播的直接通道。

如果连续堆叠多个残差层 $\boldsymbol{x}_{l+1} = \boldsymbol{x}_l + \boldsymbol{F}_l(\boldsymbol{x}_l)$, 那么从表示角度看，可以递推得到

$$\boldsymbol{x}_L = \boldsymbol{x}_0 + \sum_{l=0}^{L-1} \boldsymbol{F}_l(\boldsymbol{x}_l).$$

因此，深层网络的最终隐藏状态可以理解为初始表示与多层增量更新的累积。

Transformer 中的两条残差支路 一个标准 Transformer Block 通常包含两个主要子层：

1. Attention；

2. Feed Forward Network（FFN）。

因此，一个 Block 通常包含两次残差相加。暂时忽略 Normalization，可以写成

$$\boldsymbol{H} = \boldsymbol{X} + \mathrm{Attention}(\boldsymbol{X}),$$

以及

$$\boldsymbol{Y} = \boldsymbol{H} + \mathrm{FFN}(\boldsymbol{H}).$$

其数据流可以概括为

$$\boldsymbol{X} \longrightarrow \boldsymbol{X} + \mathrm{Attention}(\boldsymbol{X}) \longrightarrow \boldsymbol{H} + \mathrm{FFN}(\boldsymbol{H}).$$

需要注意，残差相加要求两条分支的 Shape 一致。因此 Transformer 中 Attention 输出投影与 FFN 最终投影都会重新回到 d_{model}：

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{model}}].$$

Python 实现：Residual Connection 残差连接本身非常简单。下面定义一个通用残差模块，其中 `fn` 可以是 Attention、FFN 或其他保持 Shape 不变的子层。

```python
import torch
import torch.nn as nn

class Residual(nn.Module):
    def __init__(self, fn):
        super().__init__()
        self.fn = fn

    def forward(self, x):
        return x + self.fn(x)
```

如果子层使用一个简单的两层网络，可以写成

```
d_model = 512
ffn = nn.Sequential(nn.Linear(d_model, 2048), nn.GELU(), nn.Linear(2048, d_model))
residual = Residual(ffn)

x = torch.randn(2, 16, d_model)
y = residual(x)

print(x.shape)
print(y.shape)
```

输入输出均保持

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{model}}].$$

3.2 LayerNorm：层归一化

主要思想 Transformer 的隐藏状态中，每个 token 都对应一个 d_{model} 维向量。对于某个 token，记其隐藏向量为

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_d), \qquad d = d_{\text{model}}.$$

LayerNorm 的基本思想是：

对单个 token 的隐藏维度进行归一化，使该 token 的特征具有稳定的均值与尺度。

LayerNorm 不需要统计整个 batch 的均值和方差。它直接对当前 token 的最后一个隐藏维度进行计算，因此特别适合序列模型。

对于输入 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d}$, LayerNorm 分别对每一个 (b, l) 位置对应的 d 维向量执行归一化。

LayerNorm 的数学表示 首先计算均值

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i.$$

再计算方差

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2.$$

标准化后的第 i 个分量为

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}},$$

其中 ε 是一个很小的正数，用于避免分母接近 0。

如果只做标准化，所有 token 都会被强制映射到类似的统计尺度。为了让模型仍然能够学习合适的特征尺度与偏移，LayerNorm 进一步加入两个可学习参数 $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$.

最终得到

$$[\text{LayerNorm}(\boldsymbol{x})]_i = \gamma_i \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_i.$$

因此 LayerNorm 可以拆成三个步骤：

$$\boldsymbol{x} \longrightarrow \boldsymbol{x} - \mu \longrightarrow \frac{\boldsymbol{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}} \longrightarrow \boldsymbol{\gamma} \odot \hat{\boldsymbol{x}} + \boldsymbol{\beta}.$$

其中 $\odot$ 表示逐元素乘法。

LayerNorm 的参数量与 Tensor Shape 如果隐藏维度为 d，标准 LayerNorm 包含

$$\boldsymbol{\gamma} \in \mathbb{R}^d, \qquad \boldsymbol{\beta} \in \mathbb{R}^d.$$

因此可学习参数量为 $2d$。

LayerNorm 不改变 Tensor Shape：

$$[B, L, d] \longrightarrow [B, L, d].$$

它改变的是每个 token 隐藏向量内部的数值分布，而不是序列长度、batch size 或隐藏维度。

LayerNorm 与 BatchNorm 的区别 BatchNorm 与 LayerNorm 的归一化维度不同。对于 Transformer，最重要的区别可以概括为：

比较项	BatchNorm	LayerNorm
统计范围	依赖 batch 中多个样本的统计量	对单个 token 的隐藏维度计算统计量
对 batch size 的依赖	较明显	基本不依赖 batch size
序列模型中的使用	需要处理 batch 与序列维的组织方式	可以直接作用于最后一个隐藏维度
Transformer 中的常见程度	较少作为主归一化结构	经典 Transformer 的标准组件

对于语言模型而言，一个 batch 中不同样本的 token 内容和长度可能差异很大。LayerNorm 对每个 token 独立处理，不需要依赖其他样本的统计量，因此与 Transformer 的 token-wise 表示方式更加自然。

Python 实现：手写 LayerNorm 下面给出与公式对应的 LayerNorm 实现。

```python
import torch
import torch.nn as nn

class LayerNorm(nn.Module):
    def __init__(self, d_model, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model))
        self.bias = nn.Parameter(torch.zeros(d_model))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = (x - mean).pow(2).mean(dim=-1, keepdim=True)
        x_hat = (x - mean) * torch.rsqrt(var + self.eps)
        return self.weight * x_hat + self.bias
```

这里 `mean(dim=-1)` 表示沿隐藏维度计算均值，而 `keepdim=True` 用于保留最后一维，使均值和方差能够直接广播到原始 Tensor。

可以与 PyTorch 自带实现进行对照：

```
x = torch.randn(2, 16, 512)

manual_ln = LayerNorm(512)
torch_ln = nn.LayerNorm(512)

manual_ln.weight.data.copy_(torch_ln.weight.data)
manual_ln.bias.data.copy_(torch_ln.bias.data)

y1 = manual_ln(x)
y2 = torch_ln(x)

print(torch.allclose(y1, y2, atol=1e-5))
```

3.3 RMSNorm：均方根归一化

从 LayerNorm 到 RMSNorm LayerNorm 同时做两件事情：

1. 减去均值，对向量进行中心化；
2. 除以标准差，对向量尺度进行归一化。

RMSNorm 对这一过程进行了简化。它不再显式减去均值，而只根据输入向量的均方根控制整体尺度。

不做均值中心化，只根据隐藏向量的 Root Mean Square 对其进行缩放。

RMSNorm 的数学表示 对于 $\boldsymbol{x} = (x_1, x_2, \ldots, x_d)$, 定义均方根

$$\mathrm{RMS}(\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \varepsilon}.$$

归一化后得到

$$\hat{x}_i = \frac{x_i}{\mathrm{RMS}(\boldsymbol{x})}.$$

再乘以可学习的缩放参数 $\boldsymbol{g} \in \mathbb{R}^d$, 最终得到

$$[\mathrm{RMSNorm}(\boldsymbol{x})]_i = g_i \frac{x_i}{\sqrt{\frac{1}{d}\sum_{j=1}^{d} x_j^2 + \varepsilon}}.$$

与标准 LayerNorm 相比，RMSNorm 通常没有减均值操作，也通常不需要额外的 bias 参数。

两者可以直接对比为

$$\mathrm{LayerNorm}(\boldsymbol{x}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{x} - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \boldsymbol{\beta},$$

以及

$$\mathrm{RMSNorm}(\boldsymbol{x}) = \boldsymbol{g} \odot \frac{\boldsymbol{x}}{\sqrt{\mathrm{mean}(\boldsymbol{x}^2) + \varepsilon}}.$$

RMSNorm 的参数量 若隐藏维度为 d，常见 RMSNorm 只有一个可学习缩放参数 $\boldsymbol{g} \in \mathbb{R}^d$. 因此参数量为 d。

与包含 $\boldsymbol{\gamma}$ 和 $\boldsymbol{\beta}$ 的标准 LayerNorm 相比，参数量由 $2d$ 减少为 d。

不过对于大语言模型而言，这部分参数本身并不是主要参数来源。RMSNorm 更重要的特点是计算路径更简单：它不需要显式计算并减去均值。

Python 实现：手写 RMSNorm

```python
class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model))

    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return self.weight * x * rms
```

这里 `x.pow(2).mean(dim=-1)` 对应

$$\frac{1}{d}\sum_{i=1}^{d} x_i^2,$$

而 `torch.rsqrt(z)` 计算 $\frac{1}{\sqrt{z}}$.

因此代码中的 `x * rms` 正好对应

$$\frac{\boldsymbol{x}}{\sqrt{\mathrm{mean}(\boldsymbol{x}^2) + \varepsilon}}.$$

RMSNorm 同样保持 Tensor Shape 不变：

$$[B, L, d] \longrightarrow [B, L, d].$$

3.4 Pre-Norm 与 Post-Norm

Normalization 应该放在哪里 Residual Connection 与 Normalization 都已经确定之后，还需要回答一个结构问题：

Normalization 应该放在子层 F 的前面，还是放在残差相加之后？

这对应两种经典结构：

- Post-Norm：先执行子层和残差相加，再进行归一化；
- Pre-Norm：先对输入归一化，再送入子层，最后进行残差相加。

两种结构使用的 Attention、FFN 以及残差思想都可以完全相同，真正变化的是 Norm 的位置。

Post-Norm Post-Norm 的基本形式为

$$\boldsymbol{y} = \mathrm{Norm}\left(\boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x})\right).$$

对于包含 Attention 和 FFN 的 Transformer Block，可以写成

$$\boldsymbol{h} = \mathrm{Norm}_1\left(\boldsymbol{x} + \mathrm{Attention}(\boldsymbol{x})\right),$$

$$\boldsymbol{y} = \mathrm{Norm}_2\left(\boldsymbol{h} + \mathrm{FFN}(\boldsymbol{h})\right).$$

其数据流为

$$\boldsymbol{x} \longrightarrow \boldsymbol{F}(\boldsymbol{x}) \longrightarrow \boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x}) \longrightarrow \mathrm{Norm}.$$

原始 Transformer 采用的就是典型 Post-Norm 结构。

Post-Norm 的特点是：每个子层输出在进入下一子层之前已经经过归一化，因此每层输出的尺度受到直接控制。但从深层网络的反向传播路径看，Normalization 位于相邻残差块之间，梯度需要反复经过 Norm 的 Jacobian。

Pre-Norm Pre-Norm 将 Normalization 移到子层之前：

$$\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{F}(\mathrm{Norm}(\boldsymbol{x})).$$

对于完整 Transformer Block，可以写成

$$\boldsymbol{h} = \boldsymbol{x} + \mathrm{Attention}\left(\mathrm{Norm}_1(\boldsymbol{x})\right),$$

$$\boldsymbol{y} = \boldsymbol{h} + \mathrm{FFN}\left(\mathrm{Norm}_2(\boldsymbol{h})\right).$$

其数据流为

$$\boldsymbol{x} \longrightarrow \mathrm{Norm}(\boldsymbol{x}) \longrightarrow \boldsymbol{F} \longrightarrow \boldsymbol{x} + \boldsymbol{F}(\mathrm{Norm}(\boldsymbol{x})).$$

此时，Residual Stream 本身可以沿恒等路径直接从当前层传到下一层。对于多层 Pre-Norm 网络，

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_l + \boldsymbol{F}_l(\mathrm{Norm}(\boldsymbol{x}_l)).$$

递推可得

$$\boldsymbol{x}_L = \boldsymbol{x}_0 + \sum_{l=0}^{L-1} \boldsymbol{F}_l(\mathrm{Norm}(\boldsymbol{x}_l)).$$

可以看到，主残差流中始终保留显式的 $\boldsymbol{x}_l \to \boldsymbol{x}_{l+1}$ 恒等分支。

Pre-Norm 的最终归一化 Pre-Norm 中每个子层的残差相加之后通常不会立刻执行 Norm。因此，在堆叠多个 Transformer Block 后，模型通常还会在最终输出位置增加一次 Normalization：

$$\boldsymbol{h}_{\text{final}} = \mathrm{Norm}(\boldsymbol{x}_L).$$

随后再进入 LM Head：

$$\boldsymbol{z} = \boldsymbol{h}_{\text{final}} \boldsymbol{W}_{\text{vocab}}.$$

Pre-Norm 与 Post-Norm 的统一比较

比较项	Post-Norm	Pre-Norm
基本形式	$\mathrm{Norm}(\boldsymbol{x}+\boldsymbol{F}(\boldsymbol{x}))$	$\boldsymbol{x}+\boldsymbol{F}(\mathrm{Norm}(\boldsymbol{x}))$
Norm 位置	残差相加之后	子层计算之前
Residual Stream	每层输出需要经过 Norm 后进入下一层	恒等残差路径更直接
深层训练	通常更依赖合适的初始化、学习率与训练策略	通常更容易获得稳定的深层梯度传播
最终 Norm	每个子层之后已经归一化	通常在全部 Block 之后再增加 Final Norm

为什么现代 LLM 常用 RMSNorm：减少不必要的归一化操作 LayerNorm 使用 $\boldsymbol{x}-\mu$ 进行中心化，并使用标准差控制尺度。

RMSNorm 则只计算 $\sqrt{\mathrm{mean}(\boldsymbol{x}^2)+\varepsilon}$.

因此 RMSNorm 省去了显式均值中心化步骤。对于每一个 token、每一层都需要执行 Norm 的大模型而言，简化单次归一化路径具有实际工程意义。

两者的主要差别可以写成

$$\begin{aligned}\text{LayerNorm:}&\quad \text{Centering}+\text{Scaling}+\text{Affine},\\ \text{RMSNorm:}&\quad \text{Scaling}+\text{Learnable Gain}.\end{aligned}$$

模型真正需要控制的是隐藏状态尺度 在深层 Transformer 中，Normalization 的一个关键作用是防止隐藏状态尺度在层与层之间发生过大的漂移。

RMSNorm 直接约束

$$\frac{1}{d}\sum_{i=1}^{d}x_i^2,$$

即隐藏向量的二阶尺度。对很多现代 Decoder-only LLM 而言，仅控制这一尺度就能够满足稳定训练的需求，因此不一定必须在每个 Norm 中显式减去均值。

这也是 RMSNorm 的设计取舍：

保留对隐藏状态尺度的控制，同时移除 LayerNorm 中的均值中心化步骤。

更简单的参数与计算结构 标准 LayerNorm 常见参数为 $\boldsymbol{\gamma},\boldsymbol{\beta}\in\mathbb{R}^d$，而常见 RMSNorm 只保留 $\boldsymbol{g}\in\mathbb{R}^d$.

从实现角度，RMSNorm 的核心代码只有

```
rms = torch.rsqrt(x.pow(2).mean(dim=-1, keepdim=True) + eps)
y = weight * x * rms
```

因此它非常适合与大规模 Transformer Block 一起重复堆叠。

RMSNorm 是现代 LLM 中常见的选择，但并不意味着 LayerNorm 已经失效，也不意味着所有 LLM 都必须使用 RMSNorm。

Normalization 的最终选择仍然属于模型架构设计的一部分。

为什么现代 LLM 常使用 Pre-Norm：更直接的残差梯度路径 考虑 Pre-Norm：

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_l + \boldsymbol{F}_l(\mathrm{Norm}(\boldsymbol{x}_l)).$$

其 Jacobian 为

$$\frac{\partial \boldsymbol{x}_{l+1}}{\partial \boldsymbol{x}_l} = \boldsymbol{I} + \frac{\partial \boldsymbol{F}_l}{\partial \mathrm{Norm}(\boldsymbol{x}_l)} \frac{\partial \mathrm{Norm}(\boldsymbol{x}_l)}{\partial \boldsymbol{x}_l}.$$

其中仍然显式包含恒等项 $\boldsymbol{I}$.

因此，在很多层连续堆叠时，梯度能够通过 Residual Stream 保留一条较直接的传播路径。

Post-Norm 则写成

$$\boldsymbol{x}_{l+1} = \mathrm{Norm}\left(\boldsymbol{x}_l + \boldsymbol{F}_l(\boldsymbol{x}_l)\right).$$

对应 Jacobian 为

$$\frac{\partial \boldsymbol{x}_{l+1}}{\partial \boldsymbol{x}_l} = \frac{\partial \mathrm{Norm}}{\partial(\cdot)} \left(\boldsymbol{I} + \frac{\partial \boldsymbol{F}_l}{\partial \boldsymbol{x}_l}\right).$$

这里每层的主路径都会经过 Normalization 的 Jacobian。当层数很深时，训练稳定性更容易受到多层 Jacobian 连乘、初始化和学习率等因素影响。

Pre-Norm 将 Normalization 放入残差分支内部，使 Residual Stream 的恒等路径更加直接。

更适合深层 Transformer 的稳定优化 随着 Transformer Block 数量增加，模型需要在几十层甚至更多层之间持续传播梯度。Pre-Norm 的结构让每层都表现为在 Residual Stream 上叠加一个更新项（Identity + Residual Update），从而更接近逐层增加小的修正。

这使深层网络在训练初期更容易保持数值与梯度传播的稳定性，也降低了训练对精细初始化和学习率策略的敏感程度。

因此，现代 Decoder-only LLM 中常见的 Block 可以概括为

$$\boldsymbol{h} = \boldsymbol{x} + \mathrm{Attention}(\mathrm{RMSNorm}(\boldsymbol{x})),$$
$$\boldsymbol{y} = \boldsymbol{h} + \mathrm{FFN}(\mathrm{RMSNorm}(\boldsymbol{h})).$$

这里同时结合了两个常见设计：

$$\mathrm{RMSNorm} + \text{Pre-Norm}.$$

Pre-Norm 并不意味着所有问题都消失 Pre-Norm 的优势主要是深层优化稳定性，但它并不意味着网络深度、残差尺度、初始化、学习率、Attention 数值范围等问题可以被完全忽略。

更准确的理解是：

Pre-Norm 是深层 Transformer 中一种非常实用的稳定化结构，而不是对所有训练问题的唯一解决方案。

因此，在分析一个现代 LLM 架构时，可以分别判断两个问题：

1. 模型使用什么 Normalization：LayerNorm、RMSNorm 或其他变体；
2. Normalization 放在哪里：Pre-Norm、Post-Norm 或其他改进结构。

这两个问题彼此独立。例如，可以构造 Pre-LayerNorm，也可以构造 Pre-RMSNorm。

3.5 Python 实现：LayerNorm、RMSNorm、Pre-Norm 与 Post-Norm

通用 Pre-Norm 与 Post-Norm 残差模块 首先定义 LayerNorm 和 RMSNorm，再分别实现两种 Residual + Norm 结构。

```python
import torch
import torch.nn as nn

class LayerNorm(nn.Module):
    def __init__(self, d_model, eps=1e-5):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model))
        self.bias = nn.Parameter(torch.zeros(d_model))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = (x - mean).pow(2).mean(dim=-1, keepdim=True)
        return self.weight * (x - mean) * torch.rsqrt(var + self.eps) + self.bias

class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model))

    def forward(self, x):
        scale = torch.rsqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return self.weight * x * scale

class PreNormResidual(nn.Module):
    def __init__(self, d_model, fn, norm_cls=RMSNorm):
        super().__init__()
        self.norm = norm_cls(d_model)
        self.fn = fn

    def forward(self, x):
        return x + self.fn(self.norm(x))

class PostNormResidual(nn.Module):
    def __init__(self, d_model, fn, norm_cls=LayerNorm):
        super().__init__()
        self.norm = norm_cls(d_model)
        self.fn = fn
```

```
40
41      def forward(self, x):
42          return self.norm(x + self.fn(x))
```

两种结构分别严格对应

$$\text{Pre-Norm:} \qquad \boldsymbol{x} + \boldsymbol{F}(\text{Norm}(\boldsymbol{x})),$$

以及

$$\text{Post-Norm:} \qquad \text{Norm}(\boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x})).$$

Pre-Norm Transformer Block 骨架 如果 Attention 与 FFN 都已经实现，一个典型的 Pre-Norm Transformer Block 可以写成：

```
1  class PreNormTransformerBlock(nn.Module):
2      def __init__(self, d_model, attention, ffn):
3          super().__init__()
4          self.attn_norm = RMSNorm(d_model)
5          self.ffn_norm = RMSNorm(d_model)
6          self.attention = attention
7          self.ffn = ffn
8
9      def forward(self, x):
10         x = x + self.attention(self.attn_norm(x))
11         x = x + self.ffn(self.ffn_norm(x))
12         return x
```

这段代码对应

$$\boldsymbol{h} = \boldsymbol{x} + \text{Attention}(\text{RMSNorm}(\boldsymbol{x})),$$

$$\boldsymbol{y} = \boldsymbol{h} + \text{FFN}(\text{RMSNorm}(\boldsymbol{h})).$$

如果整个模型由多个 Pre-Norm Block 组成，还可以增加 Final RMSNorm：

```python
class DecoderBackbone(nn.Module):
    def __init__(self, d_model, blocks):
        super().__init__()
        self.blocks = nn.ModuleList(blocks)
        self.norm = RMSNorm(d_model)

    def forward(self, x):
        for block in self.blocks:
            x = block(x)
        return self.norm(x)
```

完整运行测试 下面使用一个简单的两层 MLP 代替 Attention 或 FFN，验证 LayerNorm、RMSNorm、Pre-Norm 和 Post-Norm 的输入输出 Shape。

```python
def test_normalization():
    torch.manual_seed(42)

    B, L, D = 2, 16, 64
    x = torch.randn(B, L, D)

    fn1 = nn.Sequential(nn.Linear(D, 4 * D), nn.GELU(), nn.Linear(4 * D, D))
    fn2 = nn.Sequential(nn.Linear(D, 4 * D), nn.GELU(), nn.Linear(4 * D, D))

    layer_norm = LayerNorm(D)
    rms_norm = RMSNorm(D)
    pre_norm = PreNormResidual(D, fn1, RMSNorm)
    post_norm = PostNormResidual(D, fn2, LayerNorm)

    outputs = {
        "LayerNorm": layer_norm(x),
        "RMSNorm": rms_norm(x),
        "PreNorm": pre_norm(x),
        "PostNorm": post_norm(x),
    }

    for name, y in outputs.items():
        print(f"{name}: {tuple(y.shape)}")
        assert y.shape == x.shape
```

```
    print("all tests passed")

if __name__ == "__main__":
    test_normalization()
```

四种模块都保持

$$[B, L, D] \longrightarrow [B, L, D].$$

这也是 Transformer 中 Normalization 和 Residual Connection 能够被反复堆叠的重要结构条件。

3.6 Normalization 与残差结构的统一理解

四个层次 学习 Transformer 中的 Normalization 与残差连接时，可以分成四个层次。

第一层是 Residual Connection：$\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x})$. 它决定主干信息如何跨越子层传播。

第二层是 Normalization 的具体计算方式：

$$\text{LayerNorm} \quad \text{或} \quad \text{RMSNorm}.$$

第三层是 Normalization 的放置位置：

$$\text{Pre-Norm} \quad \text{或} \quad \text{Post-Norm}.$$

第四层才是将这些结构与 Attention、FFN 组合成完整 Transformer Block：

$$\text{Norm} + \text{Attention} + \text{Residual} + \text{Norm} + \text{FFN} + \text{Residual}.$$

因此，“RMSNorm” 和 “Pre-Norm” 描述的不是同一件事情。RMSNorm 回答 Norm 如何计算, 而 Pre-Norm 回答 Norm 放在什么位置.

主要结构对比

结构	核心数学形式	主要作用	是否改变 Shape
Residual	$\boldsymbol{y} = \boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x})$	保留恒等路径，累积子层更新	否
LayerNorm	$\boldsymbol{\gamma} \odot (\boldsymbol{x} - \mu)/\sqrt{\sigma^2 + \varepsilon} + \boldsymbol{\beta}$	中心化并控制隐藏状态尺度	否
RMSNorm	$\boldsymbol{g} \odot \boldsymbol{x}/\sqrt{\text{mean}(\boldsymbol{x}^2) + \varepsilon}$	主要控制隐藏状态 RMS 尺度	否
Post-Norm	$\text{Norm}(\boldsymbol{x} + \boldsymbol{F}(\boldsymbol{x}))$	残差相加后归一化	否
Pre-Norm	$\boldsymbol{x} + \boldsymbol{F}(\text{Norm}(\boldsymbol{x}))$	子层前归一化，保留更直接的 Residual Stream	否

现代 Decoder-only LLM 中的典型组合 现代 Decoder-only LLM 中常见的组合可以抽象为

$$\boldsymbol{x} \longrightarrow \text{RMSNorm} \longrightarrow \text{Attention} \longrightarrow \text{Residual} \longrightarrow \text{RMSNorm} \longrightarrow \text{FFN} \longrightarrow \text{Residual}\,.$$

数学上写成

$$\boldsymbol{h} = \boldsymbol{x} + \text{Attention}\left(\text{RMSNorm}(\boldsymbol{x})\right),$$

$$\boldsymbol{y} = \boldsymbol{h} + \text{FFN}\left(\text{RMSNorm}(\boldsymbol{h})\right).$$

堆叠 N 个 Block 后，再执行

$$\boldsymbol{h}_{\text{final}} = \text{RMSNorm}(\boldsymbol{x}_N),$$

然后送入 LM Head。

因此，从已经学习过的注意力机制继续向完整 LLM 结构推进时，可以把一个现代 Transformer Block 逐步拼成

$$\text{RoPE / Attention} + \text{RMSNorm} + \text{Residual} + \text{FFN}.$$

其中 Attention 负责 token 之间的信息混合；Normalization 控制隐藏状态尺度；Residual Connection 负责保持主干信息与梯度路径；FFN 则负责每个 token 内部的非线性特征变换。

第 4 章

前馈网络（Feed Forward Network，FFN）

在前面的注意力机制、位置编码以及 Normalization 与残差连接中，已经逐步建立了 Transformer Block 的主要骨架。Attention 负责让不同 token 之间交换信息，而 Feed Forward Network（FFN）负责在每一个 token 内部进行非线性特征变换。二者承担的角色不同：Attention 主要沿序列维混合信息，FFN 则主要沿隐藏维度重新组织和变换特征。

设 Transformer 某一层的输入为 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$，其中 B 表示 batch size，L 表示序列长度，d_{model} 表示隐藏维度。经典 Transformer FFN 可以写成

$$\text{FFN}(\boldsymbol{X}) = \sigma(\boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2,$$

本章统一采用行向量右乘约定；在省略 bias 时可简写为

$$\text{FFN}(\boldsymbol{x}) = \sigma(\boldsymbol{x}\boldsymbol{W}_1)\boldsymbol{W}_2.$$

其核心过程是先把隐藏维度从 d_{model} 扩展到更大的中间维度 d_{ff}，经过非线性激活后再投影回 d_{model}：

$$d_{\text{model}} \longrightarrow d_{\text{ff}} \longrightarrow d_{\text{model}}.$$

FFN 中的激活函数与门控结构从 ReLU 扩展到 GELU / SiLU，并进一步发展出 GLU、GEGLU 和 SwiGLU 等形式。

4.1 Transformer FFN：逐位置前馈网络

主要思想 Self-Attention 会让一个 token 的表示与其他 token 发生交互。假设 Attention 输出为 $\boldsymbol{H} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$，FFN 并不会再次计算 token 与 token 之间的注意力关系，而是对每个位置上的隐藏向量独立应用同一组参数。

对于第 i 个 token 的隐藏向量 $\boldsymbol{h}_i \in \mathbb{R}^{d_{\text{model}}}$, 经典 FFN 为

$$\text{FFN}(\boldsymbol{h}_i) = \sigma(\boldsymbol{h}_i \boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2.$$

所有 token 都使用相同的 $\boldsymbol{W}_1$、$\boldsymbol{W}_2$、$\boldsymbol{b}_1$ 与 $\boldsymbol{b}_2$。因此 FFN 又被称为 Position-wise Feed Forward Network。

Attention 在不同 token 之间混合信息；FFN 在每一个 token 内部沿隐藏维度进行非线性特征变换。

如果把输入写成整个 Tensor $\boldsymbol{X}$，实际代码通常直接对最后一维应用 Linear，因此

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{ff}}] \longrightarrow [B, L, d_{\text{model}}].$$

Batch 维与序列维都保持不变。

两层线性变换 设第一层投影矩阵为 $\boldsymbol{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, 第二层投影矩阵为 $\boldsymbol{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$.

第一层先扩展特征维度：$\boldsymbol{z} = \boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{b}_1$, 其中 $\boldsymbol{z} \in \mathbb{R}^{B \times L \times d_{\text{ff}}}$.

随后经过激活函数 $\boldsymbol{a} = \sigma(\boldsymbol{z})$, 最后再投影回模型隐藏维度：$\boldsymbol{y} = \boldsymbol{a}\boldsymbol{W}_2 + \boldsymbol{b}_2$.

因此完整过程可以表示为

$$\boldsymbol{X} \longrightarrow \boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{b}_1 \longrightarrow \sigma \longrightarrow \boldsymbol{W}_2 + \boldsymbol{b}_2 \longrightarrow \boldsymbol{Y}.$$

为什么中间维度通常更大 经典 Transformer 常令 $d_{\text{ff}} > d_{\text{model}}$. 例如原始 Transformer 中常见

$$d_{\text{model}} = 512, \qquad d_{\text{ff}} = 2048 = 4d_{\text{model}}.$$

从 Shape 看：

$$512 \longrightarrow 2048 \longrightarrow 512.$$

先扩展维度可以让模型在更大的特征空间中进行非线性变换，再把结果压回 Residual Stream 所使用的隐藏维度。这也是为什么 FFN 往往占据 Transformer 参数量与计算量中的很大一部分。

如果暂时忽略 bias，经典 FFN 的参数量约为

$$d_{\text{model}}d_{\text{ff}} + d_{\text{ff}}d_{\text{model}} = 2d_{\text{model}}d_{\text{ff}}.$$

当 $d_{\text{ff}} = 4d_{\text{model}}$ 时，参数量约为 $8d_{\text{model}}^2$。

Python 实现：经典 FFN

```python
import torch
import torch.nn as nn

class TransformerFFN(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.w1 = nn.Linear(d_model, d_ff)
        self.act = nn.ReLU()
        self.w2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.w2(self.act(self.w1(x)))
```

输入输出 Shape 为

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{model}}].$$

因此 FFN 可以直接接入前一章介绍的 Residual Connection：

$$\boldsymbol{y} = \boldsymbol{x} + \text{FFN}(\boldsymbol{x}).$$

ReLU：经典 FFN 激活函数 ReLU（Rectified Linear Unit）定义为 $\text{ReLU}(x) = \max(0, x)$. 分段形式为

$$\text{ReLU}(x) = \begin{cases} 0, & x \le 0, \\ x, & x > 0. \end{cases}$$

其导数在 $x \neq 0$ 时为

$$\text{ReLU}'(x) = \begin{cases} 0, & x < 0, \\ 1, & x > 0. \end{cases}$$

ReLU 的特点是计算简单：正数保持不变，负数直接截断为 0。

ReLU 在 FFN 中的作用 如果没有激活函数，两层线性变换 $(\boldsymbol{x}\boldsymbol{W}_1)\boldsymbol{W}_2$ 仍然可以合并为一个线性变换：$\boldsymbol{x}(\boldsymbol{W}_1\boldsymbol{W}_2) = \boldsymbol{x}\boldsymbol{W}$. 因此，无论堆叠多少纯 Linear 层，本质上都无法提供真正的非线性表示能力。

加入 ReLU 后：

$$\mathrm{FFN}(\boldsymbol{x}) = \mathrm{ReLU}(\boldsymbol{x}\boldsymbol{W}_1)\boldsymbol{W}_2,$$

两层 Linear 之间不再能够直接合并。

这说明激活函数的核心作用是：

在线性投影之间引入非线性，使 FFN 能够学习比单个线性映射更复杂的特征变换。

ReLU 的局限 ReLU 会把所有负输入直接变为 0：$x < 0 \implies \mathrm{ReLU}(x) = 0.$

这意味着负半轴上的梯度也为 0。对于现代大语言模型，研究与工程实践逐渐转向更加平滑的激活函数，例如 GELU 和 SiLU，以及进一步加入门控结构的 GEGLU 与 SwiGLU。

```
def relu(x):
    return torch.clamp(x, min=0.0)
```

GELU：平滑的概率式门控 GELU（Gaussian Error Linear Unit）不像 ReLU 一样通过 0 进行硬截断，而是根据输入大小进行平滑缩放。

GELU 的精确定义为 $\mathrm{GELU}(x) = x\Phi(x)$，其中 $\Phi(x)$ 是标准正态分布的累积分布函数：

$$\Phi(x) = \frac{1}{2}\left(1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right).$$

因此

$$\mathrm{GELU}(x) = \frac{x}{2}\left(1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right).$$

与 ReLU 的区别 ReLU 采用硬门：$x \le 0 \Rightarrow 0.$

GELU 则使用连续的缩放因子 $\Phi(x)$：$\mathrm{GELU}(x) = x\Phi(x).$

当 x 很大且为正时，$\Phi(x) \to 1$, 因此 $\mathrm{GELU}(x) \approx x.$

当 x 很大且为负时，$\Phi(x) \to 0$，因此输出逐渐接近 0，但不像 ReLU 那样在整个负半轴直接截断。

这种平滑性使 GELU 成为很多 Transformer 模型中的重要激活函数。

常用近似公式 工程实现中常见近似形式为

$$\mathrm{GELU}(x) \approx \frac{x}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right].$$

PyTorch 已经直接提供

```
x = torch.randn(2, 16, 512)
y = torch.nn.functional.gelu(x)
```

GELU FFN 的前向计算：

```
class GELUFFN(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.w1 = nn.Linear(d_model, d_ff)
        self.w2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.w2(torch.nn.functional.gelu(self.w1(x)))
```

SiLU / Swish：平滑的自门控激活 SiLU（Sigmoid Linear Unit）定义为 $\mathrm{SiLU}(x) = x\sigma(x)$, 其中 $\sigma(x) = \frac{1}{1+\mathrm{e}^{-x}}$.

因此 $\mathrm{SiLU}(x) = \frac{x}{1+\mathrm{e}^{-x}}$.

Swish 的一般形式为 $\mathrm{Swish}_\beta(x) = x\sigma(\beta x)$. 当 $\beta = 1$ 时，就是常用的 SiLU：$\mathrm{Swish}_1(x) = \mathrm{SiLU}(x)$.

SiLU 的直观理解 SiLU 可以看成输入 x 乘以一个由 x 自己产生的软门：$x \times \sigma(x)$.

当 x 很大且为正时，$\sigma(x) \approx 1$, 因此 $\mathrm{SiLU}(x) \approx x$.

当 x 很大且为负时，$\sigma(x) \approx 0$, 输出逐渐接近 0。

与 ReLU 相比，SiLU 是平滑函数，并且负输入仍然可以产生小的非零输出。

```
def silu(x):
    return x * torch.sigmoid(x)
```

PyTorch 中也可以直接使用

```
act = nn.SiLU()
y = act(x)
```

SiLU 本身已经具有“输入乘门”的形式，而 SwiGLU 会进一步把这个思想扩展为两个不同线性投影之间的门控。

GLU：从单分支激活到门控 FFN 经典 FFN 使用一个中间投影：$z = \boldsymbol{x}\boldsymbol{W}_1$, 然后执行 $\sigma(z)$.

GLU（Gated Linear Unit）引入两条独立的线性分支。一条分支产生内容表示，另一条分支产生门控值，再进行逐元素相乘。

一种常见写法为

$$\mathrm{GLU}(\boldsymbol{x}) = (\boldsymbol{x}\boldsymbol{W}_a) \odot \sigma(\boldsymbol{x}\boldsymbol{W}_b),$$

其中 $\odot$ 表示逐元素乘法。

如果加上输出投影，可以写成

$$\mathrm{FFN}_{\mathrm{GLU}}(\boldsymbol{x}) = [(\boldsymbol{x}\boldsymbol{W}_a) \odot \sigma(\boldsymbol{x}\boldsymbol{W}_b)]\,\boldsymbol{W}_2.$$

其结构可以表示为

$$\boldsymbol{x} \longrightarrow \boldsymbol{x}\boldsymbol{W}_a \longrightarrow \text{value branch},$$
$$\boldsymbol{x} \longrightarrow \boldsymbol{x}\boldsymbol{W}_b \longrightarrow \sigma \longrightarrow \text{gate branch},$$

然后 value $\odot$ gate.

GLU 的核心思想是：

不再只对一个线性投影做激活，而是让一个投影决定“传递什么”，另一个投影决定“通过多少”。

Tensor Shape 假设两条分支都投影到 d_{ff}：

$$\boldsymbol{W}_a, \boldsymbol{W}_b : d_{\mathrm{model}} \longrightarrow d_{\mathrm{ff}}.$$

则

$$\boldsymbol{X}\boldsymbol{W}_a, \boldsymbol{X}\boldsymbol{W}_b \in \mathbb{R}^{B\times L\times d_{\mathrm{ff}}}.$$

逐元素乘法要求两条分支 Shape 相同：

$$[B, L, d_{\mathrm{ff}}] \odot [B, L, d_{\mathrm{ff}}] \longrightarrow [B, L, d_{\mathrm{ff}}].$$

最后输出投影再变为

$$[B, L, d_{\mathrm{ff}}] \longrightarrow [B, L, d_{\mathrm{model}}].$$

Python 实现：GLU FFN

```python
class GLUFFN(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.value_proj = nn.Linear(d_model, d_ff, bias=False)
        self.gate_proj = nn.Linear(d_model, d_ff, bias=False)
        self.down_proj = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        value = self.value_proj(x)
        gate = torch.sigmoid(self.gate_proj(x))
        return self.down_proj(value * gate)
```

GEGLU：GELU 门控机制 GLU 的门控分支使用 Sigmoid：

$$(\boldsymbol{x}\boldsymbol{W}_a) \odot \sigma(\boldsymbol{x}\boldsymbol{W}_b).$$

GEGLU 将门控函数替换为 GELU。常见形式为

$$\mathrm{GEGLU}(\boldsymbol{x}) = \mathrm{GELU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3.$$

加入输出投影后：

$$\mathrm{FFN}_{\mathrm{GEGLU}}(\boldsymbol{x}) = [\mathrm{GELU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3]\,\boldsymbol{W}_2.$$

不同代码库可能交换两条分支的命名，例如把 GELU 放在 $\boldsymbol{x}\boldsymbol{W}_3$ 而不是 $\boldsymbol{x}\boldsymbol{W}_1$ 上。只要明确哪一条分支经过激活、哪一条作为线性 value 分支，门控结构本身是等价的。

Python 实现：GEGLU

```python
class GEGLU(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.gate_proj = nn.Linear(d_model, d_ff, bias=False)
        self.up_proj = nn.Linear(d_model, d_ff, bias=False)
        self.down_proj = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        gate = torch.nn.functional.gelu(self.gate_proj(x))
```

```
        value = self.up_proj(x)
        return self.down_proj(gate * value)
```

从结构上看，GEGLU 已经非常接近现代 LLM 常用的 SwiGLU，区别主要在门控激活函数：

$$\text{GEGLU}: \text{GELU}, \qquad \text{SwiGLU}: \text{SiLU/Swish}.$$

SwiGLU：现代 LLM 中的重要 FFN SwiGLU 将 Swish / SiLU 与 GLU 门控结构结合。常见形式为

$$\text{SwiGLU}(\boldsymbol{x}) = \text{SiLU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3.$$

再经过输出投影得到

$$\text{FFN}(\boldsymbol{x}) = (\text{SiLU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3)\,\boldsymbol{W}_2.$$

这也是现代 Decoder-only LLM 中非常重要的一类 FFN 形式。

SwiGLU 使用三组主要线性投影：$\boldsymbol{W}_1$ 和 $\boldsymbol{W}_3$ 将隐藏维度从 d_{model} 投影到 d_{ff}，$\boldsymbol{W}_2$ 再将其投影回 d_{model}。

因此，SwiGLU 的数据流可以写成

$$\begin{aligned} \boldsymbol{x} &\longrightarrow \boldsymbol{x}\boldsymbol{W}_1 \longrightarrow \text{SiLU} \longrightarrow \text{gate}, \\ \boldsymbol{x} &\longrightarrow \boldsymbol{x}\boldsymbol{W}_3 \longrightarrow \text{value}, \end{aligned}$$

随后

$$\text{gate} \odot \text{value} \longrightarrow \boldsymbol{W}_2 \longrightarrow \boldsymbol{y}.$$

为什么需要两个上投影 传统 FFN 中只有一个上投影 $\boldsymbol{x}\boldsymbol{W}_1$.

SwiGLU 中有两条不同投影：

$$\boldsymbol{x}\boldsymbol{W}_1, \qquad \boldsymbol{x}\boldsymbol{W}_3.$$

其中一条经过 SiLU 后承担门控作用，另一条保留线性内容表示。模型因此可以学习：

- 当前 token 在哪些隐藏通道上应该产生较强响应；
- 哪些内容特征应该被门控分支增强或抑制；
- 不同输入条件下，FFN 的有效特征通路如何动态变化。

与单分支 $\sigma(\boldsymbol{x}\boldsymbol{W}_1)$ 相比，SwiGLU 的门控值与内容值来自不同的可学习投影，因此具有更灵活的表示能力。

SwiGLU 的 Tensor Shape 设 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$.

两条上投影得到

$$\boldsymbol{G} = \boldsymbol{X}\boldsymbol{W}_1, \qquad \boldsymbol{U} = \boldsymbol{X}\boldsymbol{W}_3,$$

其中 $\boldsymbol{G}, \boldsymbol{U} \in \mathbb{R}^{B\times L\times d_{\text{ff}}}$.

执行 SiLU 和逐元素乘法：

$$\boldsymbol{H} = \text{SiLU}(\boldsymbol{G}) \odot \boldsymbol{U},$$

因此 $\boldsymbol{H} \in \mathbb{R}^{B\times L\times d_{\text{ff}}}$.

最后

$$\boldsymbol{Y} = \boldsymbol{H}\boldsymbol{W}_2 \in \mathbb{R}^{B\times L\times d_{\text{model}}}.$$

完整 Shape 为

$$[B, L, d_{\text{model}}] \longrightarrow \begin{cases} [B, L, d_{\text{ff}}] & \text{gate branch,} \\ [B, L, d_{\text{ff}}] & \text{value branch} \end{cases} \longrightarrow [B, L, d_{\text{ff}}] \longrightarrow [B, L, d_{\text{model}}].$$

为什么 LLaMA 使用 SwiGLU：从 ReLU / GELU 到门控 FFN LLaMA 的 Transformer Block 没有继续沿用最早 Transformer 中的 ReLU FFN，而是采用 SwiGLU 类门控 FFN。理解这一选择时，可以从两个层次看。

第一，SiLU 是平滑的非线性函数：$\text{SiLU}(x) = x\sigma(x)$. 它不像 ReLU 那样直接把整个负半轴截断为 0。

第二，SwiGLU 不只是更换激活函数，而是引入两条不同的线性分支：

$$\text{SiLU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3.$$

因此模型拥有一个输入相关的门控机制，可以在 FFN 内部动态调节特征通过方式。

LLaMA 采用 SwiGLU，不只是把 ReLU 换成 SiLU，而是把传统单分支 FFN 改造成带可学习门控的双上投影结构。

为什么 SwiGLU 的中间维度通常小于传统四倍隐藏维度 经典 Transformer FFN 若使用 $d_{\text{ff}} = 4d_{\text{model}}$, 忽略 bias 后参数量约为

$$2d_{\text{model}}d_{\text{ff}} = 8d_{\text{model}}^2.$$

SwiGLU 有三个主要矩阵：

$$\boldsymbol{W}_1, \qquad \boldsymbol{W}_3, \qquad \boldsymbol{W}_2.$$

因此参数量约为 $3d_{\text{model}}d_{\text{ff}}$.

如果仍然使用 $d_{\text{ff}} = 4d_{\text{model}}$, 参数量将变为 $12d_{\text{model}}^2$，明显高于传统两层 FFN。

如果希望 SwiGLU 与传统 $4d_{\text{model}}$ FFN 保持近似相同的参数规模，可以令

$$3d_{\text{model}}d_{\text{ff}} \approx 8d_{\text{model}}^2.$$

两边除以 d_{model}：

$$3d_{\text{ff}} \approx 8d_{\text{model}},$$

于是得到

$$d_{\text{ff}} \approx \frac{8}{3}d_{\text{model}}.$$

因此，现代 SwiGLU 架构中经常看到中间维度接近 $\frac{8}{3}d_{\text{model}}$, 而不是经典 FFN 的 $4d_{\text{model}}$。

实际模型通常还会为了硬件计算效率，把中间维度向某个整数倍进行取整。因此，具体模型配置中的 d_{ff} 不一定严格等于 $8d_{\text{model}}/3$。

参数量对比

结构	主要权重矩阵	忽略 bias 后参数量
经典 FFN	$\boldsymbol{W}_1, \boldsymbol{W}_2$	$2d_{\text{model}}d_{\text{ff}}$
经典 $4d$ FFN	$d_{\text{ff}} = 4d_{\text{model}}$	$8d_{\text{model}}^2$
SwiGLU	$\boldsymbol{W}_1, \boldsymbol{W}_3, \boldsymbol{W}_2$	$3d_{\text{model}}d_{\text{ff}}$
近似等参数 SwiGLU	$d_{\text{ff}} \approx 8d_{\text{model}}/3$	$\approx 8d_{\text{model}}^2$

这个推导是理解 LLaMA 风格 FFN Shape 时非常重要的一步：看到 SwiGLU 的中间维度不是 $4d$，不能简单认为模型“缩小了 FFN”，因为 SwiGLU 本身多了一条上投影分支。

4.2 Python 实现：手写 SwiGLU FFN

基础实现 下面直接实现

$$\mathrm{FFN}(\boldsymbol{x}) = \left(\mathrm{SiLU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3\right)\boldsymbol{W}_2.$$

```python
class SwiGLUFFN(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.gate_proj = nn.Linear(d_model, d_ff, bias=False)
        self.up_proj = nn.Linear(d_model, d_ff, bias=False)
        self.down_proj = nn.Linear(d_ff, d_model, bias=False)
        self.act = nn.SiLU()

    def forward(self, x):
        gate = self.act(self.gate_proj(x))
        value = self.up_proj(x)
        return self.down_proj(gate * value)
```

这里可以建立如下对应关系：

$$\begin{aligned}\texttt{gate_proj} &\leftrightarrow \boldsymbol{W}_1,\\ \texttt{up_proj} &\leftrightarrow \boldsymbol{W}_3,\\ \texttt{down_proj} &\leftrightarrow \boldsymbol{W}_2.\end{aligned}$$

因此 forward 中的核心代码

```python
gate = self.act(self.gate_proj(x))
value = self.up_proj(x)
y = self.down_proj(gate * value)
```

严格对应

$$\mathrm{SiLU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3 \longrightarrow \boldsymbol{W}_2.$$

自动计算近似等参数中间维度 如果希望根据 $d_{\mathrm{ff}} \approx \frac{8}{3} d_{\mathrm{model}}$ 自动计算中间维度，并取整到某个倍数，可以定义：

```python
def round_up(x, multiple):
    return multiple * ((x + multiple - 1) // multiple)

class LlamaStyleSwiGLU(nn.Module):
    def __init__(self, d_model, multiple_of=256):
        super().__init__()
        d_ff = round_up(int(8 * d_model / 3), multiple_of)
        self.gate_proj = nn.Linear(d_model, d_ff, bias=False)
        self.up_proj = nn.Linear(d_model, d_ff, bias=False)
        self.down_proj = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        return self.down_proj(torch.nn.functional.silu(self.gate_proj(x)) * self.
    ↪ up_proj(x))
```

这里的 `multiple_of` $= 256$ 只是用于说明“将中间维度向硬件友好的整数倍取整”这一思路。具体模型应以其配置文件中的 intermediate size 为准，而不是把固定倍数视为所有 LLM 的统一标准。

完整测试

```python
def test_ffn():
    torch.manual_seed(42)

    B, L, D = 2, 16, 512
    x = torch.randn(B, L, D)

    models = {
        "ReLU FFN": TransformerFFN(D, 4 * D),
        "GELU FFN": GELUFFN(D, 4 * D),
        "GLU FFN": GLUFFN(D, 1024),
        "GEGLU": GEGLU(D, 1024),
        "SwiGLU": SwiGLUFFN(D, 1024),
        "LLaMA-style": LlamaStyleSwiGLU(D),
```

```
    }

    for name, model in models.items():
        y = model(x)
        print(f"{name:12s}: {tuple(y.shape)}")
        assert y.shape == x.shape

    print("all tests passed")

if __name__ == "__main__":
    test_ffn()
```

无论内部使用单分支 FFN 还是门控 FFN，最终都应保持

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{model}}],$$

这样才能继续与 Residual Stream 相加。

4.3 不同 FFN 与激活函数的比较

从“激活”与“门控”两个层次理解 ReLU、GELU 与 SiLU 首先属于激活函数层面的变化：

$$\begin{aligned}\text{ReLU}(x) &= \max(0, x),\\ \text{GELU}(x) &= x\Phi(x),\\ \text{SiLU}(x) &= x\sigma(x).\end{aligned}$$

它们都可以放进经典单分支 FFN：$\sigma(\boldsymbol{x}\boldsymbol{W}_1)\boldsymbol{W}_2$.

GLU、GEGLU 与 SwiGLU 则进一步改变了 FFN 的结构，由单分支变成双上投影门控：

$$[\phi(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3]\, \boldsymbol{W}_2.$$

其中 ϕ 的不同产生不同变体：

$$\begin{aligned}\text{GLU:}\quad & \phi = \text{Sigmoid},\\ \text{GEGLU:}\quad & \phi = \text{GELU},\\ \text{SwiGLU:}\quad & \phi = \text{SiLU/Swish}\,.\end{aligned}$$

因此

ReLU、GELU、SiLU 主要改变“用什么非线性”；GLU、GEGLU、SwiGLU 进一步改变“FFN 如何组织特征与门控分支”。

主要方法对比

方法	核心形式	主要特点	结构类型
ReLU	$\max(0, x)$	计算简单，负半轴硬截断	激活函数
GELU	$x\Phi(x)$	平滑缩放，Transformer 中广泛使用	激活函数
SiLU / Swish	$x\sigma(x)$	平滑、自门控形式	激活函数
GLU	$(\boldsymbol{x}\boldsymbol{W}_a) \odot \sigma(\boldsymbol{x}\boldsymbol{W}_b)$	双分支，Sigmoid 控制门	门控 FFN
GEGLU	$\text{GELU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3$	GELU 与门控结构结合	门控 FFN
SwiGLU	$\text{SiLU}(\boldsymbol{x}\boldsymbol{W}_1) \odot \boldsymbol{x}\boldsymbol{W}_3$	SiLU 与门控结构结合，现代 LLM 常见	门控 FFN

激活函数与门控结构的演进 FFN 的演进主要体现在两个方面：激活函数从 ReLU 扩展到 GELU / SiLU 等平滑形式；结构从单分支扩展为内容分支与门控分支的乘法交互。

4.4 FFN 在现代 Transformer Block 中的位置

一个典型的 Pre-Norm Decoder-only Transformer Block 由 Attention、位置编码、RMSNorm、Residual Connection 与 FFN 组成，其数据流为

$$h = x + \text{Attention}\left(\text{RMSNorm}(x)\right),$$

然后

$$y = h + \text{SwiGLU}\left(\text{RMSNorm}(h)\right).$$

如果进一步展开第二个式子：

$$y = h + \left[\text{SiLU}\left(\text{RMSNorm}(h)\boldsymbol{W}_1\right) \odot \text{RMSNorm}(h)\boldsymbol{W}_3\right]\boldsymbol{W}_2.$$

从功能角度可以分别理解为：

- Attention：在 token 之间交换信息；
- Position Encoding：向 Attention 提供顺序或相对位置信息；
- RMSNorm：控制进入子层前的隐藏状态尺度；
- SwiGLU FFN：在单个 token 内部执行门控非线性特征变换；
- Residual Connection：把子层更新累积到主 Residual Stream。

这样，前面几个章节的内容已经能够组合成一个接近现代 LLM 的完整 Transformer Block。

第 5 章

Mixture of Experts：稀疏专家模型与现代 MoE LLM

前面的 Feed Forward Network 章节已经介绍了经典 FFN、GLU、GEGLU 与 SwiGLU。对于普通 Dense Transformer，每一个 token 在经过某一层 FFN 时，都会使用同一组 FFN 参数。随着模型规模不断扩大，如果只是继续增大 Dense FFN 的隐藏维度，那么参数量、训练计算量和推理计算量通常会一起增长。

Mixture of Experts（MoE，混合专家）提供了另一条扩展路线：将一个 FFN 替换为多个 Expert，并使用 Router 根据当前 token 的隐藏状态，只选择其中少量 Expert 参与计算。这样可以在显著增加模型总参数量的同时，控制单个 token 实际激活的参数量。

设输入 token 的隐藏向量为 $\boldsymbol{x} \in \mathbb{R}^{d_{\text{model}}}$，共有 N 个 Expert。Router 首先计算 $\boldsymbol{z} = \boldsymbol{W}_r \boldsymbol{x}$，其中

$$\boldsymbol{W}_r \in \mathbb{R}^{N \times d_{\text{model}}}, \qquad \boldsymbol{z} \in \mathbb{R}^{N}.$$

然后得到路由概率 $\boldsymbol{p}(\boldsymbol{x}) = \text{softmax}(\boldsymbol{z})$. 若只选择概率最高的 K 个 Expert，则记 $\mathcal{T}(\boldsymbol{x}) = \text{TopK}(\boldsymbol{p}(\boldsymbol{x}))$. Sparse MoE 的输出为

$$\text{MoE}(\boldsymbol{x}) = \sum_{i \in \mathcal{T}(\boldsymbol{x})} g_i(\boldsymbol{x}) E_i(\boldsymbol{x}),$$

其中 E_i 表示第 i 个 Expert，$g_i(\boldsymbol{x})$ 表示经过选择与归一化后的 Gate Weight。

理解 MoE 最重要的第一件事是区分 Total Parameters 与 Active Parameters：模型可以拥有很多 Expert，但每个 token 只激活其中少数 Expert。

5.1 从 Dense FFN 到 Mixture of Experts

Dense FFN 回顾 经典 Transformer 中，每个 token 都通过相同的 FFN。若使用两层 FFN，可以写成

$$\mathrm{FFN}(\boldsymbol{x}) = \sigma(\boldsymbol{x}\boldsymbol{W}_1)\boldsymbol{W}_2.$$

若使用现代 Decoder-only LLM 中常见的 SwiGLU，则可以写成

$$\mathrm{SwiGLU}(\boldsymbol{x}) = \left(\mathrm{SiLU}(\boldsymbol{x}\boldsymbol{W}_{\mathrm{gate}}) \odot \boldsymbol{x}\boldsymbol{W}_{\mathrm{up}}\right)\boldsymbol{W}_{\mathrm{down}}.$$

Dense FFN 的特点是：对于同一层中的所有 token，都执行同一组参数。假设 FFN 参数量为 P_{ffn}，那么从 FFN 这一部分来看，每个 token 都会经过这组参数所对应的 Dense 计算。

从单一 FFN 到多个 Expert MoE 将这个单一 FFN 扩展为 $E_1, E_2, \ldots, E_N$，其中每个 E_i 都可以看作一个独立 FFN。模型不再让所有 token 固定经过同一个 FFN，而是增加一个 Router：

$$\boldsymbol{x} \longrightarrow \mathrm{Router}(\boldsymbol{x}) \longrightarrow \text{Selected Experts} \longrightarrow \mathrm{MoE}(\boldsymbol{x}).$$

Total Parameters 与 Active Parameters 如果总共有 N 个 Expert，而每个 token 只选择 K 个 Expert，通常满足 $K \ll N$。于是 Expert 部分的总参数量大致与 NP_{expert} 成正比，而单个 token 实际执行的 Expert 计算只与 KP_{expert} 直接相关。

这里必须区分两个概念：

$$P_{\mathrm{total}} \neq P_{\mathrm{active}}.$$

更准确地说，对于 Sparse MoE，通常希望达到

$$P_{\mathrm{total}} \gg P_{\mathrm{active}}.$$

维度	Dense FFN	Sparse MoE FFN
FFN 数量	每层通常只有一组主要 FFN	每层存在多个 Expert FFN
Token 路径	所有 token 使用同一 FFN	Router 为不同 token 选择不同 Expert
Router	不需要	需要
总参数量	与 Dense 层规模直接相关	可以通过增加 Expert 数量显著扩大
单 token 激活参数	基本等于该 FFN 的全部参数	主要来自被选择的 K 个 Expert
计算方式	Dense Compute	Sparse / Conditional Compute
系统复杂度	相对较低	需要 Routing、Dispatch、Gather 与跨设备通信

MoE 的核心价值并不是“让大模型参数不占内存”，而是把“模型总容量”与“单 token 实际计算量”在一定程度上解耦。

5.2 Router、Gate 与 Top-K Routing

Router Logits 与 Softmax Gate Router 决定当前 token 应该交给哪些 Expert。最简单的 Router 是一个线性投影：$\boldsymbol{z} = \boldsymbol{W}_r\boldsymbol{x} + \boldsymbol{b}_r$, 其中 $\boldsymbol{z} \in \mathbb{R}^N$. z_i 可以称为第 i 个 Expert 的 Router Logit。

若使用 Softmax Gate，则

$$p_i(\boldsymbol{x}) = \frac{\exp(z_i)}{\sum_{j=1}^{N} \exp(z_j)}.$$

因此 $\sum_{i=1}^{N} p_i(\boldsymbol{x}) = 1$.

Top-K Sparse Routing 如果直接计算

$$\sum_{i=1}^{N} p_i(\boldsymbol{x}) E_i(\boldsymbol{x}),$$

那么所有 Expert 都需要执行，计算仍然是 Dense 的。Sparse MoE 的关键步骤是只保留 Top-K Expert：

$$\mathcal{T}(\boldsymbol{x}) = \mathrm{TopK}\left(p_1(\boldsymbol{x}), \ldots, p_N(\boldsymbol{x})\right).$$

假设被选中的 Expert 索引为 $i_1, \ldots, i_K$，则常见做法是把对应 Gate Weight 再归一化：

$$g_{i_j}(\boldsymbol{x}) = \frac{p_{i_j}(\boldsymbol{x})}{\sum_{\ell=1}^{K} p_{i_\ell}(\boldsymbol{x})}.$$

于是 $\sum_{j=1}^{K} g_{i_j}(\boldsymbol{x}) = 1$.

最终输出为

$$\boldsymbol{y} = \sum_{j=1}^{K} g_{i_j}(\boldsymbol{x}) E_{i_j}(\boldsymbol{x}).$$

Router 与 Attention 的区别 Router 与 Attention 都会产生某种“权重”，但二者解决的问题完全不同。Attention 在 token 之间建立依赖；Router 在 Expert 之间选择计算路径。

维度	Attention	MoE Router
选择对象	序列中的 token / KV 位置	Expert
主要输入	Query、Key、Value	当前 token 的隐藏状态
权重意义	token 之间的信息聚合权重	Expert 的选择与混合权重
输出作用	混合不同 token 的信息	决定当前 token 执行哪些 Expert

Top-1、Top-2 与 Top-K Top-1、Top-2 与一般 Top-K 可以统一理解：

Routing	特点	主要权衡
Top-1	每个 token 只执行一个 Routed Expert	计算和通信更简单，但路由决策更集中
Top-2	每个 token 执行两个 Routed Expert	增加组合能力，也增加 Expert 计算与通信
Top-K	每个 token 执行 K 个 Routed Expert	K 越大，激活容量越大，但 Sparse 优势会逐渐减弱

Switch Transformer 的代表性设计是 Top-1 Routing；Mixtral 8x7B 在每层的 8 个 Expert 中为每个 token 选择 2 个；现代架构还会根据 Expert 粒度、Shared Expert 与系统设计使用不同的 K。

Top-K 的目的不是寻找一个“永远最好的 Expert”，而是让 Routing 依赖当前 token，使不同 token 可以动态使用不同参数子网络。

5.3 Expert FFN 与 Sparse Activation

SwiGLU Expert 在主流 Transformer MoE 中，Expert 通常不是新的 Attention 模块，而是 FFN 的多个副本或变体。若每个 Expert 使用 SwiGLU，可以写成

$$E_i(\boldsymbol{x}) = \left(\mathrm{SiLU}\left(\boldsymbol{x}\boldsymbol{W}_{\mathrm{gate},i}\right) \odot \boldsymbol{x}\boldsymbol{W}_{\mathrm{up},i}\right)\boldsymbol{W}_{\mathrm{down},i}.$$

Expert 参数量 设

$$\boldsymbol{W}_{\mathrm{gate},i}, \boldsymbol{W}_{\mathrm{up},i} \in \mathbb{R}^{d_{\mathrm{model}} \times d_{\mathrm{ff}}},$$

以及

$$\boldsymbol{W}_{\mathrm{down},i} \in \mathbb{R}^{d_{\mathrm{ff}} \times d_{\mathrm{model}}}.$$

忽略 bias 时，一个 SwiGLU Expert 的参数量约为

$$P_{\mathrm{expert}} \approx 3 d_{\mathrm{model}} d_{\mathrm{ff}}.$$

若有 N 个相同尺寸的 Expert，则 Expert 总参数量约为

$$P_{\mathrm{experts}} \approx 3 N d_{\mathrm{model}} d_{\mathrm{ff}}.$$

Router 的参数量约为

$$P_{\mathrm{router}} \approx N d_{\mathrm{model}},$$

通常远小于全部 Expert 参数量。

Sparse Activation 与 Conditional Computation 如果每个 token 只激活 K 个 Expert，那么单 token 的 Expert 主体计算近似与 $3Kd_{\mathrm{model}}d_{\mathrm{ff}}$ 同阶，而不是 $3Nd_{\mathrm{model}}d_{\mathrm{ff}}$.

这种机制称为 Sparse Activation 或 Conditional Computation：模型拥有大规模参数集合，但每次前向传播只条件化地调用其中一小部分。

概念	含义	MoE 中的作用
Expert	一个可独立执行的参数子网络，通常为 FFN	提供模型容量
Router	根据 token 计算 Expert Score	决定 token 的计算路径
Top-K	只保留最高的 K 个 Routing Score	形成稀疏激活
Active Parameters	当前 token 实际参与主要计算的参数	决定单 token 计算规模的重要指标
Total Parameters	模型中所有 Expert 与其他模块参数之和	描述模型总容量与存储规模

MoE 的 Tensor Shape、参数量与计算流程：从 Batch Sequence 到 Token Matrix 设 MoE 层输入为 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$. 为了进行 token-level Routing，通常先把 Batch 与 Sequence 两个维度展平。记 $T = BL$，则

$$\boldsymbol{X}_{\text{flat}} \in \mathbb{R}^{T\times d_{\text{model}}}.$$

Router 与 Top-K 的 Tensor Shape Router 计算

$$\boldsymbol{Z} = \boldsymbol{X}_{\text{flat}}\boldsymbol{W}_r^{\mathsf{T}},$$

其中 $\boldsymbol{Z} \in \mathbb{R}^{T\times N}$. 经过 Softmax：

$$\boldsymbol{P} = \text{softmax}(\boldsymbol{Z}), \qquad \boldsymbol{P} \in \mathbb{R}^{T\times N}.$$

Top-K 后得到两类 Tensor：

$$\boldsymbol{I} \in \mathbb{N}^{T\times K},$$

表示 Expert Index，以及

$$\boldsymbol{G} \in \mathbb{R}^{T\times K},$$

表示对应 Gate Weight。

因此 Router 的主要 Tensor Shape 可以整理为：

Tensor	Shape	含义
输入 $\boldsymbol{X}$	$[B, L, d_{\text{model}}]$	Transformer 隐藏状态
展平输入 $\boldsymbol{X}_{\text{flat}}$	$[T, d_{\text{model}}]$	$T = BL$ 个 token
Router Logits $\boldsymbol{Z}$	$[T, N]$	每个 token 对 N 个 Expert 的分数
Routing Probability $\boldsymbol{P}$	$[T, N]$	Expert 概率或 Gate Score
Top-K Index $\boldsymbol{I}$	$[T, K]$	每个 token 被分配到的 Expert
Top-K Weight $\boldsymbol{G}$	$[T, K]$	被选中 Expert 的混合权重
MoE 输出	$[B, L, d_{\text{model}}]$	与输入保持相同隐藏维度

Dispatch、Expert Compute 与 Gather 真正执行 Expert 时，系统会根据 I 把 token 分组。例如第 e 个 Expert 接收到 T_e 个 token，则其局部输入 Shape 为 $[T_e, d_{\text{model}}]$. 经过 Expert 后仍然输出 $[T_e, d_{\text{model}}]$.

随后系统再根据原 token 索引执行 Gather / Scatter，将不同 Expert 的结果加权累积回 $[T, d_{\text{model}}]$, 最后 reshape 为 $[B, L, d_{\text{model}}]$.

理论稀疏与实际效率 需要注意：虽然 Expert 理论 FLOPs 可以通过 $K \ll N$ 保持稀疏，但实际运行时间还受到 Dispatch、内存访问、负载不均衡和跨设备通信影响。因此 MoE 的“理论稀疏计算”不等于“硬件上自动获得等比例加速”。

Expert Specialization 与 Shared Expert MoE 经常被描述为让不同 Expert 学习不同“专业能力”，但这种描述需要谨慎理解。Expert 的参数并不是通常由人工直接指定为“数学 Expert”“代码 Expert”或“英语 Expert”。更一般地，Router 与 Expert 在训练中共同优化，Expert 的分工是数据、Routing 和优化过程共同形成的结果。

如果 Routing 足够有效，不同 Expert 可能在 token 类型、语言、语法模式、领域特征或更抽象的隐藏表示上形成不同程度的 Specialization。但这种分工未必对应人类可直接解释的领域标签。

Routed Expert 与 Shared Expert 传统 Routed Expert 结构可以写成

$$\boldsymbol{y} = \sum_{i \in \mathcal{T}(\boldsymbol{x})} g_i(\boldsymbol{x}) E_i(\boldsymbol{x}).$$

另一种设计是增加 Shared Expert。Shared Expert 不经过 Top-K 竞争，而是对 token 固定激活。此时可以概念化为

$$\boldsymbol{y} = E_{\text{shared}}(\boldsymbol{x}) + \sum_{i \in \mathcal{T}(\boldsymbol{x})} g_i(\boldsymbol{x}) E_i(\boldsymbol{x}).$$

这种设计希望把更通用、容易被不同 token 共同需要的表示交给 Shared Expert，而 Routed Expert 更专注于条件化的稀疏容量。

Fine-Grained Expert Segmentation DeepSeekMoE 提出了两个重要方向：一是把 Expert 进一步细粒度切分，使相同激活计算预算下可以组合更多、更小的 Expert；二是隔离 Shared Expert，用于承载共同知识并减少 Routed Expert 之间的冗余。

结构	Routed Expert	Shared Expert
是否由 Top-K 选择	是	通常否
是否对所有 token 固定执行	否	通常是
主要目标	提供条件化、稀疏的专家容量	承载较通用的共享表示
主要问题	需要 Routing 与负载均衡	会增加固定激活计算
典型组合	多个 Routed Expert 参与竞争	与 Routed Expert 并行后再合并

Shared Expert 与 Routed Expert 不是“通用知识”和“专业知识”的绝对人工分区，而是一种架构先验；实际学习到什么仍然由训练过程决定。

5.4 Load Balancing、Capacity 与 Routing Stability

Load Imbalance 与 Expert Collapse Sparse Routing 带来一个新的优化问题：如果 Router 总是偏爱少数 Expert，MoE 就无法充分利用全部参数。

例如有 8 个 Expert，理想情况下希望一个较大的 Batch 中 token 分配不要极端失衡。但训练中可能出现 $n_1 \gg n_2, n_3, \ldots, n_8$，其中 n_i 表示分配给 Expert i 的 token 数量。

这会产生多个问题：

- Hot Expert 承担过多 token，计算与通信成为瓶颈；

- 冷门 Expert 得到的训练信号太少，参数利用率低；
- Router 可能形成正反馈，使少数 Expert 越来越频繁地被选择；
- 分布式训练中，某些设备过载，而其他设备处于等待状态。

Load Balancing Objective 传统方法通常在 Language Modeling Loss 外增加 Load Balancing Auxiliary Loss：

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \mathcal{L}_{\text{balance}},$$

其中 λ 控制路由均衡目标的权重。

不同论文会采用不同的 Balance Loss 公式，因此这里更重要的是理解其目标：

$$\text{Language Modeling Quality} \quad + \quad \text{Reasonable Expert Utilization.}$$

Expert Capacity 与 Token Overflow 另一方面，Expert Capacity 描述单个 Expert 在一个 routing group 或 batch 中允许处理的 token 数量。若 Expert Capacity 为 C，但某 Expert 收到 $T_e > C$ 个 token，就会出现 Overflow。历史 MoE 系统可能采用 Token Dropping、重新路由或增加 Capacity Factor 等方式处理。

可以把常见问题统一整理为：

问题	表现	典型处理思路
Load Imbalance	少数 Expert 收到大量 token	Balance Loss、Routing Bias、全局统计等
Expert Collapse	Router 长期只使用少量 Expert	调整路由目标、初始化与正则化
Hot Expert	某些设备计算与通信明显过载	容量控制、Expert Placement、负载均衡
Token Overflow	分配 token 超过 Expert Capacity	Capacity Factor、重新路由或其他策略
Router Instability	路由分布剧烈变化或数值不稳定	Router 精度、归一化、辅助目标与训练策略

现代负载均衡策略 现代 MoE 并不一定都依赖相同形式的 Auxiliary Loss。例如 DeepSeek-V3 技术报告提出了 Auxiliary-Loss-Free 的负载均衡策略，因此不能把“MoE 必须使用某一个固定 Balance Loss”当成绝对规则。

MoE 的训练难点不只是把 FFN 复制 N 份，而是让 Router、Expert 与分布式系统长期保持稳定、有效且相对均衡的协同。

MoE Transformer Block：Dense Block 与 MoE Block MoE 在 Transformer 中通常替换的是 FFN 子层，而不是 Attention 子层。

一个现代 Dense Pre-Norm Block 可以写成

$$\boldsymbol{h} = \boldsymbol{x} + \text{Attention}\left(\text{RMSNorm}(\boldsymbol{x})\right),$$

$$\boldsymbol{y} = \boldsymbol{h} + \text{SwiGLU}\left(\text{RMSNorm}(\boldsymbol{h})\right).$$

将 FFN 替换为 MoE 后：

$$\boldsymbol{h} = \boldsymbol{x} + \text{Attention}\left(\text{RMSNorm}(\boldsymbol{x})\right),$$

$$\boldsymbol{y} = \boldsymbol{h} + \text{MoE}\left(\text{RMSNorm}(\boldsymbol{h})\right).$$

MoE 子层的数据流 所以数据流从

变成

Dense FFN 在 Norm 后对每个 token 执行同一个 FFN；MoE 则先经 Router 选择 Top-K Expert，再合并专家输出。

模块	Dense Transformer Block	MoE Transformer Block
Attention	MHA / MQA / GQA / MLA 等	同样可以使用 MHA / MQA / GQA / MLA 等
位置机制	RoPE、相对位置等	与 Dense 模型相同维度的选择
Normalization	LayerNorm / RMSNorm	LayerNorm / RMSNorm
FFN	单个 Dense FFN / SwiGLU	多个 Expert + Router
Residual	保留	保留
新增系统问题	较少	Routing、Load Balance、Dispatch、Expert Parallel

MoE 与 Attention、RoPE、RMSNorm 的关系 这也解释了为什么同一个 MoE 模型仍然可以使用 GQA、RoPE、RMSNorm、KV Cache 等前面已经学习过的机制：MoE 与这些组件并不是互相替代关系，而是处于 Transformer 的不同结构层次。

5.5 MoE 架构的发展与代表模型

从大规模 Sparse Transformer 开始 MoE 并不是近几年才出现的概念，但大语言模型时代的关键变化是：如何让 Sparse Routing 在大规模 Transformer 上更稳定、更高效，并能与现代 Decoder-only LLM 结合。

Switch Transformer：Top-1 Routing Switch Transformer 的重要贡献之一是把 Routing 简化为 Top-1 Expert。它展示了非常大的 Sparse Transformer 可以在保持相对受控单 token 计算量的同时扩展总参数规模，也突出讨论了通信开销与训练稳定性问题。

Mixtral：现代 Decoder-only Sparse MoE Mixtral 8x7B 将 Sparse MoE 带入现代开源 Decoder-only LLM 的主流视野。它的每个 MoE 层包含 8 个 FFN Expert，Router 为每个 token 选择其中 2 个 Expert，并对输出进行组合。其论文给出的模型总可访问参数约为 47B，而推理时每个 token 使用约 13B Active Parameters。

DeepSeekMoE：Fine-Grained 与 Shared Expert DeepSeekMoE 则进一步强调 Expert Specialization。它通过 Fine-Grained Expert Segmentation 把 Expert 切得更细，并引入 Shared Expert，让共享知识与 Routed Expert 的稀疏容量在结构上有所分工。

架构	Routing 特征	Expert 设计	学习重点
Switch Transformer	代表性 Top-1 Routing	Sparse FFN Expert	简化 Routing、控制通信与训练不稳定
Mixtral 8x7B	8 个 Expert 中 Top-2	Decoder-only 中的 Sparse FFN	现代开源 LLM 中的 Top-2 Sparse MoE
DeepSeekMoE	Top-K 稀疏路由	Fine-Grained Routed Expert + Shared Expert	Expert Specialization 与减少知识冗余

现代 MoE LLM：DeepSeek、Qwen 与 Llama 现代 MoE 的重要趋势不是单纯增加 Expert 数量，而是同时优化 Active Parameters、Routing、Attention、Shared Expert、长上下文和系统效率。

DeepSeek-V3 DeepSeek-V3 是一个具有 671B Total Parameters、每个 token 约 37B Activated Parameters 的 MoE 模型。其架构同时结合 Multi-head Latent Attention（MLA）

与 DeepSeekMoE，并在负载均衡上采用 Auxiliary-Loss-Free 策略。这里尤其应该注意：

$$671\text{B Total} \neq 671\text{B Active per Token.}$$

Qwen3 MoE Qwen3 同时提供 Dense 与 MoE 两条路线。代表性的 Qwen3-30B-A3B 约有 30B Total Parameters、3B Activated Parameters；Qwen3-235B-A22B 约有 235B Total Parameters、22B Activated Parameters。官方公开配置中，两者都有 128 个 Expert，并为每个 token 激活 8 个 Expert。

Llama 4 MoE Llama 4 是 Llama 系列首次采用 MoE 的一代。Llama 4 Scout 为约 109B Total、17B Active、16 Experts；Llama 4 Maverick 为约 400B Total、17B Active、128 Experts。Maverick 采用 Dense 与 MoE 层交替的设计，其 MoE 层包含 Shared Expert，并为每个 token 选择一个 Routed Expert。

现代 MoE 架构统一比较 这些模型可以用下表进行统一观察：

模型	Total / Active	Expert 信息	架构关注点
Mixtral 8x7B	约 47B / 13B	8 个 Expert，Top-2	Sparse MoE + Mistral-style Decoder
DeepSeek-V3	671B / 37B	DeepSeekMoE	MLA、Fine-Grained / Shared Expert、负载均衡
Qwen3-30B-A3B	30B / 3B	128 / 8 激活	Dense 与 MoE 并行产品路线
Qwen3-235B-A22B	235B / 22B	128 / 8 激活	大规模稀疏容量与较低 Active Parameters
Llama 4 Scout	109B / 17B	16 Experts	Llama 系列的 MoE 与原生多模态路线
Llama 4 Maverick	400B / 17B	128 Routed Experts + Shared Expert	Dense / MoE 层交替、Shared + Routed

表中的数字用于理解架构量级与 Total / Active 的区别。学习时不应只比较“总参数谁更大”，还应该同时观察：

Total Parameters, Active Parameters, K, N, Attention Type, System Cost.

判断一个 MoE 模型的计算规模时，Active Parameters 往往比 Total Parameters 更接近单 token 主体计算量；判断模型存储与部署压力时，则不能忽略 Total Parameters。

5.6 Expert Parallelism 与分布式 MoE

Expert Placement 当 Expert 数量较多时，不可能总把所有 Expert 都复制到每一张 GPU。常见做法是把不同 Expert 放到不同设备，例如

$$\begin{aligned}&\text{GPU } 0: \ E_0, E_1,\\ &\text{GPU } 1: \ E_2, E_3,\\ &\text{GPU } 2: \ E_4, E_5,\\ &\text{GPU } 3: \ E_6, E_7.\end{aligned}$$

All-to-All Dispatch 与 Gather 如果 GPU 0 上的某个 token 被 Router 分配给 E_5，这个 token 的隐藏状态就需要发送到 GPU 2。于是 MoE 的分布式前向通常包含

分布式 MoE 根据 Router 结果通过 All-to-All Dispatch 将 token 发送到对应 Expert，完成计算后再将结果返回并合并。

Expert Parallel 与其他并行方式 这种把 Expert 分布在不同设备上的并行方式称为 Expert Parallelism。它可以与 Data Parallelism、Tensor Parallelism、Pipeline Parallelism 组合。

并行方式	划分对象	在 MoE 中的作用
Data Parallelism	不同设备处理不同数据副本	提高吞吐量
Tensor Parallelism	一个大矩阵的不同维度	切分 Attention / FFN / Expert 内部矩阵
Pipeline Parallelism	不同层放在不同 Stage	处理模型深度
Expert Parallelism	不同 Expert 放在不同设备	MoE 特有的重要容量扩展手段

通信开销与负载不均衡 MoE 的系统难点之一就在于 Expert Parallelism 需要大量 token dispatch。若 Routing 不均衡，All-to-All 通信也会失衡：某些 GPU 收到大量 token，而另一些 GPU 收到很少。

因此 MoE 的实际效率是

Sparse FLOPs + Routing Cost + Communication Cost + Load Imbalance Cost

共同决定的，而不是只由 Active Parameters 决定。

MoE 推理、显存与通信开销：Total Parameter Memory 与 Active Compute 一个常见误解是："MoE 每个 token 只激活少量参数，因此没有激活的 Expert 就不占显存。"这并不成立。

若模型所有 Expert 都常驻 GPU，权重存储仍然需要覆盖 P_{total}. Sparse Routing 主要降低的是每个 token 实际执行的 Expert 计算，而不是自动删除未激活 Expert 的权重。

因此需要同时区分：Parameter Memory 与 Per-token Compute.

Expert Placement 与 Offloading 在某些部署系统中，可以把部分 Expert 放在 CPU、不同 GPU 或其他层级存储中，并根据 Routing 做 Expert Offloading，但这又会引入数据搬运和延迟问题。

Expert 权重与 KV Cache MoE 推理还需要把 Expert 参数显存与 KV Cache 显存分开理解。KV Cache 来自 Attention 的历史 Key / Value：

$$\text{KV Cache} \propto B \times L \times h_{\text{kv}} \times d_{\text{head}} \times \text{layers}.$$

而 MoE Expert 权重属于 FFN 参数存储。二者的增长来源完全不同。

推理资源统一比较

资源	主要来源	与 MoE 的关系
模型权重显存	Attention、Expert、Embedding 等参数	Total Experts 仍需要存储或分片
单 token Expert FLOPs	被 Routing 选中的 Expert	主要与 Active Experts 数量有关
KV Cache	Attention 历史 K/V	不由 Expert 数量直接决定
通信量	Token Dispatch / Gather、并行策略	Expert Parallel 时尤其重要
延迟	计算、通信、Batch、负载均衡	Sparse FLOPs 低不代表延迟必然同比降低

MoE 的优势是稀疏计算，不是“稀疏存储自动免费”。部署时必须同时考虑 Total Parameter Memory、Active Compute 和 Expert Communication。

5.7 Python 实现：最小 Sparse MoE

实现目标与数据流 该 Sparse MoE 示例的每个 Expert 使用 SwiGLU，展示 Router、Top-K 选择、Token Dispatch、Expert Compute 与加权合并；它未实现高性能 All-to-All Kernel。

SwiGLU Expert、Top-K Router 与 SparseMoE

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SwiGLUExpert(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.gate = nn.Linear(d_model, d_ff, bias=False)
        self.up = nn.Linear(d_model, d_ff, bias=False)
        self.down = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        return self.down(F.silu(self.gate(x)) * self.up(x))
```

```
14
15 class TopKRouter(nn.Module):
16     def __init__(self, d_model, n_experts, top_k):
17         super().__init__()
18         if not 1 <= top_k <= n_experts:
19             raise ValueError("top_k must satisfy 1 <= top_k <= n_experts")
20         self.n_experts = n_experts
21         self.top_k = top_k
22         self.proj = nn.Linear(d_model, n_experts, bias=False)
23
24     def forward(self, x):
25         logits = self.proj(x)
26         probs = torch.softmax(logits, dim=-1)
27         weights, indices = torch.topk(probs, self.top_k, dim=-1)
28         weights = weights / weights.sum(dim=-1, keepdim=True)
29         return weights, indices, logits
30
31 class SparseMoE(nn.Module):
32     def __init__(self, d_model, d_ff, n_experts=8, top_k=2):
33         super().__init__()
34         self.n_experts = n_experts
35         self.router = TopKRouter(d_model, n_experts, top_k)
36         self.experts = nn.ModuleList(
37             [SwiGLUExpert(d_model, d_ff) for _ in range(n_experts)]
38         )
39
40     def forward(self, x):
41         shape = x.shape
42         flat = x.reshape(-1, shape[-1])
43         weights, indices, logits = self.router(flat)
44         output = torch.zeros_like(flat)
45
46         for expert_id, expert in enumerate(self.experts):
47             token_ids, slots = torch.where(indices == expert_id)
48             if token_ids.numel() == 0:
49                 continue
50             expert_in = flat[token_ids]
51             expert_out = expert(expert_in)
52             gate = weights[token_ids, slots].unsqueeze(-1)
53             output.index_add_(0, token_ids, expert_out * gate)
```

```
        counts = torch.bincount(
            indices.reshape(-1), minlength=self.n_experts
        )
        return output.reshape(shape), logits, counts
```

Token Dispatch 与 Weighted Gather 这里的 indices Shape 为 $[T, K]$，所以 torch.where(indices == expert_id) 会同时返回 token index 与该 Expert 位于 Top-K 中的 slot index。

index_add_ 的作用是把多个 Expert 对同一个 token 的输出按照 Gate Weight 累加回原 token 位置。

该实现使用 Python for-loop 逐 Expert 分组执行，缺少 fused dispatch、Grouped GEMM、Expert Parallel 与专门通信调度，因此不适合大规模高性能训练。

Python 实现：完整 MoE Transformer Block 将 Sparse MoE 接入 Pre-Norm Transformer Block。Attention 使用 PyTorch 的 nn.MultiheadAttention，以观察 MoE 替换 FFN 的位置。

```
class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model))

    def forward(self, x):
        scale = torch.rsqrt(
            x.pow(2).mean(dim=-1, keepdim=True) + self.eps
        )
        return self.weight * x * scale

def causal_mask(seq_len, device):
    return torch.triu(
        torch.ones(seq_len, seq_len, dtype=torch.bool, device=device),
        diagonal=1,
    )

class MoETransformerBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff,
```

```
                n_experts=8, top_k=2):
        super().__init__()
        self.norm1 = RMSNorm(d_model)
        self.norm2 = RMSNorm(d_model)
        self.attn = nn.MultiheadAttention(
            d_model, num_heads, batch_first=True
        )
        self.moe = SparseMoE(
            d_model, d_ff, n_experts=n_experts, top_k=top_k
        )

    def forward(self, x):
        h = self.norm1(x)
        mask = causal_mask(x.size(1), x.device)
        attn_out, _ = self.attn(
            h, h, h, attn_mask=mask, need_weights=False
        )
        x = x + attn_out

        moe_out, router_logits, counts = self.moe(self.norm2(x))
        x = x + moe_out
        return x, router_logits, counts
```

Shape 与 Routing Assignment 测试 完整 Shape 测试如下：

```
def test_moe():
    torch.manual_seed(42)
    B, L, D = 2, 16, 64

    x = torch.randn(B, L, D)
    moe = SparseMoE(
        d_model=D,
        d_ff=128,
        n_experts=8,
        top_k=2,
    )

    y, logits, counts = moe(x)

```

```
    print("MoE input: ", x.shape)
    print("MoE output:", y.shape)
    print("router:    ", logits.shape)
    print("counts:    ", counts.tolist())

    assert y.shape == x.shape
    assert logits.shape == (B * L, 8)
    assert counts.sum().item() == B * L * 2

    block = MoETransformerBlock(
        d_model=D,
        num_heads=4,
        d_ff=128,
        n_experts=8,
        top_k=2,
    )
    out, _, block_counts = block(x)

    print("Block:     ", out.shape)
    print("Block cnt: ", block_counts.tolist())

    assert out.shape == x.shape
    print("all tests passed")

if __name__ == "__main__":
    test_moe()
```

测试结果如何理解 假设 $B = 2,\quad L = 16,\quad K = 2$, 则共有 $T = BL = 32$ 个 token，每个 token 选择 2 个 Expert，因此 Top-K Assignment 总数应为 $TK = 64$。这就是代码中 `counts.sum() == B * L * 2` 的含义。

该测试同时验证了 MoE 层保持

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{model}}]$$

的接口，因此可以直接放在 Transformer 残差结构中。

5.8 Dense 与 MoE 的比较及常见误区

Dense 与 MoE 的结构对比 Dense 与 MoE 并不是“旧架构”和“新架构”的简单替代关系。Dense 模型具有实现简单、计算规则、通信较少等优势；MoE 则用更复杂的 Routing 与系统设计换取更大的稀疏参数容量。

维度	Dense LLM	MoE LLM
FFN	单个 Dense FFN	多个 Expert FFN
Router	无	有
Token 使用参数	使用当前 Dense FFN	只使用被选中的 Routed Experts，可能再加 Shared Expert
Total vs Active	两者差距相对较小	Total 可以显著大于 Active
Load Balance	无 Expert Routing 问题	训练核心问题之一
Expert Parallel	不适用	常用
All-to-All	通常不是 FFN 主流程所需	分布式 MoE 中常见
权重存储	与 Dense 总参数对应	所有 Expert 仍需存储或分片
计算规则性	高	Routing 使 token 分布动态变化
工程复杂度	较低	较高

常见误区 常见误区包括：

常见误区	更准确的理解
“671B MoE 每个 token 都计算 671B 参数”	Total Parameters 与 Active Parameters 不同，DeepSeek-V3 报告的是 671B Total、约 37B Activated per token
“没激活的 Expert 不占显存”	Sparse Routing 减少主要计算，但 Expert 权重仍然需要存储、分片或 Offload
“每个 Expert 都有人为指定专业领域”	Expert Specialization 通常由训练动态形成，并不一定对应可解释的人类领域标签
“MoE 替换了 Attention”	Transformer MoE 通常替换 FFN；Attention 仍然独立存在
“Top-K 越大越好”	更大的 K 增加激活容量，也提高计算与通信成本
“Expert 越多越好”	Expert 数量还受到训练信号、负载、通信、显存和系统效率约束
“Active Parameters 就等于真实延迟”	实际延迟还取决于硬件利用率、Routing、Batch、通信与 Kernel 效率
“Router 与 Attention 是同一机制”	Attention 选择信息来源，Router 选择参数子网络

统一公式与四个分析问题 如果只记住一个统一公式，可以使用

$$\mathrm{MoE}(\boldsymbol{x}) = \sum_{i \in \mathcal{T}(\boldsymbol{x})} g_i(\boldsymbol{x}) E_i(\boldsymbol{x}),$$

并始终同时追问四个问题：

1. 一共有多少个 Expert，即 N 是多少？
2. 每个 token 激活多少个 Expert，即 K 是多少？
3. Router 如何选择并平衡这些 Expert？
4. Expert 如何分布到硬件上并完成 Dispatch / Gather？

只要这四个问题清楚，就可以进一步分析 Mixtral、DeepSeekMoE、DeepSeek-V3、Qwen3 MoE、Llama 4 以及后续新的 MoE 架构，而不需要把每一个模型当作完全独立的新知识点。

第 6 章

Transformer 模型类型与现代 LLM 架构演进

前面的章节已经分别讨论了 QKV 注意力机制、位置编码、Normalization 与残差连接，以及 Feed Forward Network。到这里，一个 Transformer Block 的主要内部组件已经基本齐全。本章进一步回答一个更高层的问题：**这些 Block 如何组合成完整模型，以及为什么不同模型会形成 Encoder-only、Decoder-only 和 Encoder-Decoder 三种主要结构**。

从单个 Block 的角度看，现代 Transformer 可以抽象为

$$\text{Attention} + \text{Position Information} + \text{Normalization} + \text{Residual} + \text{FFN}.$$

但从完整模型的信息流角度看，真正决定模型类型的关键并不是 FFN 使用 GELU 还是 SwiGLU，也不是位置编码使用 RoPE 还是相对位置 Bias，而是：

一个 token 在 Attention 中能够看到哪些位置，以及模型是否存在独立的 Encoder 与 Decoder。

经典 Transformer 架构可以分为三类：

$$\text{Encoder-only}, \qquad \text{Decoder-only}, \qquad \text{Encoder-Decoder}.$$

其中 BERT 是 Encoder-only 的典型代表，GPT、LLaMA、Qwen 等生成式大语言模型主要属于 Decoder-only，T5 则属于 Encoder-Decoder。

随着 LLM 的发展，Decoder-only 又进一步出现 Dense、Mixture-of-Experts（MoE）、局部与全局 Attention 混合、Multi-head Latent Attention（MLA）以及原生多模态等变化。理解这些架构时，应比较其**信息流、Attention 结构和计算组织方式**，而不是简单罗列模型名称。

6.1 从 Transformer Block 到完整模型

从 Block 组件到完整模型信息流 设输入隐藏状态为 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$. 一个现代 Pre-Norm Transformer Block 可以抽象为

$$\boldsymbol{H} = \boldsymbol{X} + \text{Attention}\left(\text{Norm}(\boldsymbol{X})\right),$$

$$\boldsymbol{Y} = \boldsymbol{H} + \text{FFN}\left(\text{Norm}(\boldsymbol{H})\right).$$

如果采用现代 Decoder-only LLM 中常见的组件，则可以进一步写成

$$\text{Norm} = \text{RMSNorm}, \qquad \text{FFN} = \text{SwiGLU}.$$

Attention 内部还可能使用 RoPE、GQA、Sliding Window、MLA 等机制。

因此，前面各章学习的组件并不是彼此独立的知识点，而是会在完整模型中被组合为统一的数据流：

完整模型先将 Token 映射为 Embedding，经 Transformer Blocks 处理，再通过 Final Norm 和 Output Head 生成 logits。

从结构图上看，Encoder Block 与 Decoder Block 都可能包含 Self-Attention、FFN、Normalization 和 Residual Connection。二者真正的区别首先来自 Self-Attention 的可见范围。

Encoder Self-Attention 通常允许位置 i 访问整个输入序列：$1, 2, \ldots, L$. 因此它是双向的。

Decoder Self-Attention 在自回归语言模型中必须满足因果约束。位置 i 只能看到 $1, 2, \ldots, i$, 而不能看到未来位置 $i+1, \ldots, L$. 因此它需要 Causal Mask。

如果模型同时具有 Encoder 与 Decoder，则 Decoder 除了 Causal Self-Attention 之外，还会增加 Cross-Attention，用于读取 Encoder 输出。

Attention Mask 与三种模型的信息流 可以用三个问题快速判断一个 Transformer 属于哪一类：

1. Self-Attention 是双向还是因果的？
2. 是否存在独立的 Encoder 与 Decoder 两套网络？
3. Decoder 是否通过 Cross-Attention 读取 Encoder 输出？

因此，可以把三种结构概括为

$$\text{Encoder-only: Bidirectional Self-Attention,}$$

$$\text{Decoder-only: Causal Self-Attention,}$$

$$\text{Encoder-Decoder: Encoder Self-Attention}$$

$$+\text{Decoder Causal Self-Attention} + \text{Cross-Attention.}$$

定义 Attention Score

$$\boldsymbol{S} = \frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}.$$

Mask 的本质是在 Softmax 之前修改可见位置：$\boldsymbol{S}' = \boldsymbol{S} + \boldsymbol{M}$. 对于允许访问的位置，可以令 $M_{ij} = 0$, 对于禁止访问的位置，可以令 $M_{ij} = -\infty$. 经过 Softmax 后，被 Mask 的位置权重趋近于 0。

因此，Mask 并不是一个简单的实现细节，而是直接决定模型的信息流方向。

6.2 三种经典 Transformer 架构

Encoder-only：双向 Self-Attention Encoder-only 模型由 Transformer Encoder Block 堆叠而成，信息流如下：

每个 token 最终都会得到一个结合左右上下文的信息表示。对于第 i 个 token，隐藏状态可以同时依赖 $x_1, \ldots, x_{i-1}, x_i, x_{i+1}, \ldots, x_L$.

这使 Encoder-only 模型非常适合需要“理解整个输入”的任务。

Encoder-only 中的 Self-Attention 通常不使用因果遮罩，因此第 i 个 Query 可以与所有 Key 计算注意力：

$$\text{Attention}_i = \sum_{j=1}^{L} \alpha_{ij} \boldsymbol{v}_j.$$

其中

$$\alpha_{ij} = \text{softmax}_j\left(\frac{\boldsymbol{q}_i^{\mathsf{T}} \boldsymbol{k}_j}{\sqrt{d_k}}\right).$$

这种双向信息流意味着 token 表示可以同时利用左侧和右侧上下文。

例如在句子 The bank is near the river 中，bank 的表示可以同时利用后面的 river 来判断语义。

BERT、RoBERTa 与 DeBERTa BERT 是 Encoder-only Transformer 的经典代表。它使用多层双向 Transformer Encoder，并通过 Masked Language Modeling 等预训练目标学习上下文化表示。

对于被 Mask 的位置，模型根据左右上下文预测原 token：$P(x_i \mid x_{\setminus i})$. 这里 $x_{\setminus i}$ 表示除目标位置以外可见的上下文。

BERT 的核心能力不是逐 token 自回归生成，而是构造高质量的双向表示。因此它天然适合分类、序列标注、抽取式问答和表示学习等任务。

RoBERTa 保留了 BERT 的 Encoder-only 主体结构，但对训练数据规模、训练策略和预训练配置进行了重新设计。它说明：即使总体架构保持不变，训练方式本身也可以显著影响模型能力。

DeBERTa 同样属于 Encoder-only 路线，但在 Attention 中显式区分 Content 与 Position，并使用解耦位置表示。前面的位置编码章节已经讨论过其 Content-to-Content、Content-to-Position 与 Position-to-Content 结构。

因此，Encoder-only 内部仍然存在丰富的架构变化，但它们共同保留一个最重要的性质：

主干 Self-Attention 是双向的，目标是得到对完整输入上下文的表示。

Encoder-only 的典型任务与局限 Encoder-only 适合：

- 文本分类；
- Token Classification；
- Named Entity Recognition；
- 向量表示与 Embedding；
- 检索与 Reranking；
- 抽取式问答。

但它不是天然的自回归文本生成结构。若希望连续生成长序列，需要额外设计输出机制，因此现代通用生成式 LLM 并不主要采用纯 Encoder-only 主干。

Decoder-only Transformer：Decoder-only 结构与 Causal Attention Decoder-only 模型只堆叠带 Causal Self-Attention 的 Decoder Block。对于位置 t，模型只能使用当前位置及之前的 token：$x_{\le t} = (x_1, x_2, \ldots, x_t)$. 然后预测下一个 token：$P(x_{t+1} \mid x_{\le t})$.

对于序列长度 L，定义 Causal Mask

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

Attention Score 为

$$S_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}} + M_{ij}.$$

当 $j > i$ 时，$S_{ij} = -\infty$, 经过 Softmax 后对应权重为 0。因此当前位置无法读取未来 token。

对于长度为 4 的序列，可见关系可以表示为

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

训练与自回归生成 Decoder-only LLM 将序列概率分解为

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t}).$$

训练时，一个序列中的多个位置可以并行计算 Next Token Prediction Loss。例如输入 $[x_1, x_2, x_3, x_4]$ 对应预测目标为 $[x_2, x_3, x_4, x_5]$.

因此虽然模型是“自回归”的，但训练阶段并不需要真的一个 token 一个 token 串行计算；Causal Mask 允许所有位置并行前向传播，同时保证每个位置看不到未来答案。

推理时则不同。假设 Prompt 为 $x_1, x_2, \ldots, x_T$, 模型首先得到 $P(x_{T+1} \mid x_{\le T})$, 采样或选择出 x_{T+1} 后，再计算 $P(x_{T+2} \mid x_{\le T+1})$.

因此生成过程为

$$\text{Prompt} \to x_{T+1} \to x_{T+2} \to \cdots.$$

这就是现代聊天模型、代码模型与通用生成式 LLM 的基础解码方式。

从 GPT 到现代 LLaMA 风格 Decoder Block GPT 系列将 Decoder-only Transformer 用于大规模自回归语言建模。其核心训练目标十分统一：给定前文预测下一个 token。

随着 GPT-2、GPT-3 等模型扩大参数量、训练数据和计算规模，Decoder-only Transformer 展现出很强的 Few-shot 与 In-Context Learning 能力。

这条路线最终形成现代生成式 LLM 的主流基础结构之一。

现代 Decoder-only LLM 已经与早期 GPT Block 有明显工程差异。一个典型的 LLaMA 风格 Block 可以抽象为

$$\boldsymbol{H} = \boldsymbol{X} + \text{Attention}\left(\text{RMSNorm}(\boldsymbol{X})\right),$$

$$\boldsymbol{Y} = \boldsymbol{H} + \text{SwiGLU}\left(\text{RMSNorm}(\boldsymbol{H})\right).$$

Attention 内部通常还会结合 RoPE，以及 MHA、GQA 等 Head 组织方式。

因此可以把前几章知识组合为

$$\text{RoPE} + \text{GQA} + \text{RMSNorm} + \text{Pre-Norm} + \text{SwiGLU}.$$

Encoder-Decoder Transformer：整体结构与三种 Attention Encoder-Decoder 同时包含两部分：

$$\text{Encoder} \quad + \quad \text{Decoder}.$$

Encoder 首先将输入序列 $x = (x_1, \ldots, x_m)$ 编码为隐藏表示

$$H^{\text{enc}} = (h_1^{\text{enc}}, \ldots, h_m^{\text{enc}}).$$

Decoder 在生成输出序列 $y = (y_1, \ldots, y_n)$ 时，一方面通过 Causal Self-Attention 读取已经生成的输出前缀，另一方面通过 Cross-Attention 读取 Encoder 表示。

Encoder 内部使用双向 Self-Attention：

$$H^{\text{enc}} = \text{Encoder}(X).$$

每个输入位置可以关注其他输入位置，从而得到完整输入语义表示。

Decoder 内部仍然需要 Causal Mask：

$$H_t^{\text{dec}} = \text{DecoderSelfAttention}(y_{\leq t}).$$

这保证生成第 $t+1$ 个输出 token 时不会读取未来目标 token。

Encoder-Decoder 最有代表性的结构是 Cross-Attention。

在 Cross-Attention 中，Query 来自 Decoder：$\boldsymbol{Q} = \boldsymbol{H}^{\text{dec}}\boldsymbol{W}^{\text{Q}}$, 而 Key 与 Value 来自 Encoder：

$$\boldsymbol{K} = \boldsymbol{H}^{\text{enc}}\boldsymbol{W}^{\text{K}}, \qquad \boldsymbol{V} = \boldsymbol{H}^{\text{enc}}\boldsymbol{W}^{\text{V}}.$$

因此

$$\text{CrossAttention} = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\text{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}.$$

这与 Self-Attention 最重要的区别是：

Self-Attention 的 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 来自同一条表示流；Cross-Attention 的 $\boldsymbol{Q}$ 来自 Decoder，而 $\boldsymbol{K}$、$\boldsymbol{V}$ 来自 Encoder。

条件生成与 T5 系列 Encoder-Decoder 的输出概率可以写成

$$P(y \mid x) = \prod_{t=1}^{n} P(y_t \mid y_{<t}, x).$$

与纯 Decoder-only 的 $P(x_t \mid x_{<t})$ 相比，这里显式存在一个条件输入 x，并通过独立 Encoder 表示输入内容。

T5 是 Encoder-Decoder Transformer 的代表模型。其重要思想是把不同 NLP 任务统一表示为 Text-to-Text：

T5 将多种 NLP 任务统一为文本输入、文本输出的条件生成形式。

翻译、摘要、问答、分类等任务都可以转化为文本条件生成。

FLAN-T5 则在 T5 路线上进一步进行指令微调，使模型能够更好地根据自然语言任务描述完成不同任务。

Decoder-only 成为大模型主流后，Encoder-Decoder 并没有消失。T5Gemma 重新探索了现代 Encoder-Decoder LLM：它以预训练 Decoder-only Gemma 模型为初始化来源，通过模型适配构造 Encoder-Decoder 结构，再进行继续预训练。

这说明一种值得注意的现代路线：

它也说明 Encoder-Decoder 与 Decoder-only 并不是互相淘汰的关系，而是适用于不同任务与计算需求的两类结构。

三种 Transformer 的 Attention 结构对比 Encoder-only 的核心是

$$Q, K, V \leftarrow H^{\text{enc}},$$

并允许任意输入位置之间建立双向依赖。

Decoder-only 同样有

$$Q, K, V \leftarrow H^{\text{dec}},$$

但 Attention Score 需要加入 Causal Mask，使 $j > i$ 的位置不可见。

Encoder-Decoder 同时包含三种 Attention 流程：

$$\text{Encoder Bidirectional Self-Attention},$$

$$\text{Decoder Causal Self-Attention},$$

$$\text{Decoder-to-Encoder Cross-Attention}.$$

因此，其单层计算结构通常比纯 Decoder-only 更复杂。

类型	Self-Attention	Cross-Attention	主要信息流
Encoder-only	双向	无	所有输入 token 相互可见
Decoder-only	因果	无	每个位置只能读取当前及过去 token
Encoder-Decoder	Encoder 双向；Decoder 因果	有	Decoder 既读取输出前缀，也读取 Encoder 表示

为什么现代生成式 LLM 通常采用 Decoder-only：统一的训练目标与输入输出接口

Decoder-only 的训练目标非常统一：

$$\mathcal{L} = -\sum_{t} \log P(x_t \mid x_{<t}).$$

网页、书籍、代码、对话以及各种结构化文本都可以转化成 token 序列，然后使用同一个目标训练。

这使预训练数据管线非常简单：

对于问答任务，可以把 Question 和 Answer 拼接为一个序列：[Question, Answer].

对于聊天模型，可以把 System、User 和 Assistant 消息都编码成统一 Chat Template。模型始终处理一条 token 流，而不需要单独定义 Encoder 输入和 Decoder 输入。

生成式模型的核心动作是

Decoder-only 的 Causal Attention 与这一过程完全匹配，因此训练和推理使用同一套信息流约束。

KV Cache、Prompt 与 In-Context Learning 自回归生成时，历史 token 的 Key 和 Value 可以缓存：$K_{1:t}, V_{1:t}$. 生成新 token 时，只需要计算新的 $Q_{t+1}, K_{t+1}, V_{t+1}$.

因此 Decoder-only 可以结合 KV Cache、GQA、MQA、PagedAttention 等工程技术高效服务。

Few-shot 示例、工具调用记录、长文档和任务指令都可以直接作为前缀放入上下文：

$$\text{Context} + \text{Instruction} + \text{Examples} + \text{Query}.$$

模型只需要继续完成下一个 token 预测，而不需要更改网络结构。

Decoder-only 的优势边界 Decoder-only 的主流地位并不意味着它对所有问题都最优。对于需要高效输入编码、固定条件生成、检索表示、分类或双向理解的任务，Encoder-only 与 Encoder-Decoder 仍然可能具有更自然的结构优势。

因此更准确的结论是：

Decoder-only 具有训练目标统一、扩展简单和自回归生成天然匹配等优势，因此成为通用生成式 LLM 的主流架构之一，而不是唯一有效的 Transformer 形式。

6.3 现代 LLM 架构的扩展

现代 Dense Decoder Block 的共同结构 早期 GPT 已经建立了 Decoder-only 自回归语言模型的基本形式，但现代 LLM 对 Block 内部组件进行了大量更新。

现代 Dense Decoder Block 常见的组件包括 RoPE、RMSNorm、SwiGLU，以及 GQA / MQA 等高效 Attention；这些是常见配置，并非严格的先后替代关系。

现代 Dense Decoder-only LLM 通常先执行 Embedding，再重复执行 Norm、Attention、Residual 与 Norm、Gated FFN、Residual，最后经过 Final Norm 和 Output Head 得到 logits。

最重要的是：Dense 描述的是每个 token 都经过相同的完整 FFN 参数，而不是说明模型一定使用哪一种 Attention。

LLaMA、Mistral、Gemma 与 Qwen 的架构变化 LLaMA 系列是现代开源 Decoder-only 架构的重要代表。其 Block 设计使 RoPE、RMSNorm、Pre-Norm 和 SwiGLU 等组件成为现代 LLM 学习中的高频知识点。

LLaMA 后续代际进一步加强了长上下文、GQA、Tokenizer 与训练规模等方面的设计，使其成为理解现代 Dense Decoder-only 的典型案例。

Mistral 7B 展示了另一条重要思路：在保持 Decoder-only 主体的同时，引入 Grouped-Query Attention 与 Sliding Window Attention，以降低长序列计算和 KV Cache 压力。

Sliding Window Attention 让某些层主要关注局部窗口，例如对于窗口大小 w，位置 i 只关注近似范围 $[i-w+1, i]$. 这与所有 token 全局互相计算 Attention 的方式不同。

Gemma 系列同样属于现代 Decoder 主干的重要代表。Gemma 3 进一步增加了局部 Attention 与全局 Attention 的交错比例，以降低长上下文下 KV Cache 的压力，同时保留周期性的全局信息交换。

现代 Dense Transformer 已不一定要求“每一层都使用完全相同的全局 Attention”。

Qwen 系列是另一条现代 Decoder-only 路线。Qwen3 同时提供 Dense 与 MoE 两类模型，其中 Dense 模型仍然可以用经典 Decoder-only 主干理解，并在 Head 数量、上下文长度和训练方式上进行扩展。

Qwen3 Dense 的 Query Head 数通常大于 KV Head 数，因此可以继续用前面学习过的 GQA 视角理解：$h_q > h_{\text{kv}}$.

从 Dense Transformer 到 Mixture-of-Experts：从 Dense FFN 到 Sparse MoE

Dense Transformer 中，每个 token 都经过同一个 FFN：$y = \text{FFN}(x)$.

MoE 则准备多个 Expert：$E_1, E_2, \ldots, E_N$, 然后通过 Router 为每个 token 选择少量 Expert。

一个简化形式为

$$y = \sum_{i \in \text{TopK}(x)} g_i(x) E_i(x),$$

其中 $g_i(x)$ 是 Router 给第 i 个 Expert 的权重。

Dense 模型增加参数量时，每个 token 的计算量通常也随之增加。MoE 希望实现：

$$\text{Total Parameters} \gg \text{Activated Parameters per Token}.$$

也就是说，模型可以拥有很大的总容量，但每个 token 只激活其中一部分参数。

因此 MoE 的核心目标不是简单“让模型更大”，而是：

把模型容量与单 token 计算量部分解耦。

Router 根据 token 隐藏状态 $\boldsymbol{x}$ 产生 Expert 分数：$\boldsymbol{r} = \boldsymbol{W}_r\boldsymbol{x}$. 再通过 Softmax 得到路由概率：$\boldsymbol{p} = \text{softmax}(\boldsymbol{r})$. 最后选择概率最高的 K 个 Expert：$\text{TopK}(\boldsymbol{p})$.

实际 MoE 还需要处理 Expert 负载均衡、通信与容量限制等问题，因此实现复杂度明显高于 Dense FFN。

Mixtral、DeepSeek、Qwen 与 Llama 4 的 MoE 路线 Mixtral 是现代 Sparse MoE 的代表之一。其思路是在 Transformer 中使用多个 FFN Expert，并让每个 token 只路由到少数 Expert。

在这些模型中，Attention Block 不一定改变，主要差异发生在 FFN：Dense 模型为每个 token 使用同一个 FFN，MoE 则由 Router 为每个 token 选择少量 Expert。

DeepSeek-V3 是现代 MoE Decoder 架构的重要案例。其技术报告描述模型具有很大的总参数量，但每个 token 只激活其中一部分参数，并结合 Multi-head Latent Attention（MLA）与 DeepSeekMoE。

DeepSeek-V3 同时在 Attention 与 FFN 路径上采用不同的优化：

$$\begin{aligned} \text{Attention Side:} &\quad \text{MLA}, \\ \text{FFN Side:} &\quad \text{DeepSeekMoE}. \end{aligned}$$

这说明现代 LLM 的架构创新往往不是只改一个组件，而是同时优化 Attention、KV Cache、FFN 与训练系统。

Qwen3 同时发布 Dense 与 MoE 模型，说明 Decoder-only 架构既可以使用 Dense FFN，也可以使用 MoE FFN：

$$\text{Decoder-only} \begin{cases} \text{Dense FFN}, \\ \text{MoE FFN}. \end{cases}$$

这也说明：

Decoder-only 描述信息流方向；Dense / MoE 描述参数与 FFN 的计算组织方式。二者属于不同分类维度。

Llama 4 是 Llama 家族首次引入 MoE 的代表性代际。Meta 对其中部分模型采用 Dense Layer 与 MoE Layer 交错的方式，并同时引入原生多模态能力。

Llama 4 的架构同时包含：

$$\text{Decoder-style Backbone} + \text{MoE} + \text{Multimodal Fusion}.$$

这进一步说明，现代“LLM 架构类型”已经不能只用 Encoder / Decoder 一个维度描述。

现代 Decoder-only 中 Attention 的进一步变化：KV Cache 驱动的 Attention 演进 在 Decoder-only 模型中，KV Head 数量直接影响 KV Cache 的规模：

$$h_q \geq h_{\text{kv}} \geq 1.$$

减少 KV Head 可以直接降低 KV Cache 存储和读取压力，因此在长上下文 LLM 中尤其重要。

MLA 关注的核心问题之一是 KV Cache。标准 MHA 需要缓存每层多个 Head 的 K/V，而 MLA 通过低维潜在表示对 Key/Value 信息进行压缩与重构，从而减少推理阶段的缓存压力。

MLA 通过低维潜在表示压缩 KV 信息；MQA / GQA 则通过减少 KV Head 数量降低缓存开销。

自回归 Decode 阶段，每生成一个 token 都需要访问历史 K/V。上下文长度增加后，KV Cache 会成为显存与带宽的重要开销。

现代 Attention 的 KV 成本优化主要有三类：减少 KV Head（GQA / MQA）、压缩 KV 表示（MLA）和限制可见范围（Sliding Window / Local Attention）。

Local、Sliding Window 与 Global Attention 标准全局 Attention 的 Score Matrix 为 $L \times L$，计算复杂度随序列长度近似二次增长。

Sliding Window Attention 让每个 token 只关注局部窗口，因此 Attention 连接数可以从近似 $O(L^2)$ 降低为 $O(Lw)$, 其中 w 是窗口大小。

完全局部 Attention 可能削弱长距离信息交换，因此一些模型采用局部与全局层交错：

Gemma 3 就是这一思想的代表案例之一。局部层负责高效处理附近 token，全局层周期性建立远距离依赖。

多模态 Transformer 对模型类型的扩展：Decoder 主干与视觉输入 “Decoder-only”描述的是 Transformer 主干中的信息流方式，而不是输入模态。

图像可以先经过 Vision Encoder 得到视觉表示：

$$I \longrightarrow H^{\text{vision}}.$$

再将这些表示映射为 LLM 可以处理的视觉 token 或特征：

$$H^{\text{vision}} \longrightarrow V_1, V_2, \ldots, V_M.$$

随后文本 token 与视觉 token 可以进入统一语言主干。

一个模型可以拥有 Decoder-style 语言主干，同时接受图像、文本甚至其他模态。

一种常见多模态结构为

多模态模型通常先由 Vision Encoder 提取图像特征，再由 Projector 将视觉维度映射到语言模型隐藏空间，最后由 LLM 融合上下文并生成文本。

其中 Vision Encoder 负责提取图像特征，Projector 负责把视觉维度映射到语言模型隐藏空间，LLM 则负责融合上下文并生成文本。

Multimodal Fusion 与代表模型 更进一步的多模态架构会在较早阶段把不同模态 token 放入统一 Backbone，使模型能够在预训练阶段直接学习跨模态关系。

Llama 4 将这一思路描述为 Early Fusion：文本 token 与视觉 token 被整合到统一模型主干中进行联合建模。

Gemma 3 与 Llama 4 都代表了开放模型从纯文本向多模态扩展的趋势，但具体融合方式不同。

对学习者而言，重要的不是记住每个视觉模块的细节，而是建立如下分类：

$$\begin{gathered}\text{Language Backbone Type,}\\ \text{Vision Encoder Type,}\\ \text{Fusion Strategy.}\end{gathered}$$

这三个维度共同决定一个多模态 LLM 的总体架构。

Encoder-Decoder 为什么仍然值得学习：输入理解、Cross-Attention 与条件生成

Encoder-Decoder 显式把 Input Understanding 和 Output Generation 分成两个网络。

Encoder 可以完整双向处理输入，而 Decoder 只负责根据输入表示进行条件生成。这种结构对于输入和输出角色非常明确的任务仍然具有吸引力。

Decoder-only 通常把输入和输出拼入同一上下文，而 Encoder-Decoder 可以先把输入编码为固定 Memory：H^{enc}. Decoder 每层通过 Cross-Attention 读取这份 Memory。

因此在某些任务中，可以更加明确地分离“输入建模”与“输出生成”。

Encoder-Decoder 长期用于：

- Machine Translation；

- Summarization；
- Conditional Generation；
- Question Answering；
- 文本转换任务。

这些任务都天然具有
结构。

T5Gemma 与现代 Encoder-Decoder 再探索 T5Gemma 说明现代 Decoder-only 预训练能力可以被重新利用到 Encoder-Decoder 架构中。这种思路模糊了“模型从一开始就必须固定属于某种结构”的边界。

可以把它理解为：Pretraining Knowledge 与 Architecture Topology 并不是完全绑定的。

Transformer 之外与 Hybrid 架构：从 Attention 长序列成本到 State Space Model 标准全局 Self-Attention 需要构造 $\boldsymbol{QK}^{\mathsf{T}} \in \mathbb{R}^{L\times L}$. 因此计算和中间 Attention Matrix 的成本随 L 增大而快速增长。

这推动了两类研究：

1. 优化 Attention 本身，例如 FlashAttention、Sliding Window、Sparse Attention；
2. 使用不同的序列建模算子，例如 State Space Model。

State Space Model（SSM）使用状态递推描述序列：

$$\boldsymbol{h}_t = \boldsymbol{A}\boldsymbol{h}_{t-1} + \boldsymbol{B}\boldsymbol{x}_t,$$

$$\boldsymbol{y}_t = \boldsymbol{C}\boldsymbol{h}_t + \boldsymbol{D}\boldsymbol{x}_t.$$

Mamba 在这一方向上引入了输入相关的选择机制，并针对现代硬件优化序列计算。

它与 Transformer 的核心区别在于：序列交互不再主要通过显式的 $L \times L$ Attention Matrix 完成。

Transformer、SSM 与 Hybrid 的分类关系 实际模型还可以把不同层组合：

$$\text{Attention Layer} + \text{SSM / Recurrent Layer}.$$

这种 Hybrid 架构试图同时利用 Attention 的灵活全局交互与线性递推结构的长序列效率。

这里必须区分两个分类维度。

Encoder-only、Decoder-only 与 Encoder-Decoder 描述 Transformer 的信息流拓扑；Transformer、SSM 与 Hybrid 则描述序列建模所采用的算子类型。

因此二者不能直接放在同一条单轴分类上。理解现代模型时，应分别回答“主干算子是什么”和“信息流如何组织”两个问题。

6.4 Python 实现：三种 Transformer 的最小结构

公共组件 最小 PyTorch 示例以 RMSNorm、Pre-Norm 和 SwiGLU 作为公共组件，再用 `nn.MultiheadAttention` 展示 Encoder-only、Encoder-Decoder 与 Decoder-only 的信息流差异。

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model))

    def forward(self, x):
        scale = torch.rsqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return self.weight * x * scale

class SwiGLU(nn.Module):
    def __init__(self, d_model, d_ff):
        super().__init__()
        self.gate = nn.Linear(d_model, d_ff, bias=False)
        self.up = nn.Linear(d_model, d_ff, bias=False)
        self.down = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        return self.down(F.silu(self.gate(x)) * self.up(x))
```

Encoder、Decoder 与 Encoder-Decoder Block Encoder-only 不使用 Causal Mask，因此所有输入位置互相可见。

```
class EncoderBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super().__init__()
        self.norm1 = RMSNorm(d_model)
        self.norm2 = RMSNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, num_heads, batch_first=True)
        self.ffn = SwiGLU(d_model, d_ff)

    def forward(self, x, padding_mask=None):
        h = self.norm1(x)
        attn_out, _ = self.attn(h, h, h, key_padding_mask=padding_mask, need_weights
    ↪ =False)
        x = x + attn_out
        x = x + self.ffn(self.norm2(x))
        return x
```

这里的 $Q = K = V = h$，并且没有 causal attention mask，因此对应双向 Encoder Self-Attention。

```
def create_causal_mask(seq_len, device=None):
    return torch.triu(torch.ones(seq_len, seq_len, dtype=torch.bool, device=device),
    ↪  diagonal=1)

class DecoderBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super().__init__()
        self.norm1 = RMSNorm(d_model)
        self.norm2 = RMSNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, num_heads, batch_first=True)
        self.ffn = SwiGLU(d_model, d_ff)

    def forward(self, x):
        h = self.norm1(x)
        mask = create_causal_mask(x.size(1), x.device)
        attn_out, _ = self.attn(h, h, h, attn_mask=mask, need_weights=False)
        x = x + attn_out
```

```
        x = x + self.ffn(self.norm2(x))
        return x
```

这里唯一决定 Decoder 信息流的关键变化是 `attn_mask=mask`.

Encoder-Decoder 的 Decoder 需要两个 Attention：一个 Causal Self-Attention，一个 Cross-Attention。

```
class EncoderDecoderBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff):
        super().__init__()
        self.norm1 = RMSNorm(d_model)
        self.norm2 = RMSNorm(d_model)
        self.norm3 = RMSNorm(d_model)
        self.self_attn = nn.MultiheadAttention(d_model, num_heads, batch_first=True)
        self.cross_attn = nn.MultiheadAttention(d_model, num_heads, batch_first=True
    ↪ )
        self.ffn = SwiGLU(d_model, d_ff)

    def forward(self, x, memory):
        h = self.norm1(x)
        mask = create_causal_mask(x.size(1), x.device)
        out, _ = self.self_attn(h, h, h, attn_mask=mask, need_weights=False)
        x = x + out

        q = self.norm2(x)
        out, _ = self.cross_attn(q, memory, memory, need_weights=False)
        x = x + out
        x = x + self.ffn(self.norm3(x))
        return x
```

Cross-Attention 对应

$$Q \leftarrow \text{Decoder}, \qquad K, V \leftarrow \text{Encoder Memory}.$$

Tensor Shape 与信息流测试

```python
def test_model_types():
    torch.manual_seed(42)
    B, L_SRC, L_TGT, D = 2, 12, 8, 64

    src = torch.randn(B, L_SRC, D)
    tgt = torch.randn(B, L_TGT, D)

    encoder = EncoderBlock(D, 4, 128)
    decoder = DecoderBlock(D, 4, 128)
    enc_dec = EncoderDecoderBlock(D, 4, 128)

    memory = encoder(src)
    decoder_out = decoder(tgt)
    enc_dec_out = enc_dec(tgt, memory)

    print("encoder:         ", memory.shape)
    print("decoder:         ", decoder_out.shape)
    print("encoder-decoder:", enc_dec_out.shape)

    assert memory.shape == src.shape
    assert decoder_out.shape == tgt.shape
    assert enc_dec_out.shape == tgt.shape

if __name__ == "__main__":
    test_model_types()
```

三种模型内部的信息流不同，但 Transformer Block 本身仍然保持隐藏维度接口：

$$[B, L, d_{\text{model}}] \longrightarrow [B, L, d_{\text{model}}].$$

6.5 三种模型类型的比较

从单一模型类型到多维架构描述

类型	Attention 可见范围	典型训练形式	代表模型
Encoder-only	输入内部双向可见	Masked / Representation Learning	BERT、RoBERTa、DeBERTa
Decoder-only	当前位置只能看到过去与当前	Next Token Prediction	GPT、LLaMA、Mistral、Qwen 等
Encoder-Decoder	Encoder 双向；Decoder 因果；包含 Cross-Attention	Conditional Generation	Transformer、T5、FLAN-T5、T5Gemma

Encoder-only 通常一次性计算完整输入表示：

$$X \longrightarrow H.$$

Decoder-only 需要自回归生成：

$$H_t \longrightarrow x_{t+1} \longrightarrow H_{t+1}.$$

Encoder-Decoder 则先一次性计算 Encoder Memory：

$$X \longrightarrow H^{\text{enc}},$$

再让 Decoder 自回归生成，并在每一步读取同一份 Encoder Memory。

分析一个现代模型时，不应只问“它是 Decoder-only 吗”。更完整的问题应该包括：

1. 信息流：Encoder-only / Decoder-only / Encoder-Decoder?
2. 参数组织：Dense / MoE?
3. Attention：MHA / GQA / MQA / MLA / Local-Global?
4. Position：RoPE / Relative Bias / 其他?
5. Norm：LayerNorm / RMSNorm? Pre-Norm / Post-Norm?
6. FFN：GELU / SwiGLU / MoE Expert?
7. Modality：Text-only / Multimodal?

例如，一个模型可以同时是

$$\text{Decoder-style} + \text{MoE} + \text{Multimodal} + \text{Efficient Attention}.$$

这比单独贴一个“Decoder-only”标签更接近现代 LLM 的真实架构描述。

现代 LLM 架构的主要变化 Transformer 按信息流结构分为 Encoder-only、Encoder-Decoder 和 Decoder-only。现代 Decoder-only LLM 常结合 GQA、局部 / 全局 Attention、长上下文、MoE、MLA 与多模态融合等技术；这些方向并非严格的线性替代关系，Dense 与 MoE 也可以并存。

第 7 章

LLM 推理与文本生成：Decoding、Sampling 与性能指标

完成预训练和 Post-Training 之后，语言模型已经能够根据上下文输出下一个 token 的 logits，但“模型会预测 logits”并不等价于“系统已经知道怎样生成文本”。真正的文本生成还需要解决两个问题。

第一，给定 Vocabulary 上的一组 logits，系统需要选择下一个 token。Greedy Search、Beam Search、Temperature、Top-K、Top-P 和 Repetition Penalty 都会影响这一选择。

第二，生成过程怎样高效执行？真实推理通常分为 Prefill 和 Decode 两个阶段，它们具有不同的计算特征。TTFT、TPOT、Latency、Throughput、Tokens/s 等指标，则从用户体验和系统吞吐两个角度描述推理性能。

Decoding / Sampling 决定“生成什么”；Prefill / Decode 与 Serving 参数决定“以什么成本生成”。

7.1 从 Logits 到最终生成文本

自回归生成的基本循环 对于 Decoder-only LLM，给定已经存在的 token 序列 $x_1, x_2, \ldots, x_t$，模型输出条件分布 $P(x_{t+1} \mid x_{\leq t})$.

实际模型首先输出 Vocabulary 上的 logits：

$$\boldsymbol{z}_t \in \mathbb{R}^{|\mathcal{V}|},$$

其中 $|\mathcal{V}|$ 表示 Vocabulary Size。

经过 token selection 后得到下一个 token x_{t+1}，再把它追加到已有序列：

$$(x_1, \ldots, x_t) \longrightarrow (x_1, \ldots, x_t, x_{t+1}).$$

生成时，模型对当前上下文执行前向计算得到 logits，再通过 Decoding / Sampling 选择并追加下一个 token，直到满足停止条件。

训练时，整个序列上的多个位置可以并行计算 Next Token Prediction Loss；生成时，未来 token 尚不存在，因此必须按时间步逐步产生。

Logits、Probability 与 Token Selection 对 logits $\boldsymbol{z} = (z_1, z_2, \ldots, z_{|\mathcal{V}|})$, Softmax 给出

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}.$$

有了概率分布后，可以采用不同策略：

- Greedy Search：选择最大概率 token；
- Beam Search：保留多条高分候选序列；
- Sampling：按照概率分布随机采样；
- Top-K / Top-P：先缩小候选集合，再采样。

这些方法不是模型参数的一部分，而是推理阶段的 decoding policy。相同的 LLM 权重，在不同 decoding 配置下可以产生明显不同的文本。

语言模型给出的是条件分布；Decoding Policy 决定如何从这个分布中产生离散 token。

7.2 Decoding 与 Sampling 的基本分类

Deterministic Search 与 Stochastic Sampling 文本生成方法可以先粗略分为两类。

类别	典型方法	核心特点
Deterministic / Search	Greedy、Beam Search	根据分数确定性选择高分路径
Stochastic Sampling	Temperature、Top-K、Top-P + Sampling	保留随机性，从概率分布中抽样

需要注意，Temperature 本身并不是一种完整搜索算法。它修改 logits 的相对尺度：

$$z_i \longrightarrow \frac{z_i}{T},$$

然后通常仍需要 Softmax 和 Sampling。

Top-K 与 Top-P 会先筛选候选 token，再从候选集合中采样。

Logits 处理顺序与框架差异 不同 Framework 中 Logits Processor、Warper 与 Filter 的执行顺序可能不同，工程使用时应以对应实现为准；理解各类操作分别如何改变 Logits 与候选集合更为重要。

Greedy Search：Greedy Decoding 与适用场景 Greedy Search 在每一步选择当前概率最大的 token：

$$x_{t+1} = \arg\max_i P(x_{t+1} = i \mid x_{\leq t}).$$

如果用 logits 表示，由于 Softmax 保持大小顺序，也可以直接写成

$$x_{t+1} = \arg\max_i z_i.$$

它的优点是：

- 实现简单；
- 每一步只保留一个候选；
- 在固定模型和输入下通常具有较高确定性；
- 不需要随机采样。

但 Greedy Search 只保证当前时间步局部最优：

$$\max_{x_{t+1}} P(x_{t+1} \mid x_{\leq t}),$$

并不保证整条序列 $P(y \mid x)$ 达到最大。

Greedy 更适合需要稳定输出的任务，例如部分结构化生成、代码补全中的确定性模式或低随机性评测。对于开放式写作，它可能产生较单一或重复的输出。

Beam Search 与序列评分 Greedy Search 每一步只保留一条路径。Beam Search 则保留 B 条高分候选，其中 B 称为 Beam Width。

对于候选回答 $y = (y_1, y_2, \ldots, y_T)$, 序列 log probability 为

$$\log P(y \mid x) = \sum_{t=1}^{T} \log P(y_t \mid x, y_{<t}).$$

Beam Search 在每一步扩展当前 Beam 中的候选序列，并根据累计分数保留新的 Top-B 路径。

Beam Width、Length Bias 与 Greedy 对比 因为每个 token 的 log probability 通常为负数，直接累加可能对长序列产生长度偏置。因此 Beam Search 实现中常见 Length Penalty 或 Length Normalization。

长度归一化评分可定义为

$$s(y) = \frac{\log P(y \mid x)}{|y|^{\alpha}},$$

其中 α 控制长度修正。具体 Framework 的公式可能不同。

增大 Beam Width 通常会增加搜索开销，但不保证生成质量单调提升。

方法	优点	局限
Greedy	快、简单、确定性强	容易陷入局部选择
Beam Search	同时探索多条高分序列	计算更高，开放式生成不一定更自然
Sampling	多样性强	存在随机性和低质量尾部风险

Temperature：概率分布锐度与生成行为 Temperature 定义为

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

当 $T = 1$ 时，保持原始 Softmax 分布。

当 $0 < T < 1$ logits 的差异被放大，概率分布更尖锐，高概率 token 更占优势。

当 $T > 1$ logits 差异被压缩，概率分布更平坦，低概率 token 获得更多采样机会。

需要注意，数学表达式要求 $T > 0$。一些推理 API 将 “temperature = 0” 特殊处理为 Greedy 或近似确定性生成，而不是直接计算 $z_i/0$.

```python
import torch

logits = torch.tensor([4.0, 2.0, 1.0])

for temperature in [0.5, 1.0, 2.0]:
    probs = torch.softmax(logits / temperature, dim=-1)
    print(temperature, probs)
```

随着 Temperature 增大，最大 logit 对应 token 的概率会下降，剩余 token 的概率质量会增加。

Top-K 与 Top-P Sampling：Top-K Sampling Top-K Sampling 只保留概率或 logits 最高的 K 个 token。

设排序后

$$z_{(1)} \geq z_{(2)} \geq \cdots \geq z_{(|\mathcal{V}|)}.$$

只保留 $\{z_{(1)}, \ldots, z_{(K)}\}$，其余位置设为不可采样状态，例如 $-\infty$。

然后在剩余候选上重新 Softmax 并采样。

K 很小时，候选空间较窄；K 增大时，更多低概率 token 可以进入候选集合。

Top-P / Nucleus Sampling Top-P 不固定候选 token 数，而是固定累计概率质量。

将 token 按概率排序：

$$p_{(1)} \geq p_{(2)} \geq \cdots .$$

找到最小集合

$$S = \{1, \ldots, m\}$$

使得

$$\sum_{i=1}^{m} p_{(i)} \geq P.$$

然后只在集合 S 中采样。

Top-P 的候选集合大小随概率分布变化：分布越尖锐，满足阈值所需的候选集合通常越小；分布越平坦，候选集合通常越大。

Top-K 与 Top-P 的统一比较

方法	控制对象	Candidate Size
Top-K	保留固定数量的最高分 token	固定为 K
Top-P	保留达到累计概率阈值的最小集合	随分布动态变化

Top-K 控制“候选数量”；Top-P 控制“候选概率质量”。

Sampling 参数的组合：Temperature、Top-K 与 Top-P 实际生成经常组合多个操作：Temperature 改变分布形状，Top-K / Top-P 再限制尾部候选。

例如较低 Temperature 和较低 Top-P 往往产生更保守的生成；较高 Temperature 配合更大的候选集合通常提高多样性，但也可能引入不稳定 token。

参数组合与生成风格 不能给所有任务规定一组统一“最佳参数”。更合理的是按任务目标理解参数。

配置倾向	常见生成行为
Low Temperature + Narrow Candidate Set	更保守、更确定
Moderate Temperature + Top-P	多样性与稳定性折中
High Temperature + Wide Candidate Set	更多随机性与创造性，也更容易采到低质量 token

具体行为还受到模型本身、Prompt、Vocabulary、训练分布和任务类型影响。

Repetition、Frequency 与 Presence Penalty：重复生成与 Logit Penalty 自回归生成可能出现 token 或短语重复：very very very very . . . 原因之一是某些已出现 token 在当前上下文下持续保持较高概率。

Repetition Penalty 会调整已生成 token 的 Logit，以降低重复生成的倾向。

不同推理 Framework 的具体数学定义并不完全相同，因此不能把某一种实现公式视为所有系统的统一标准。

Frequency、Presence 与 Repetition 的区别 一种简化的线性惩罚形式为

$$z_i' = z_i - \alpha c_i - \beta \mathbf{1}[c_i > 0] \,,$$

其中：

$$c_i = \text{token } i \text{ 已出现次数},$$

α 对应 Frequency Penalty，β 对应 Presence Penalty。

Frequency Penalty 根据 token 的出现次数调整 Logits；Presence Penalty 则只判断 token 是否出现过。

Repetition Penalty 则是更广义的一类“对已生成 token 调整 logits”的机制，具体实现可能采用乘法、除法或其他规则。

EOS、Stop Token 与生成终止：EOS、Stop Sequence 与 Length Limit EOS（End Of Sequence）通常是 Vocabulary 中的一个特殊 token。若模型生成 EOS：

$$x_{t+1} = \text{EOS},$$

推理 Runtime 可以结束当前序列。

Runtime 通常在生成文本或 token 序列中检测 Stop Sequence / Stop String；命中 Stop Pattern 后停止当前生成。

它们需要与长度限制区分：

机制	作用
EOS Token	模型生成特殊结束 token
Stop Token / Sequence	Runtime 检测指定 token 或序列
Stop String	Runtime 检测文本字符串
max_new_tokens	限制最多新生成多少 token
max_length	限制总序列长度，具体语义依 Framework 而定

完整 Decode Loop 需要在每一步检查：

EOS? Stop? Length Limit?

EOS 是模型 Vocabulary 中的特殊 token；Stop String 通常是推理系统的外部终止条件。

7.3 Python：完整 Sampling 与 Decode Loop

Sampling Functions Temperature、Top-K、Top-P 与简化 Frequency / Presence Penalty 的 PyTorch 实现如下。

```python
import torch
import torch.nn.functional as F

def apply_penalties(
    logits,
    generated_ids,
    frequency_penalty=0.0,
    presence_penalty=0.0,
):
    logits = logits.clone()

    if not generated_ids:
        return logits

    ids = torch.tensor(generated_ids, device=logits.device)
    counts = torch.bincount(ids, minlength=logits.numel()).to(logits.dtype)

    logits -= frequency_penalty * counts
    logits -= presence_penalty * (counts > 0).to(logits.dtype)
    return logits

def apply_temperature(logits, temperature=1.0):
    if temperature <= 0:
        raise ValueError("temperature must be > 0")
    return logits / temperature

def top_k_filter(logits, top_k=None):
    if top_k is None or top_k <= 0 or top_k >= logits.numel():
        return logits

    threshold = torch.topk(logits, top_k).values[-1]
    return logits.masked_fill(logits < threshold, float("-inf"))

def top_p_filter(logits, top_p=1.0):
```

```
    if top_p >= 1.0:
        return logits
    if top_p <= 0.0:
        raise ValueError("top_p must be in (0, 1]")

    sorted_logits, sorted_indices = torch.sort(logits, descending=True)
    sorted_probs = F.softmax(sorted_logits, dim=-1)
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

    remove = cumulative_probs > top_p
    remove[1:] = remove[:-1].clone()
    remove[0] = False

    filtered = logits.clone()
    filtered[sorted_indices[remove]] = float("-inf")
    return filtered

def sample_next_token(
    logits,
    generated_ids,
    temperature=1.0,
    top_k=None,
    top_p=1.0,
    frequency_penalty=0.0,
    presence_penalty=0.0,
    do_sample=True,
):
    logits = apply_penalties(
        logits,
        generated_ids,
        frequency_penalty,
        presence_penalty,
    )

    if not do_sample:
        return torch.argmax(logits).item()

    logits = apply_temperature(logits, temperature)
    logits = top_k_filter(logits, top_k)
    logits = top_p_filter(logits, top_p)
```

```
75
76      probs = F.softmax(logits, dim=-1)
77      return torch.multinomial(probs, num_samples=1).item()
```

Autoregressive Generation 为使示例可独立运行，这里使用 Toy Autoregressive Model；真实 LLM 只需提供相同接口的 logits：`model.next_logits()` 替换为模型最后一个位置的 logits。

```
1  class ToyAutoregressiveModel:
2      def __init__(self, transition_logits):
3          self.transition_logits = transition_logits
4
5      def next_logits(self, input_ids):
6          last_token = input_ids[-1]
7          return self.transition_logits[last_token]
8
9  def generate(
10     model,
11     input_ids,
12     max_new_tokens=16,
13     eos_token_id=None,
14     **sampling_kwargs,
15 ):
16     output = list(input_ids)
17
18     for _ in range(max_new_tokens):
19         logits = model.next_logits(output)
20         next_token = sample_next_token(
21             logits,
22             generated_ids=output,
23             **sampling_kwargs,
24         )
25
26         output.append(next_token)
27
28         if eos_token_id is not None and next_token == eos_token_id:
29             break
30
31     return output
```

```
32
33 if __name__ == "__main__":
34     torch.manual_seed(42)
35
36     transition_logits = torch.tensor([
37         [0.0, 3.0, 1.0, -1.0],
38         [0.0, 0.5, 3.0, -1.0],
39         [0.0, 0.5, 0.5, 4.0],
40         [0.0, 0.0, 0.0, 5.0],
41     ])
42
43     model = ToyAutoregressiveModel(transition_logits)
44
45     greedy = generate(
46         model,
47         input_ids=[0],
48         max_new_tokens=8,
49         eos_token_id=3,
50         do_sample=False,
51     )
52     print("greedy:", greedy)
53
54     sampled = generate(
55         model,
56         input_ids=[0],
57         max_new_tokens=8,
58         eos_token_id=3,
59         do_sample=True,
60         temperature=0.8,
61         top_k=3,
62         top_p=0.9,
63     )
64     print("sampled:", sampled)
65
66     assert greedy[-1] == 3
```

代码先调整 logits，再执行 Temperature、候选过滤与采样；停止条件由生成循环负责。

7.4 Prefill 与 Decode

Prefill 给定 Prompt：$x_1, x_2, \ldots, x_L$, Prefill 阶段一次处理整个 Prompt。

对 Transformer 来说，Prompt 中多个位置可以并行执行矩阵计算，同时为每一层生成历史 Key / Value：

$$K_{1:L}, \qquad V_{1:L}.$$

Prefill 输出至少包含：

$$\text{Next-token Logits} + \text{KV Cache}.$$

由于一次输入多个 token，Prefill 中矩阵乘法规模较大，通常能够获得较高并行度。在很多大型模型和现代 GPU 场景中，Prefill 更容易表现出较强的 compute pressure，但具体瓶颈仍取决于模型、Kernel、Batch、Sequence Length 和硬件。

Decode Prefill 产生第一个 token 后，进入 Decode。

每一步只新增一个 token：x_{L+t}.

由于历史 token 的 $\boldsymbol{K}$ / $\boldsymbol{V}$ 已经缓存在 KV Cache 中，新一步主要需要：

$$\boldsymbol{Q}_{\text{new}}, \quad \boldsymbol{K}_{\text{new}}, \quad \boldsymbol{V}_{\text{new}},$$

并让新的 Query 读取历史 KV。

因此 Decode 具有：

- token-by-token；
- 时间步之间存在自回归依赖；
- 单步矩阵规模相对较小；
- 频繁读取模型权重和 KV Cache；
- 在许多大模型场景中容易受到 memory bandwidth 与 memory access 影响。

两个阶段的计算差异

维度	Prefill	Decode
输入 token	整个 Prompt / Chunk	通常每序列 1 个新 token
并行度	较高	时间步之间串行
KV Cache	构建历史 KV	读取并追加新 KV
典型用户指标	TTFT 的重要组成部分	TPOT / Inter-token Latency
常见性能特征	大规模矩阵计算	Memory / KV Access 更关键

Prefill 与 Decode 虽然使用同一模型权重，但工作负载形态不同，因此高性能推理系统往往分别优化这两个阶段。

KV Cache 与增量生成：为什么需要 KV Cache 如果完全不缓存历史状态，第 t 个 Decode Step 可能重新计算整个序列：$x_1, \ldots, x_{L+t}$.

这样此前 token 的 $\boldsymbol{K}$ / $\boldsymbol{V}$ 会被反复计算。

KV Cache 保存每一层历史 token 的 Key 和 Value：

$$\boldsymbol{K}_{\text{cache}}, \qquad \boldsymbol{V}_{\text{cache}}.$$

新 token 到来时只计算：

$$\boldsymbol{K}_{\text{new}}, \qquad \boldsymbol{V}_{\text{new}},$$

然后追加：

$$\boldsymbol{K}_{\text{cache}} \leftarrow [\boldsymbol{K}_{\text{cache}}; \boldsymbol{K}_{\text{new}}],$$

$$\boldsymbol{V}_{\text{cache}} \leftarrow [\boldsymbol{V}_{\text{cache}}; \boldsymbol{V}_{\text{new}}].$$

KV Cache 减少历史 K/V 的重复计算，但会增加显存占用。

Context Length 与 KV Cache 设模型共有 N 层，KV Head 数为 H_{kv}，每个 Head Dimension 为 D_h，每个元素占 b bytes，序列长度为 L。

单层 KV Cache 的理论存储量近似为

$$M_{\text{KV,layer}} \approx 2LH_{\text{kv}}D_h b,$$

其中系数 2 来自 $\boldsymbol{K}$ 和 $\boldsymbol{V}$。

整个模型：

$$M_{\text{KV,total}} \approx 2NLH_{\text{kv}}D_h b.$$

如果还有 Batch Size B：

$$M_{\text{KV,total}} \approx 2BNLH_{\text{kv}}D_h b.$$

这也是 MQA、GQA、MLA、PagedAttention 和 KV Cache Quantization 等技术的重要出发点。

7.5 LLM 推理性能指标

Latency 与 Throughput Latency 描述一个 Request 完成需要多久。端到端 Latency 可以抽象为

$$T_{\text{E2E}} = T_{\text{queue}} + T_{\text{prefill}} + T_{\text{decode}} + T_{\text{other}}.$$

Throughput 描述单位时间内系统完成多少工作，例如：requests/s 或 tokens/s.

两者并不等价。通过扩大 Batch，系统可能提高 GPU Utilization 和总 Throughput，但单个请求也可能等待更久。

TTFT、TPOT 与 Tokens/s TTFT（Time To First Token）定义为：

$$T_{\text{TTFT}} = T_{\text{first output}} - T_{\text{request start}}.$$

它通常包含：

$$\text{Queueing} + \text{Scheduling} + \text{Prefill} + \text{First Decode Work}.$$

TPOT（Time Per Output Token）通常用于描述首 token 之后输出 token 的平均生成间隔。若输出 token 数为 N_{out}，一种常见平均形式可以写成

$$T_{\text{TPOT}} \approx \frac{T_{\text{last}} - T_{\text{first}}}{N_{\text{out}} - 1}.$$

对应单请求生成速率近似为

$$\text{tokens/s} \approx \frac{1}{T_{\text{TPOT}}},$$

前提是 TPOT 使用秒 / token。

需要区分：

- Single-request Tokens/s；
- Aggregate System Tokens/s。

后者可能同时统计多个请求，因此不能直接通过单请求 TPOT 推导。

Batch Size、Context Length 与性能：Batch Size 增大 Batch Size 通常可以让 GPU 同时处理更多请求、提高硬件利用率，但也会增加：Activation / KV Memory, 并可能增加排队延迟（Queueing Delay）。

更大的 Batch 不保证更低的单请求延迟。

服务系统真正希望实现的是：

$$\text{High Utilization} + \text{Acceptable Latency}.$$

Input / Output Length Prompt Length 增大时，Prefill 计算量与 TTFT 通常增加；Output Length 增大时，Decode Step 数增加，因此总生成延迟通常增加。

需要明确区分：

$$\text{Context Length}, \quad \text{Input Length}, \quad \text{Output Length}.$$

最大 Context Window 描述模型允许的总上下文容量，不等于每个请求实际都使用完整长度。

单请求性能与 Serving Throughput：User-facing 与 System-facing Metrics 用户通常最直接感受到：

$$\text{TTFT}, \quad \text{Inter-token Latency}, \quad \text{End-to-End Latency}.$$

服务端更关注：

$$\text{Requests/s}, \quad \text{Tokens/s}, \quad \text{GPU Utilization}, \quad \text{Queue Length}, \quad \text{Memory Usage}.$$

这些指标需要同时观察。

例如一个系统可能拥有很高的 Aggregate Tokens/s，但如果请求排队时间很长：$T_{\text{queue}} \uparrow$, 用户的 TTFT 仍然可能很差。

Latency / Throughput Trade-off 静态小 Batch：

Low Waiting but Utilization may be low.

更大 Batch：

Throughput may increase but Waiting may increase.

因此现代 Serving Runtime 会进一步使用：

Continuous Batching, Paged KV Cache, Chunked Prefill,

动态组织请求。这些内容将在后续高性能推理章节继续展开。

7.6 生成策略与推理性能的统一理解

Sampling Quality 生成质量侧主要关注：

Temperature, Top-K, Top-P, Penalty, Search Strategy.

它们主要影响：

Diversity, Determinism, Repetition, Sequence Quality.

Prefill / Decode Performance 推理性能侧主要关注：

Batch Size, Prompt Length, Output Length, KV Cache, Scheduler.

它们主要影响：

TTFT, TPOT, Memory, Throughput.

Quality、Latency 与 Throughput 文本生成系统最终需要在多个目标之间平衡：

Quality, Latency, Throughput, Memory.

例如 Beam Search 可能扩大每个请求的计算量；更大的 Batch 可能提高系统吞吐；更长 Context 增加 Prefill 和 KV Cache 成本；更开放的 Sampling 参数则主要改变输出分布。

因此推理优化不是一个单变量问题。

Decoding Parameter 与 Serving Parameter 处在不同层次：前者主要改变输出分布，后者主要改变运行时成本与调度。

常见误区与实现注意事项 常见误区可以统一整理如下。

- Temperature 越高不代表质量越好，它只是使概率分布更平坦；
- Top-K 越大不代表质量一定越高，更大的候选集合也包含更多低概率 token；
- Top-K 与 Top-P 不完全等价，一个固定候选数量，一个固定累计概率质量；
- Beam Search 不保证在所有任务上优于 Greedy 或 Sampling；
- Repetition、Frequency 与 Presence Penalty 在不同 Framework 中可能采用不同实现；
- EOS Token 与 Stop String 不属于完全相同的终止机制；
- TTFT 与 TPOT 描述不同阶段的延迟，不能互相替代；
- Aggregate Tokens/s 高不代表单用户 TPOT 一定低；
- Prefill 与 Decode 使用相同模型，但工作负载形态不同；
- KV Cache 主要减少重复计算，但会增加显存占用；
- Batch Size 增大可能提高 Throughput，却不保证降低单请求 Latency；
- Context Window 越大不代表实际推理成本固定不变，真实成本与实际输入、输出长度有关。

第 8 章

语言模型训练：从 Next Token Prediction 到预训练

前面的章节已经从模型结构角度建立了 Transformer 的主要组件：Attention 负责 token 之间的信息交互，位置编码负责注入序列顺序，Normalization 与 Residual Connection 用于稳定深层训练，FFN 负责逐 token 的非线性特征变换，而 Encoder-only、Decoder-only 与 Encoder-Decoder 则描述了这些 Block 如何组织成完整模型。

但一个结构完整的 Transformer 还不是一个真正完成训练的语言模型。要让模型学会语言，还需要回答一条完整的训练链路：原始文本如何变成 token，模型如何输出 Vocabulary 上的预测分数，如何计算损失，梯度如何更新参数，以及当模型规模扩展到数十亿甚至更多参数时，数据量与计算量应该如何匹配。

对于现代 Decoder-only LLM，可以把训练主线概括为

预训练文本先转为 Token IDs，再经 Transformer 与 LM Head 生成 logits，最后根据目标 token 计算训练损失。

这一章真正要理解的是：LLM 的“训练”不是一个单独的 Loss 公式，而是一条从数据、概率建模、损失函数到优化与 Scaling 的完整流水线。

8.1 语言模型目标与损失函数

从 Transformer Hidden State 到训练 Loss 假设最后一个 Transformer Block 输出隐藏状态 $\boldsymbol{H} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$. 这些向量已经包含上下文信息，但它们仍位于隐藏表示空间中，并不是 Vocabulary 上的概率。为了预测下一个 token，需要把每个位置的隐藏向量投影到词表维度。设词表大小为 V，则 LM Head 执行 $\boldsymbol{Z} = \boldsymbol{H}\boldsymbol{W}_{\text{vocab}}$，其中

$$\boldsymbol{W}_{\text{vocab}} \in \mathbb{R}^{d_{\text{model}}\times V}, \qquad \boldsymbol{Z} \in \mathbb{R}^{B\times L\times V}.$$

$\boldsymbol{Z}$ 就是 Logits。对最后一维执行 Softmax 后，可以得到每个位置对 V 个 token 的概率分布。

一个 Decoder-only LLM 的训练路径可以写成

$$\begin{aligned}&\text{Token IDs } [B, L] \longrightarrow \text{Embedding } [B, L, d_{\text{model}}]\\ &\longrightarrow \text{Transformer Blocks } [B, L, d_{\text{model}}] \longrightarrow \text{LM Head } [B, L, V]\\ &\longrightarrow \text{Cross Entropy Loss} \longrightarrow \text{Backward} \longrightarrow \text{Optimizer Step}.\end{aligned}$$

其中模型结构决定如何计算 Hidden State，而语言模型目标决定这些 Hidden State 应该学习成什么样的表示。

训练与推理的区别 训练时，一个长度为 L 的序列可以在 Causal Mask 约束下并行计算所有位置：位置 t 只读取 $x_{<t}$，但所有 t 的计算可以同时在 GPU 上执行。

推理时则不同。当模型需要生成未知的下一个 token 时，必须先得到 x_t，再把它作为下一步输入的一部分继续生成 x_{t+1}. 因此自回归生成天然具有逐 token 的时间依赖。

训练可以并行预测序列中的多个 next token；推理却必须把模型自己刚生成的 token 继续作为后续上下文。

自回归语言模型：Chain Rule、Causal Mask 与 Decoder-only 建模 设一个 token 序列为 $x_1, x_2, \ldots, x_T$. 根据概率论中的链式法则，联合概率可以分解为

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t}),$$

其中 $x_{<t} = (x_1, \ldots, x_{t-1})$.

这个分解不要求模型一次性直接建模整个联合分布，而是把任务转化为一系列条件概率预测：给定过去 token，预测当前 token。

取对数后有

$$\log P(x_1, \ldots, x_T) = \sum_{t=1}^{T} \log P(x_t \mid x_{<t}).$$

最大化序列似然等价于最小化 Negative Log Likelihood：

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(x_t \mid x_{<t}).$$

自回归概率分解要求位置 t 的预测不能读取未来 token，否则模型在训练时就会提前看到正确答案。因此 Decoder-only Transformer 使用 Causal Mask，使

$$\text{Attention}(t, j) = 0, \qquad j > t.$$

从概率建模角度，自回归语言模型与因果 Self-Attention 是相互对应的：

$$P(x_t \mid x_{<t}) \quad \Longleftrightarrow \quad \text{position } t \text{ only attends to positions } \leq t.$$

Next Token Prediction：Shifted Labels 与 Tensor Shape 考虑原始 token 序列 $[x_1, x_2, x_3, x_4, x_5]$. 如果模型输入为 $[x_1, x_2, x_3, x_4]$, 训练目标就是 $[x_2, x_3, x_4, x_5]$. 也就是说，输入和标签只相差一个位置。

在完整序列训练中，通常把同一个 Tensor 同时看作输入与标签，再在计算 Loss 时执行 shift：shifted logits $= \boldsymbol{Z}[:, 0 : L-1, :]$, shifted labels $= \boldsymbol{X}[:, 1 : L]$.

因此位置 t 的输出用于预测位置 $t+1$ 的 token。

设 input_ids $\in \{0, \ldots, V-1\}^{B \times L}$. 模型输出 logits $\in \mathbb{R}^{B \times L \times V}$. Shift 后得到 shift_logits $\in \mathbb{R}^{B \times (L-1) \times V}$, 以及 shift_labels $\in \{0, \ldots, V-1\}^{B \times (L-1)}$.

为了使用常见 Cross Entropy 接口，通常进一步 reshape 为 $[B(L-1), V]$ 和 $[B(L-1)]$。

Next Token Prediction 最常见的实现错误不是公式本身，而是 Logits 与 Labels shift 的方向错一位。

Teacher Forcing Teacher Forcing 指训练时把真实历史 token 作为模型上下文，而不是逐步把模型自己的采样结果喂回模型。

例如训练序列为 $A\ B\ C\ D$. 当模型学习预测 D 时，它看到的历史上下文是数据中的真实 $A\ B\ C$, 而不是模型之前可能错误生成出的 token。

由于所有真实 token 在训练样本中已经存在，因此可以一次把整个序列输入 Transformer，并使用 Causal Mask 阻止未来信息泄漏。这就允许 GPU 并行计算多个位置的条件概率。

推理时则逐步预测并追加下一个 token，直到满足停止条件。因此 Teacher Forcing 与 Causal Mask 并不冲突：前者决定训练时历史 token 来自真实数据，后者决定每个位置能够读取哪些历史位置。

Causal Language Modeling Causal Language Modeling（CLM）就是把自回归条件概率作为语言模型训练目标。对长度为 T 的序列，其平均 Loss 常写成

$$\mathcal{L}_{\text{CLM}} = -\frac{1}{T-1}\sum_{t=1}^{T-1}\log P(x_{t+1} \mid x_{\le t}).$$

对于 GPT、LLaMA、Qwen 等 Decoder-only 模型，这种目标与架构的信息流天然一致：模型训练的是“给定左侧上下文预测右侧下一个 token”。

如果序列包含 Padding 或需要忽略的 token，还可以定义 Mask $m_t \in \{0,1\}$：

$$\mathcal{L} = -\frac{\sum_t m_t \log P(x_{t+1} \mid x_{\le t})}{\sum_t m_t}.$$

这种形式也为后面的 SFT Loss Mask 提供了基础。

Masked Language Modeling Masked Language Modeling（MLM）采用不同思路。给定原始序列 $x_1, x_2, \ldots, x_T$，选择一个位置集合 M，将这些 token 替换或扰动，然后让模型根据两侧上下文恢复被遮挡内容：

$$\mathcal{L}_{\text{MLM}} = -\sum_{t\in M}\log P(x_t \mid x_{\setminus M}).$$

BERT 是 MLM 路线的经典代表。由于 Encoder Self-Attention 可以同时看到左右两侧上下文，MLM 非常适合学习双向语义表示。

CLM 根据左侧上下文预测下一个 token；MLM 则利用双向上下文预测被 Mask 的 token。

BERT 不天然执行逐 token 自回归生成，因为其训练目标和双向 Attention 都不是按照“只看过去预测未来”构造的。

Prefix Language Modeling Prefix Language Modeling 在一个序列中组合双向 Prefix 与因果生成区域。设前 k 个 token 构成 Prefix，则 Prefix 内部可以相互可见，而生成区域的第 t 个位置只能读取 Prefix 与已经出现的生成 token。

其可见性可以抽象为

$$\text{Prefix: bidirectional}, \qquad \text{Generation: causal}.$$

因此 Prefix LM 的 Attention Mask 不再是纯下三角结构，而是左上角 Prefix 区域全可见，同时生成区域保持因果约束。

这种设计可以理解为把“输入条件”与“生成目标”放在同一个 Transformer 序列中，而不一定显式使用独立 Encoder 与 Decoder。

三种语言模型目标的统一比较

目标	Attention 可见范围	Loss 位置	典型用途 / 模型
CLM	只能读取当前位置及左侧历史	通常几乎所有有效 next-token 位置	自回归生成；GPT、LLaMA、Qwen
MLM	双向上下文	被 Mask 的位置	表示学习；BERT
Prefix LM	Prefix 双向，生成区域因果	生成目标位置	条件生成、统一输入输出格式

从信息流角度看，三种目标真正改变的是“每个位置能够看到什么”以及“哪些位置参与 Loss”。

LM Head 与 Vocabulary Projection 设最终隐藏状态为 $\boldsymbol{H} \in \mathbb{R}^{B\times L\times d_{\text{model}}}$. 语言模型需要对每个位置预测 V 个候选 token，因此使用 Vocabulary Projection：$\boldsymbol{Z} = \boldsymbol{H}\boldsymbol{W}_{\text{LM}} + \boldsymbol{b}$, 其中 $\boldsymbol{W}_{\text{LM}} \in \mathbb{R}^{d_{\text{model}}\times V}$. 于是 $\boldsymbol{Z} \in \mathbb{R}^{B\times L\times V}$.

对于第 b 个样本、第 t 个位置，向量 $\boldsymbol{z}_{b,t} \in \mathbb{R}^V$ 给出了词表中每个 token 的一个未归一化分数。

Tokenizer 把每个 token 映射成 $0, 1, \ldots, V-1$. 因此模型最终必须为这 V 个离散类别分别提供一个分数。LM Head 本质上就是一个 V 类分类器，只不过同一个分类器会应用到序列中的每一个位置。

Logits Logit 是 Softmax 之前的实数分数。设某位置有 $\boldsymbol{z} = (z_1, z_2, \ldots, z_V)$. Logit 不需要满足非负，也不需要总和为 1。例如 $[-2.1,\ 0.3,\ 4.7]$ 是合法 Logits，但不是概率。

真正重要的是不同 Logit 之间的相对差值。Softmax 满足 $\frac{p_i}{p_j} = \mathrm{e}^{z_i - z_j}$, 因此两个 Logit 相差越大，它们对应概率之比也会指数级扩大。

训练阶段 Cross Entropy 直接使用 Logits；生成阶段则可以在 Logits 上应用 Temperature、Top-k、Top-p 等解码策略，然后再得到采样分布。

Softmax 对于 V 个 Logits，Softmax 定义为

$$p_i = \frac{\mathrm{e}^{z_i}}{\sum_{j=1}^{V} \mathrm{e}^{z_j}}.$$

显然

$$p_i > 0, \qquad \sum_{i=1}^{V} p_i = 1,$$

因此可以把输出解释为离散概率分布。

直接计算 e^{z_i} 可能发生数值溢出。Softmax 对整体平移不敏感：

$$\mathrm{softmax}(z) = \mathrm{softmax}(z - c).$$

实际实现通常取 $c = \max_i z_i$，从而计算

$$p_i = \frac{\mathrm{e}^{z_i - z_{\max}}}{\sum_j \mathrm{e}^{z_j - z_{\max}}}.$$

这就是常见的数值稳定 Softmax。

Log Softmax 与 Negative Log Likelihood Log Softmax 为

$$\log p_i = z_i - \log \sum_j \mathrm{e}^{z_j}.$$

与先计算 Softmax 再取对数相比，把两个步骤融合可以获得更好的数值稳定性。

如果正确类别为 y，则单个 token 的 Negative Log Likelihood 为

$$\mathcal{L}_{\mathrm{NLL}} = -\log p_y.$$

当模型给正确 token 的概率趋近于 1 时，$-\log p_y \to 0$；反之，如果正确 token 的概率很小，Loss 会迅速增大。

Cross Entropy Loss 设真实标签是 One-hot 分布 q，模型预测概率为 p，Cross Entropy 定义为

$$H(q, p) = -\sum_{i=1}^{V} q_i \log p_i.$$

如果真实类别是 y，One-hot 标签只有 $q_y = 1$，其他位置均为 0，因此 $H(q, p) = -\log p_y$.

这说明分类任务中的 Cross Entropy 与正确类别的 Negative Log Likelihood 等价。

在 LLM 训练中，最终 Loss 通常对所有有效 token 的 Cross Entropy 取平均：

$$\mathcal{L} = \frac{1}{N}\sum_{n=1}^{N} \mathrm{CE}(z_n, y_n).$$

PyTorch 的 CrossEntropyLoss 直接接收原始 Logits；不要先手动 Softmax，再把概率传给 CrossEntropyLoss。

Shifted Labels 与 Cross Entropy 的完整实现

```python
import torch
import torch.nn.functional as F

def causal_lm_loss(logits, input_ids, ignore_index=-100):
    shift_logits = logits[:, :-1, :].contiguous()
    shift_labels = input_ids[:, 1:].contiguous()

    return F.cross_entropy(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
        ignore_index=ignore_index,
    )
```

如果 Padding Token 不应该参与训练，可以在构造 labels 时把对应位置设为 -100，因为 PyTorch Cross Entropy 常用 `ignore_index=-100` 忽略这些位置。

需要区分两个 Mask：Causal Mask 控制 Attention 能看到什么；Loss Mask 控制哪些目标 token 进入 Loss。二者作用阶段不同。

Weight Tying 设 Token Embedding Matrix 为 $\boldsymbol{E} \in \mathbb{R}^{V \times d_{\text{model}}}$. 输入 token id i 时，通过第 i 行得到 embedding：$\boldsymbol{e}_i = \boldsymbol{E}_i$.

如果使用行向量记法，LM Head 可以写成 $\boldsymbol{Z} = \boldsymbol{H}\boldsymbol{E}^{\mathsf{T}}$. 这意味着输入 Embedding 与输出 Vocabulary Projection 共享同一组参数，这就是 Weight Tying。

若不共享，Embedding 与 LM Head 大约需要 $2Vd_{\text{model}}$ 个权重；共享后则约为 Vd_{model}。

Weight Tying 同时使“token 的输入表示空间”和“输出分类空间”具有直接参数联系。不过它是一种架构选择，并不是所有模型都必须采用。

Python 实现：最小 Causal Language Model Loss 下面的最小示例验证 Softmax、NLL 与 Cross Entropy 之间的关系。

```python
def manual_cross_entropy(logits, labels):
    log_probs = F.log_softmax(logits, dim=-1)
    selected = log_probs.gather(-1, labels.unsqueeze(-1)).squeeze(-1)
    return -selected.mean()

def test_cross_entropy():
    torch.manual_seed(42)
    logits = torch.randn(12, 100)
    labels = torch.randint(0, 100, (12,))

    loss_manual = manual_cross_entropy(logits, labels)
    loss_torch = F.cross_entropy(logits, labels)

    print(loss_manual.item())
    print(loss_torch.item())
    assert torch.allclose(loss_manual, loss_torch, atol=1e-6)
```

由此可以直接观察

$$\text{CrossEntropy} = \text{LogSoftmax} + \text{NLL}\,.$$

8.2 大规模预训练：数据、优化与计算规模

一个训练 Step 依次执行 Forward、Loss 计算、Backward 和 Optimizer Step。

在大规模预训练中，更常用“Step”和“训练 Token 数”而不是传统监督学习中的 Epoch 描述训练进度。原因是训练数据往往极大，而且可能来自多个数据源按权重持续采样，不一定存在简单、固定的一次完整遍历。

如果 Global Batch 中一共有 B 条长度为 L 的序列，则每步处理 token 数近似为 $T_{\text{step}} = B \times L$. 多设备、Gradient Accumulation 情况下还需要乘以对应因子。

预训练数据采集 LLM 预训练数据通常来自多种来源，例如 Web 文本、书籍、百科、代码、论文与技术文档、多语言语料以及特定领域数据。

不同数据源承担的作用不同。通用 Web 数据提供广泛知识覆盖，代码与数学数据强化结构化推理和形式语言能力，高质量书籍与技术文档能够增加长文本连贯性和专业表达，多语言数据则决定模型跨语言覆盖范围。

数据采集阶段还需要记录数据来源、许可证、抓取时间和处理版本。工程上，能够追踪一个训练样本“来自哪里、经过哪些过滤”非常重要，因为后续的数据治理、去重与问题追溯都依赖这些元数据。

数据清洗 原始数据不能直接进入训练。典型清洗过程包括：

- 去除 HTML Boilerplate、导航栏、脚注模板等非正文内容；
- 修复或过滤编码错误与乱码；
- 过滤极短、极长或异常重复的文本；
- 进行语言识别；
- 删除广告、垃圾文本和低信息密度页面；
- 对异常符号比例、重复字符比例等质量指标进行过滤。

数据清洗的目标不是单纯“减少数据”，而是提高单位训练 Token 中有效信息的密度。

数据去重 重复数据会让某些内容被模型过度重复看到，也可能造成 Benchmark、Validation Set 与训练集之间的数据泄漏。

Exact Deduplication 可以通过 Hash 判断完全相同文档。Near-Duplicate 则更复杂，常通过 N-gram、MinHash、局部敏感哈希等方法寻找高度相似文本。

可以把去重粒度分成：

$$\text{Document-level} \quad / \quad \text{Paragraph-level} \quad / \quad \text{Near-Duplicate}.$$

严格去重并不意味着必须删除所有相似表达，因为自然语言本身存在合理重复。真正需要控制的是模板复制、镜像站点、机器拼接内容等大规模冗余。

数据质量过滤与数据混合 不同数据源的质量和作用不同，因此训练集通常不是“全部数据简单拼接”，而是使用数据混合比例控制采样概率。

假设有 K 个数据源，第 k 个数据源的采样权重为 w_k，满足

$$w_k \geq 0, \qquad \sum_{k=1}^{K} w_k = 1.$$

每个 Batch 的样本来源可以根据这些权重采样。

质量过滤可以使用启发式规则，也可以使用一个质量分类器或评分模型。实际目标是在 General Text、Code、Math、Multilingual、Domain Data 等能力之间找到合理平衡。

预训练数据的关键不是“越多越好”，而是数量、质量、覆盖范围与混合比例共同决定最终训练信号。

Token 数量与训练数据规模 LLM 的数据规模通常使用 Token 而不是字符或单词衡量，因为真正进入模型的是 Token IDs。

Tokenizer 不同，同一段文本得到的 Token 数也可能不同。因此报告“1 万亿 Token”时，它隐含了一个确定的 Tokenizer。

单个训练 Step 的 Global Tokens 可以写成

$$T_{\text{step}} = B_{\text{micro}} \times L \times G \times A,$$

其中：

- B_{micro} 为每个设备 Micro Batch Size；
- L 为 Sequence Length；
- G 为数据并行设备数量；
- A 为 Gradient Accumulation Steps。

总训练 Token 数近似为

$$T_{\text{total}} = T_{\text{step}} \times N_{\text{steps}}.$$

Sequence Packing 如果每个样本长度差异很大，直接 Padding 到固定长度会浪费大量计算。例如最大长度为 4096，而很多文档只有几百 token，则大量位置都是 Padding。

Sequence Packing 会把多个短文档拼接到一个固定长度训练序列中：

$$\text{doc}_1 \parallel \text{EOS} \parallel \text{doc}_2 \parallel \text{EOS} \parallel \cdots.$$

这样可以提高有效 Token 比例。实现时需要明确 Document Boundary、EOS、Attention Mask 与 Loss Mask 的行为。某些训练实现允许跨文档因果 Attention，另一些会通过 Mask 阻止不同文档互相注意，两者属于不同工程策略。

Packing Efficiency 可以粗略理解为

$$\eta = \frac{\text{有效训练 Token 数}}{\text{实际计算 Token 位置数}}.$$

Adam 优化器：一阶矩与二阶矩 设第 t 步梯度为 $g_t = \nabla_\theta \mathcal{L}_t$。Adam 维护梯度的一阶指数移动平均

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t,$$

以及平方梯度的二阶移动平均

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2.$$

经过 Bias Correction：

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}.$$

参数更新为

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Adam 同时利用梯度方向的移动平均和各参数梯度尺度，因此长期以来是 Transformer 训练中最常见的优化器基础之一。

AdamW 与 Weight Decay AdamW 的关键是把 Weight Decay 与 Adam 的梯度归一化过程解耦。一个简化更新可以表示为

$$\theta_{t+1} = (1-\eta\lambda)\theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

其中 λ 为 Weight Decay 系数。

与“把 $\lambda\theta$ 直接加进梯度”相比，Decoupled Weight Decay 不会让正则化项再经过 Adam 的自适应二阶缩放。

实际 Transformer 训练中，常把 Norm 的 scale 参数和 bias 从 Weight Decay 中排除，而对大部分 Linear Weight 使用 Weight Decay。具体规则取决于模型与训练配置。

Learning Rate 与 Warmup Learning Rate 决定每次参数更新的步长。过大可能导致 Loss 发散，过小则训练效率低。

大模型训练初期，参数、Optimizer Moment 与激活统计都还没有稳定，因此常先使用 Warmup。Linear Warmup 可以写成

$$\eta_t = \eta_{\max} \frac{t}{T_{\text{warmup}}}, \qquad 0 \le t \le T_{\text{warmup}}.$$

Warmup 结束后 Learning Rate 到达 Peak Learning Rate，再进入衰减阶段。

Cosine Learning Rate Decay Warmup 后常使用 Cosine Decay。设衰减阶段归一化进度为

$$p = \frac{t - T_{\text{warmup}}}{T - T_{\text{warmup}}}, \qquad 0 \le p \le 1.$$

则可以写成

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos(\pi p)\right).$$

整体路线为

$$\text{Warmup} \nearrow \eta_{\max} \searrow \text{Cosine Decay}.$$

这种 Schedule 在训练前期保守增加步长，在中期维持有效优化，在后期逐渐降低更新幅度。

Gradient Clipping 如果全局梯度范数过大，单次更新可能使参数跨越到不稳定区域。设所有参数梯度拼接后的全局范数为 $\|g\|_2$. 若最大允许范数为 c，Clip by Norm 可以写成

$$g \leftarrow \begin{cases} g, & \|g\|_2 \le c, \\ g\dfrac{c}{\|g\|_2}, & \|g\|_2 > c. \end{cases}$$

PyTorch 中通常使用

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

它应该在梯度已经计算完成之后、Optimizer Step 之前执行。

Gradient Accumulation 显存可能无法容纳目标 Global Batch。Gradient Accumulation 会把一个大 Batch 拆成多个 Micro Batch，连续执行多次 backward，只在累计若干步后执行一次 optimizer step。

有效 Batch Size 为

$$B_{\text{effective}} = B_{\text{micro}} \times A \times G.$$

如果每个 Micro Batch 的 Loss 都采用 mean reduction，常见实现会先除以 Accumulation Steps：

$$\mathcal{L}_{\text{scaled}} = \frac{\mathcal{L}}{A}.$$

这样累计后的梯度尺度近似对应一个大 Batch 的平均 Loss。

Mixed Precision Training 大模型训练会大量使用低精度计算来降低显存、提高吞吐并利用现代加速器的低精度矩阵单元。

常见数值格式包括

$$\text{FP32}, \qquad \text{FP16}, \qquad \text{BF16}.$$

Mixed Precision 并不意味着模型所有计算和状态都必须使用同一低精度格式。工程实现可能让矩阵乘使用 BF16/FP16，而部分归一化、Loss、Optimizer State 或累加过程保持更高精度。

FP16 与 BF16 FP16 和 BF16 都使用 16 bit，但位分配不同。FP16 有更多 Mantissa 位，因此局部精度较高，但 Exponent 范围较小；BF16 保留与 FP32 相同数量级的 Exponent 位，因此动态范围更大。

FP16 更容易出现 Underflow 或 Overflow，所以训练时常结合 Loss Scaling。其思想是先把 Loss 放大为 $\mathcal{L}' = s\mathcal{L}$，让微小梯度进入 FP16 可表示范围，再在更新前把梯度除回尺度 s。

BF16 的动态范围更接近 FP32，因此现代支持 BF16 的硬件上，大规模 Transformer 训练经常优先采用 BF16，通常也不需要像 FP16 那样依赖 Loss Scaling。

一个完整的预训练 Step 训练 Step 通常依次执行加载 Batch、Forward、Causal LM Loss、Backward、Gradient Clipping、Optimizer Step，并按计划更新 Learning Rate。

如果使用 Gradient Accumulation，则 Optimizer Step 与 Scheduler Step 只在完成指定数量 Micro Batch 后执行。

常见代码顺序为：

```
loss = model(batch)
(loss / accumulation_steps).backward()

if should_update:
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_grad_norm)
    optimizer.step()
    scheduler.step()
```

```
    optimizer.zero_grad(set_to_none=True)
```

Python 实现：最小预训练循环 Tiny Causal LM 示例包含 Embedding、Causal Transformer、LM Head、Weight Tying、AdamW、Warmup + Cosine、Gradient Accumulation 与 Gradient Clipping，用于展示训练数据流，不是高性能训练框架。

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class RMSNorm(nn.Module):
    def __init__(self, d_model, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(d_model))

    def forward(self, x):
        scale = torch.rsqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return self.weight * x * scale

class TinyCausalLM(nn.Module):
    def __init__(self, vocab_size=256, d_model=128, num_heads=4, num_layers=2, d_ff
    ↪ =512):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=num_heads,
            dim_feedforward=d_ff,
            batch_first=True,
            norm_first=True,
            activation="gelu",
        )
        self.blocks = nn.TransformerEncoder(layer, num_layers=num_layers)
        self.norm = RMSNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        self.lm_head.weight = self.token_embedding.weight

```

```
    def forward(self, input_ids, labels=None):
        x = self.token_embedding(input_ids)
        L = input_ids.size(1)
        mask = torch.triu(
            torch.ones(L, L, dtype=torch.bool, device=input_ids.device),
            diagonal=1,
        )
        x = self.blocks(x, mask=mask)
        logits = self.lm_head(self.norm(x))

        if labels is None:
            return logits

        shift_logits = logits[:, :-1, :].contiguous()
        shift_labels = labels[:, 1:].contiguous()
        loss = F.cross_entropy(
            shift_logits.view(-1, shift_logits.size(-1)),
            shift_labels.view(-1),
        )
        return logits, loss

def warmup_cosine_lr(step, warmup_steps, total_steps, max_lr, min_lr=0.0):
    if step < warmup_steps:
        return max_lr * (step + 1) / warmup_steps
    progress = (step - warmup_steps) / max(1, total_steps - warmup_steps)
    progress = min(max(progress, 0.0), 1.0)
    return min_lr + 0.5 * (max_lr - min_lr) * (1.0 + math.cos(math.pi * progress))

def train_demo():
    torch.manual_seed(42)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = TinyCausalLM().to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.1)

    total_steps = 8
    warmup_steps = 2
    accumulation_steps = 2
    optimizer.zero_grad(set_to_none=True)

    for step in range(total_steps):
```

```
        input_ids = torch.randint(0, 256, (4, 32), device=device)

        with torch.autocast(
            device_type=device.type,
            dtype=torch.bfloat16,
            enabled=(device.type == "cuda"),
        ):
            _, loss = model(input_ids, labels=input_ids)
            scaled_loss = loss / accumulation_steps

        scaled_loss.backward()

        if (step + 1) % accumulation_steps == 0:
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            lr = warmup_cosine_lr(step, warmup_steps, total_steps, 3e-4, 3e-5)
            for group in optimizer.param_groups:
                group["lr"] = lr
            optimizer.step()
            optimizer.zero_grad(set_to_none=True)

        print(f"step={step:02d} loss={loss.item():.4f}")

if __name__ == "__main__":
    train_demo()
```

大规模训练还会加入 Distributed Data Parallel、Tensor Parallel、Pipeline Parallel、Activation Checkpointing、FlashAttention 与 Sharded Optimizer 等机制；这里的实现仅覆盖单机训练的核心逻辑。

Scaling Law Scaling Law 研究模型性能如何随模型参数量、数据量与计算量变化。一个常见的抽象形式是

$$\mathcal{L}(N, D) \approx \mathcal{L}_\infty + \frac{A}{N^\alpha} + \frac{B}{D^\beta},$$

其中：

- N 表示模型参数规模；
- D 表示训练数据 Token 数；
- $\mathcal{L}_\infty$ 表示不可约损失的近似下界；

- A, B, α, β 为经验拟合参数。

这一关系表达的不是某个永恒不变的精确公式，而是一种经验规律：扩大模型和增加数据通常都能降低 Loss，但二者都存在边际收益递减。

Chinchilla Scaling Chinchilla Scaling 关注固定 Compute Budget 下，模型参数量与训练 Token 数怎样搭配更接近 Compute-optimal。

如果模型过大但数据太少，很多参数没有得到充分训练；如果模型过小但训练数据极多，则容量可能成为瓶颈。因此真正的问题是

$$\text{Given Compute } C, \quad \text{how to choose } N \text{ and } D?$$

Chinchilla 的核心结论是：在其研究设置中，相比早期“快速扩大参数量、训练相对较少 Token”的方式，更均衡地同步扩大模型参数与训练数据通常能够获得更好的 Compute Efficiency。

其结果有时被简化记忆为“约几十个、常用近似为 20 个训练 token 对应一个参数”的经验量级，但这不是适用于所有模型、数据与训练配置的固定常数。真正应掌握的是 Model Size 与 Training Tokens 应共同 Scaling.

参数量、数据量与计算量 一个非常粗略但有用的训练计算量关系是 $C \propto N \times D$, 其中 N 为参数量，D 为训练 Token 数。

对于 Dense Transformer，每个 Token 的前向与反向传播计算量与有效参与计算的参数规模近似成比例。因此在固定计算预算下，提高 N 往往意味着能够处理的 D 会下降，反之亦然。

这形成一个三角关系：

$$\text{Parameters} \quad + \text{Training Tokens} \quad + \text{Compute Budget.}$$

Scaling 的本质就是在这三个量之间做资源分配，而不是单独追求“参数越多越好”。

语言模型训练中的关键指标 训练过程中至少需要同时观察以下几类指标：

- Training Loss 与 Validation Loss；
- Perplexity；
- Learning Rate；
- Gradient Norm；
- Tokens / Second；

- GPU / Accelerator Utilization 或 MFU 一类效率指标。

如果平均 Cross Entropy 为 $\mathcal{L}$，Perplexity 定义为 PPL $= e^{\mathcal{L}}$。因此 PPL 可以直观理解为模型在平均意义下对下一个 token 的“不确定候选规模”。Loss 越低，PPL 通常也越低。

只观察 Training Loss 不够。如果 Training Loss 持续下降而 Validation Loss 不再改善，可能出现过拟合、数据分布差异或评估集问题；如果 Tokens / Second 很低，则可能是数据管线、通信或 Kernel 效率成为瓶颈。

常见误区：Loss、Shifted Labels 与 Mask Cross Entropy 通常已经内部执行稳定的 Log Softmax，因此应输入 Logits，而不是手动 Softmax 后的概率。

对于 next-token prediction，应确保 $\boldsymbol{z}_t \rightarrow x_{t+1}$. 错误地让 $\boldsymbol{z}_t$ 预测当前位置 x_t，会使模型学习复制当前 token，而不是预测下一个 token。

如果训练 Decoder-only CLM 时允许位置读取未来 token，模型会发生 Label Leakage，训练 Loss 可能异常好看，但推理能力会失真。

Padding 位置应通过 ignore index 或 Loss Mask 排除，否则模型会浪费容量学习 Padding Token。

Gradient Accumulation 与数据治理 如果每个 Micro Batch 的 Loss 已经做平均，而又直接累加梯度，梯度尺度会随 Accumulation Steps 增大。常见做法是除以 Accumulation Steps。

预训练数据与评估集发生重复时，Benchmark 分数可能被污染。数据去重与 Evaluation Contamination 检查是训练数据治理的重要组成部分。

从模型结构到预训练的完整数据流 把本章全部内容串起来，可以得到

预训练数据先经过收集、清洗与去重，再进行 Tokenization 和 Sequence Packing，形成训练用的 Token IDs。

这条路线把“模型结构”和“模型训练”连接起来：Transformer 决定如何从上下文计算隐藏状态，LM Head 决定如何把隐藏状态映射到 Vocabulary，而 Cross Entropy 与 Optimizer 决定这些参数如何被数据更新。

8.3 训练流程与规模规律的统一理解

可以把整个知识体系分成五层。

第一层是建模目标：

$$\text{CLM} \quad / \quad \text{MLM} \quad / \quad \text{Prefix LM}.$$

第二层是输出概率：

LM Head 将 Hidden State 投影为 Logits；Softmax 再将 Logits 转为词表概率分布。

第三层是损失：

$$\text{LogSoftmax} + \text{NLL} = \text{CrossEntropy}\,.$$

第四层是优化：

$$\text{AdamW} + \text{Warmup} + \text{Cosine Decay} + \text{Gradient Clipping} + \text{Mixed Precision}.$$

第五层是 Scaling：

$$\text{Parameters} + \text{Training Tokens} + \text{Compute}.$$

真正理解预训练，需要同时掌握“预测什么、如何计算 Loss、数据从哪里来、参数怎样更新、规模怎样扩大”五个层次。

预训练模型与后训练的关系 预训练得到的是 Base / Pretrained Model。它已经学习了语言模式、知识与一定的推理能力，但还没有必然学会按照 System / User / Assistant 的对话格式稳定响应。后续 SFT、LoRA、DPO、RLHF、GRPO 等 Post-Training 方法正是在这一预训练模型之上继续进行能力塑形与偏好对齐。

第 9 章

LLM 微调与对齐：SFT、PEFT、RLHF 与偏好优化

预训练让大语言模型从海量文本中学习语言规律、知识与一定的推理能力，但 Base Model 不一定能稳定遵循指令。Post-Training 通过指令监督与偏好优化，使模型更适合对话和任务执行。

SFT（Supervised Fine-Tuning）主要解决“给定指令后应该怎样回答”；Preference Alignment 则进一步解决“多个都能回答问题的候选结果中，哪一种回答更符合目标偏好”。在工程上，参数又可以通过 Full Fine-Tuning、LoRA、QLoRA 等不同方式更新。Post-Training 有两个彼此独立的维度：

$$\begin{aligned} \text{训练目标：} &\quad \text{SFT / DPO / PPO / GRPO / …,} \\ \text{参数更新方式：} &\quad \text{Full FT / LoRA / QLoRA / ….} \end{aligned}$$

也就是说，LoRA 并不是一种与 SFT 平行的训练目标。实际训练中完全可以出现

$$\text{LoRA + SFT,} \qquad \text{LoRA + DPO,} \qquad \text{QLoRA + SFT.}$$

9.1 从预训练到 Post-Training

Base Model、Chat Model 与 Post-Training 对于 Decoder-only 语言模型，预训练的核心目标通常是 Next Token Prediction。给定 token 序列 $x_1, x_2, \ldots, x_T$，模型学习概率分解

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t}).$$

对应的负对数似然损失可以写成

$$\mathcal{L}_{\text{pretrain}} = -\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}).$$

这一目标能够让模型学习文本的局部与长距离依赖关系，也能够通过大规模数据获得知识与模式。但是预训练数据本身通常只是自然文本，并不一定显式告诉模型“用户提出问题后，应以助手身份按照某种格式回答”。

预训练 Base Model 的直接行为更接近

而 Chat Model 需要学会

二者底层仍然可以使用相同的 Causal Language Modeling，但训练数据的结构发生了变化。Chat Model 不仅需要预测“下一 token”，还需要在特定对话角色和指令上下文下生成合适回答。

预训练主要学习语言分布；SFT 进一步把这种语言建模能力约束到“按照指令完成任务”的数据分布上。

Post-Training 通常包含两个层次。

第一层是行为模仿。模型观察人工编写、模型生成后筛选或其他方式得到的高质量指令回答数据，并通过监督学习模仿这些回答，这就是 SFT。

第二层是偏好优化。即使两个回答都语法正确并且能够解决问题，人类仍可能更喜欢其中一个。Preference Alignment 利用这种偏好信息进一步调整模型。

因此可以将 Post-Training 理解为

$$\text{Capability Shaping} + \text{Behavior Shaping}.$$

前者使能力更加适合目标任务，后者使输出风格与行为更加符合目标偏好。

“微调”描述的是在预训练参数基础上继续训练模型这一大类过程；“对齐”则更强调模型输出与目标偏好、规则或奖励信号之间的一致性。

SFT 本身也会改变模型行为，因此广义上属于对齐流程的一部分，但在技术讨论中通常将 SFT 与 Preference / RL Alignment 分开讨论，以便区分监督目标和偏好目标。

Post-Training 的两个正交维度 Post-Training 配置由两个彼此独立的维度组成：训练目标包括 SFT、Preference Optimization 与 RL-based Optimization；参数更新方式包括 Full Fine-Tuning、LoRA 与 QLoRA。前者描述优化目标，后者描述哪些参数参与更新，两者可以组合选择，并非同一条互斥路线。

9.2 SFT：从 Instruction Data 到训练 Loss

SFT、Instruction Tuning 与 Chat Data 设一条训练数据由输入指令 x 和目标回答 y 组成：(x, y). 其中回答进一步写成 $y = (y_1, y_2, \ldots, y_T)$.

SFT 的目标是最大化目标回答在给定输入条件下的概率：

$$P_\theta(y \mid x) = \prod_{t=1}^{T} P_\theta(y_t \mid x, y_{<t}).$$

因此监督损失为

$$\mathcal{L}_{\text{SFT}} = -\sum_{t=1}^{T} \log P_\theta(y_t \mid x, y_{<t}).$$

从模型本身看，这仍然是 Causal LM；改变的是训练序列来自结构化的 Instruction / Conversation Dataset。

Instruction Tuning 使用大量“指令–回答”样本训练模型，使模型能够从任务描述中理解需要执行的操作。例如：

字段	示例含义
Instruction	“把下面的英文翻译成中文”
Input	待翻译文本
Response	目标中文翻译

如果只包含一个用户请求和一个助手回答，可以看作单轮 SFT；如果包含多次 User / Assistant 交替，则构成多轮对话 SFT。

现代 Chat Model 常把对话划分为不同角色：

$$\text{System}, \qquad \text{User}, \qquad \text{Assistant}.$$

System Message 用于定义模型整体行为或环境规则；User Message 表示当前用户输入；Assistant Message 是训练阶段需要模型学习生成的目标内容。

一条多轮数据可以抽象为 $(s, u_1, a_1, u_2, a_2, \ldots, u_n, a_n)$. 模型最终看到的并不是 Python 字典或 JSON，而是 Chat Template 将这些结构化消息转成的一串 token。

SFT 使用指令与对话样本监督模型生成符合任务要求的回答。

需要注意，SFT 并不会重新发明语言模型的基础结构。Attention、RoPE、RMSNorm、SwiGLU 等组件仍然保持不变；真正变化的是训练数据、标签选择以及参数更新过程。

Chat Template 与训练样本构造 对话数据在数据集里可能表示为

```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What is LoRA?"},
    {"role": "assistant", "content": "LoRA is a parameter-efficient fine-tuning
    ↪ method."},
]
```

但 Transformer 最终只能处理 token id。因此需要把角色和文本线性化成类似

序列通常包含 BOS、System、User、Assistant 与 EOS 等结构标记；具体标记取决于模型使用的 Chat Template。

的序列。

不同模型使用不同的特殊 token 和格式，这种转换规则就是 Chat Template。

Chat Template 常使用特殊标记区分角色和消息边界。例如可以抽象表示为

$$\langle \text{system} \rangle s \langle \text{user} \rangle u \langle \text{assistant} \rangle a.$$

BOS（Beginning of Sequence）表示序列起点，EOS（End of Sequence）表示回答或整个序列结束。实际模型使用的 token 名称、消息结束标记和换行规则可能不同，但核心目的都是让模型能够识别结构边界。

多轮对话 (s, u_1, a_1, u_2, a_2) 经 Template 转换后，会形成连续的 token 序列 $(z_1, \ldots, z_N)$。

模型仍然对每个位置预测下一 token，只是 Loss Mask 可以决定哪些 token 真正参与 SFT 损失。

如果训练时模型学习的结构为

$$\text{Role Token} + \text{Content} + \text{Message End},$$

而推理时使用完全不同的标记方式，那么模型会遇到与 SFT 数据分布不一致的输入格式。

Chat Template 不只是界面格式，而是模型训练分布的一部分。

因此，使用开源 Chat Model 时应优先使用其对应 tokenizer 提供的官方 Chat Template，而不是自行猜测角色 token。

Shifted Labels、Loss Mask 与 Python 实现 假设完整 token 序列为 $z_1, z_2, \ldots, z_N$. 模型在位置 t 输出 logits 来预测 z_{t+1}。因此训练时本质上执行

$$\text{Input} = z_{1:N-1}, \qquad \text{Label} = z_{2:N}.$$

这就是常说的 shifted labels。

考虑训练序列

$$\underbrace{\text{System + User}}_{\text{Prompt}} \quad \underbrace{\text{Assistant Response}}_{\text{Target}}.$$

一种做法是所有 token 都计算 Causal LM Loss；另一种常见做法是只让 Assistant Response 的 token 参与损失。

定义 mask $m_t \in \{0, 1\}$, 则可以写成

$$\mathcal{L} = -\frac{1}{\sum_t m_t} \sum_t m_t \log P_\theta(z_t \mid z_{<t}).$$

当 Prompt token 的 $m_t = 0$ 时，这些位置仍然参与前向上下文计算，但不会直接贡献监督损失。

对于单轮样本，可以将 mask 表示为

Token	S_1	S_2	U_1	U_2	A_1	A_2	A_3	A_4
m_t	0	0	0	0	1	1	1	1

多轮对话中，也可以只让每一轮 Assistant 区域参与 Loss。

这两个 Mask 容易混淆。

Mask	作用位置	主要目的
Attention / Padding Mask	Attention Score	阻止模型读取无效 Padding 或未来 token
Loss Mask	Cross Entropy Loss	决定哪些标签位置参与训练目标

Attention Mask 控制“模型能看什么”；Loss Mask 控制“哪些位置产生监督梯度”。

```python
import torch
import torch.nn.functional as F

def masked_causal_lm_loss(logits, labels, loss_mask):
    shift_logits = logits[:, :-1].contiguous()
    shift_labels = labels[:, 1:].contiguous()
    shift_mask = loss_mask[:, 1:].contiguous().float()

    token_loss = F.cross_entropy(
        shift_logits.view(-1, shift_logits.size(-1)),
        shift_labels.view(-1),
        reduction="none",
    ).view_as(shift_labels)

    return (token_loss * shift_mask).sum() / shift_mask.sum().clamp_min(1.0)
```

9.3 Full Fine-Tuning 与 PEFT

Full Fine-Tuning 与训练显存 Full Fine-Tuning 表示模型中原有参数全部保持可训练：

$$\theta \longrightarrow \theta'.$$

优化器直接根据下游训练目标更新整个模型。

如果模型包含 Attention、FFN、Embedding、Normalization 等参数，那么这些参数都可以参与反向传播。

全参数训练的显存不只有模型权重。主要来源通常包括：

- Model Parameters；
- Gradients；
- Optimizer States；
- Activations；
- 临时计算 Buffer。

对于 AdamW，每个参数通常还需要维护一阶与二阶矩估计。因此，大模型 Full Fine-Tuning 的训练显存会显著高于只存储模型权重所需要的显存。

Full Fine-Tuning 的优势是参数自由度最大，所有层都能针对目标数据分布进行调整；缺点是训练和存储成本高，并且每个任务通常需要保存一整份新的模型权重。

它更适合：

- 有足够训练资源；
- 数据规模足够大；
- 需要模型整体发生较大分布迁移；
- 希望获得最高参数自由度的场景。

当显存或模型复制成本成为主要限制时，就需要 Parameter-Efficient Fine-Tuning。

Parameter-Efficient Fine-Tuning PEFT（Parameter-Efficient Fine-Tuning）是一组只训练少量额外参数、或只修改模型很小一部分参数的技术集合。

其核心目标可以表示为

$$|\theta_{\text{trainable}}| \ll |\theta_{\text{model}}|.$$

LoRA 是目前最常见的 PEFT 方法之一，但 PEFT 并不只等于 LoRA。

Adapter 通常在 Transformer Block 中插入一个小型瓶颈网络：

$$\boldsymbol{h} \longrightarrow \boldsymbol{W}_{\text{down}} \longrightarrow \sigma \longrightarrow \boldsymbol{W}_{\text{up}} \longrightarrow \boldsymbol{h} + \Delta\boldsymbol{h}.$$

其中瓶颈维度远小于隐藏维度，因此只需要训练较少参数。

Prompt Tuning 不修改 Transformer 内部权重，而是在输入 embedding 前增加可学习的 Soft Prompt：

$$\boldsymbol{P}_{\text{soft}} \in \mathbb{R}^{m \times d_{\text{model}}}.$$

模型输入变为 $[\boldsymbol{P}_{\text{soft}}; \boldsymbol{X}]$.

训练时主要更新这些 Soft Prompt 参数。

Prefix Tuning 将可学习前缀引入 Transformer 多层的 Attention 表示中。可以抽象理解为给 $\boldsymbol{K}/\boldsymbol{V}$ 增加一组可学习前缀：

$$\boldsymbol{K}' = [\boldsymbol{K}_{\text{prefix}}; \boldsymbol{K}], \qquad \boldsymbol{V}' = [\boldsymbol{V}_{\text{prefix}}; \boldsymbol{V}].$$

它与 Prompt Tuning 都属于“冻结大部分模型，通过少量额外参数调整行为”的思路，但作用位置不同。

方法	主要作用位置	核心思想
LoRA	Linear Weight	用低秩矩阵学习权重增量
Adapter	Transformer Block 内部	插入小型瓶颈网络
Prompt Tuning	Input Embedding	学习一组 Soft Prompt Token
Prefix Tuning	Attention Prefix	学习多层可见的前缀表示

LoRA：从全参数更新到低秩更新 对一个线性层，原始权重为 $\boldsymbol{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$.

Full Fine-Tuning 直接学习完整更新量 $\Delta \boldsymbol{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, 并得到 $\boldsymbol{W}' = \boldsymbol{W} + \Delta \boldsymbol{W}$.

LoRA 假设下游微调所需要的权重更新可以用较低秩的矩阵近似：$\Delta \boldsymbol{W} = \boldsymbol{B}\boldsymbol{A}$, 其中

$$\boldsymbol{A} \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad \boldsymbol{B} \in \mathbb{R}^{d_{\text{out}} \times r},$$

并满足 $r \ll \min(d_{\text{in}}, d_{\text{out}})$.

因此前向传播变为

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + s\boldsymbol{B}\boldsymbol{A}\boldsymbol{x},$$

其中 s 是 LoRA scaling。

完整更新矩阵需要训练 $d_{\text{out}} d_{\text{in}}$ 个参数。

LoRA 只需要训练

$$r d_{\text{in}} + d_{\text{out}} r = r(d_{\text{in}} + d_{\text{out}})$$

个参数。

如果

$$d_{\text{in}} = d_{\text{out}} = 4096, \qquad r = 16,$$

完整矩阵包含 $4096^2 = 16{,}777{,}216$ 个参数，而 LoRA 只需要 $16(4096 + 4096) = 131{,}072$.

因此 LoRA 可以在冻结 Base Model 的情况下，只训练很小的一组增量参数。

Rank、Alpha、Dropout 与参数量 Rank r 控制低秩更新空间的容量。

较小的 r：参数更少，容量更低. 较大的 r：参数更多，更新空间更丰富.

需要注意，r 并不是越大越好。实际最优值取决于任务复杂度、数据规模以及 Target Module。

常见实现使用 $s = \frac{\alpha}{r}$. 因此

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A}\boldsymbol{x}.$$

α 用于控制 LoRA 分支相对于冻结主权重的整体幅度。这样在改变 r 时，可以通过 scaling 对更新尺度进行更稳定的控制。

LoRA Dropout 通常作用于进入低秩分支的输入：

$$\Delta\boldsymbol{y} = \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A}\,\mathrm{Dropout}(\boldsymbol{x}).$$

其目的与普通 Dropout 类似，用于提供一定正则化。

LoRA 的核心工程优势来自

$$\boldsymbol{W} \quad \text{frozen}, \qquad \boldsymbol{A}, \boldsymbol{B} \quad \text{trainable}.$$

因此优化器只需要维护 $\boldsymbol{A}$、$\boldsymbol{B}$ 的梯度与状态，大幅减少训练参数相关的显存开销。

Target Modules 与 LoRA Merge 一个 Attention Block 中通常包含

$$\boldsymbol{W}^{\mathrm{Q}}, \quad \boldsymbol{W}^{\mathrm{K}}, \quad \boldsymbol{W}^{\mathrm{V}}, \quad \boldsymbol{W}^{\mathrm{O}}.$$

LoRA 可以应用于这些线性投影。不同实现可能只训练 Query/Value 投影，也可能覆盖 Query/Key/Value/Output 投影。

从结构上看，这相当于

$$\boldsymbol{Q} = \boldsymbol{X}(\boldsymbol{W}^{\mathrm{Q}} + \Delta\boldsymbol{W}^{\mathrm{Q}}),$$

以及类似的 Key、Value、Output 投影。

对于 SwiGLU FFN，常见三组投影为

$$\boldsymbol{W}_{\text{gate}}, \qquad \boldsymbol{W}_{\text{up}}, \qquad \boldsymbol{W}_{\text{down}}.$$

LoRA 同样可以作用于这些线性层。

因此 Target Modules 决定了 LoRA 更新能够进入 Transformer 的哪些路径。

在 PEFT 中，Target Modules 通常通过模块名称指定。例如现代 Decoder-only 模型中可能包含：

```
target_modules = [
    "q_proj", "k_proj", "v_proj", "o_proj",
    "gate_proj", "up_proj", "down_proj",
]
```

实际模块名称必须与目标模型代码中的 Linear 层名称一致。

训练结束后有 $\boldsymbol{W}' = \boldsymbol{W} + \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A}$.

因为 $\boldsymbol{B}\boldsymbol{A}$ 与 $\boldsymbol{W}$ Shape 完全相同，所以可以预先执行

$$\boldsymbol{W} \leftarrow \boldsymbol{W} + \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A},$$

得到合并后的普通 Linear 权重。

此后推理不再需要单独计算 LoRA 分支。

LoRA 的低秩结构主要用于训练阶段的参数高效更新；Merge 后可以重新得到一个普通的稠密权重矩阵。

Python：LoRA Linear 与 Merge 验证

```python
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, rank=8, alpha=16, dropout=0.0, bias=False):
        super().__init__()
        self.rank = rank
        self.scaling = alpha / rank
        self.weight = nn.Parameter(torch.empty(out_features, in_features), requires_grad=False)
        self.bias = nn.Parameter(torch.zeros(out_features), requires_grad=False) if bias else None
        self.lora_a = nn.Parameter(torch.empty(rank, in_features))
        self.lora_b = nn.Parameter(torch.zeros(out_features, rank))
        self.dropout = nn.Dropout(dropout)
```

```
        nn.init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        nn.init.kaiming_uniform_(self.lora_a, a=math.sqrt(5))

    def forward(self, x):
        base = F.linear(x, self.weight, self.bias)
        delta = F.linear(F.linear(self.dropout(x), self.lora_a), self.lora_b)
        return base + self.scaling * delta

    @torch.no_grad()
    def merged_weight(self):
        return self.weight + self.scaling * (self.lora_b @ self.lora_a)
```

这里初始化 $\boldsymbol{B} = 0$，使训练刚开始时

$$\Delta \boldsymbol{W} = \boldsymbol{B}\boldsymbol{A} = 0,$$

因此 LoRA 模型一开始与原始 Base Linear 的输出一致。

```
def count_parameters(model):
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    return total, trainable

layer = LoRALinear(4096, 4096, rank=16, alpha=32)
total, trainable = count_parameters(layer)
print(total, trainable, trainable / total)
```

```
x = torch.randn(2, 8, 4096)
y_lora = layer(x)
y_merged = F.linear(x, layer.merged_weight(), layer.bias)
print(torch.allclose(y_lora, y_merged, atol=1e-5))
```

当 Dropout 关闭并处于确定性前向时，Merge 前后的结果应保持一致。

QLoRA 与低显存 PEFT 工程：从 LoRA 到 QLoRA LoRA 已经显著减少了可训练参数，但 Base Model 的冻结权重仍然需要存储在显存中。如果 Base Model 本身非常大，即使完全不训练这些参数，也可能难以放入单张 GPU。

QLoRA 的核心做法是：

$$\text{Frozen Quantized Base Model} + \text{Trainable LoRA Adapters}.$$

即把 Base Model 以低比特形式存储，同时在其上训练 LoRA 参数。

设原始权重为 $\boldsymbol{W}$。QLoRA 将其量化表示为 $\boldsymbol{W}_q = Q(\boldsymbol{W})$, 训练时 Base Weight 保持冻结，而 LoRA 分支仍然更新：

$$\boldsymbol{y} = \boldsymbol{W}_q\boldsymbol{x} + \frac{\alpha}{r}\boldsymbol{B}\boldsymbol{A}\boldsymbol{x}.$$

实际矩阵乘法通常会在计算时将量化权重按实现要求反量化到适合计算的精度，而不是直接对整数位模式执行普通高精度 Linear。

比较项	LoRA	QLoRA
Base Model	通常以 FP16 / BF16 等精度加载	低比特量化加载
Base Weight	Frozen	Frozen
训练参数	LoRA $\boldsymbol{A}/\boldsymbol{B}$	LoRA $\boldsymbol{A}/\boldsymbol{B}$
主要目标	减少可训练参数	同时减少 Base Model 存储显存

NF4、Double Quantization 与 Compute Dtype NF4（NormalFloat 4-bit）是一种面向近似正态分布权重设计的 4-bit 数据表示。其目标是让有限的 16 个离散值更有效地覆盖神经网络权重常见的分布区域。

在学习时应重点理解：

NF4 是 Base Model 权重的低比特存储表示，不是 LoRA 参数本身必须采用的训练精度。

量化通常需要保存 Scale 等量化元数据。Double Quantization 进一步对这些量化常数本身进行量化，从而继续减少内存开销。

可以抽象为

$$\boldsymbol{W} \xrightarrow{Q_1} (\boldsymbol{W}_q, s) \xrightarrow{Q_2} (\boldsymbol{W}_q, Q(s)).$$

QLoRA 中需要区分：

- Base Weight 的存储精度；
- 计算时的 compute dtype；
- LoRA 参数的训练精度。

因此“4-bit QLoRA”并不等价于“整个反向传播都在纯 INT4 中完成”。

PEFT 工程：Config、Adapter、Merge 与模型加载 使用 PEFT 时，可以用配置对象描述 LoRA Rank、Alpha、Dropout 和 Target Modules。

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    bias="none",
    task_type="CAUSAL_LM",
)

model = get_peft_model(model, config)
model.print_trainable_parameters()
```

LoRA 训练结束后通常只保存 Adapter：

```
model.save_pretrained("lora_adapter")
```

因为 Adapter 参数量远小于 Base Model，所以同一个 Base Model 可以对应多个任务 Adapter。

推理部署时可以将 LoRA 增量合并到 Base Weight：

```
merged_model = model.merge_and_unload()
```

合并后不再需要运行独立 LoRA 分支。

QLoRA 的 4-bit 加载配置例如：

```
import torch
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
```

```
    bnb_4bit_use_double_quant=True,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=quant_config,
)
```

具体 API 会随库版本和模型实现变化，因此工程实践时应结合对应版本文档检查参数。

Full FT、LoRA 和 QLoRA 都属于“参数如何更新与存储”的问题，而不是“使用什么训练目标”。

因此同一个 SFT Dataset 可以选择：

$$\begin{cases} \text{Full FT + SFT}, \\ \text{LoRA + SFT}, \\ \text{QLoRA + SFT}. \end{cases}$$

DPO 同样可以与 LoRA 组合。

比较项	Full FT	LoRA	QLoRA
Base Weight	可训练	冻结	低比特冻结
额外 Adapter	无	有	有
可训练参数	几乎全部	少量 $\boldsymbol{A}/\boldsymbol{B}$	少量 $\boldsymbol{A}/\boldsymbol{B}$
Base 显存	高	仍需加载 Base	进一步降低
任务权重保存	完整模型	Adapter	Adapter
部署	直接模型	Adapter 或 Merge	Adapter / Merge 后部署

9.4 从 SFT 到 Preference Alignment

SFT 的能力边界与 Preference Data SFT 数据提供一个或若干目标回答，模型学习模仿这些样本。但很多任务并不存在唯一正确回答。例如同一个问题可能有两个都正确的答案：

$$y_1, \qquad y_2.$$

人类可能因为简洁度、事实性、风格或安全性更偏好 y_1。

SFT 只能告诉模型“这个回答值得学习”，而 Preference Data 还可以告诉模型“在两个回答中更喜欢哪一个”.

典型偏好样本写成 (x, y_w, y_l)，其中 y_w = preferred / chosen response，y_l = rejected response.

这种数据可以来自人工比较、规则评分、模型评审或其他奖励信号。

SFT 更接近 Imitation Learning, Preference Alignment 则加入 $y_w \succ y_l$ 这种相对排序信息。

这为 Reward Model、RLHF 与 DPO 等方法提供了基础。

Reward Model 与 Pairwise Preference Learning Reward Model 接收 Prompt 与 Response，并输出一个标量：$r_\phi(x, y) \in \mathbb{R}$.

这个标量代表模型对回答质量或偏好的估计。

对于 chosen / rejected pair，希望满足

$$r_\phi(x, y_w) > r_\phi(x, y_l).$$

可以把 chosen 优于 rejected 的概率建模为

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right),$$

其中 $\sigma(z) = \frac{1}{1+e^{-z}}$.

Reward Model Loss 为

$$\mathcal{L}_{\text{RM}} = -\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right).$$

如果 chosen reward 已经远高于 rejected reward，则 Loss 较小；如果排序错误，则产生更大梯度。

```
def reward_model_loss(chosen_reward, rejected_reward):
    return -F.logsigmoid(chosen_reward - rejected_reward).mean()
```

Reward Model 只是人类偏好的近似。如果 Policy Model 找到 Reward Model 的漏洞，就可能产生“奖励很高但实际质量很差”的输出，这类现象通常称为 Reward Hacking。

因此 Reward Model 的数据覆盖、泛化能力以及后续 KL 约束都非常重要。

RLHF 与 PPO：经典 RLHF Pipeline SFT Model 通常作为强化学习阶段的初始化 Policy，同时保留一份冻结 Reference Model 用于约束模型不要偏离原始分布过远。

Policy、Reference、Reward、KL 与 Rollout RLHF 中常见三个重要模型：

- Policy Model π_θ：正在更新的语言模型；
- Reference Model π_{ref}：冻结的参考策略；
- Reward Model r_ϕ：给完整回答打分。

Policy 对 Prompt 采样回答 $y \sim \pi_\theta(\cdot \mid x)$, Reward Model 给出 $r_\phi(x, y)$. 然后强化学习算法根据 Reward 更新 Policy。

如果只最大化 Reward Model 分数，Policy 可能过度偏离 SFT 模型。常见目标因此加入 KL 约束：

$$R(x, y) = r_\phi(x, y) - \beta D_{\text{KL}}\left(\pi_\theta \| \pi_{\text{ref}}\right).$$

其中 β 控制偏离 Reference Policy 的代价。

RLHF 的目标不是无限制地追逐 Reward，而是在提高偏好 Reward 的同时限制 Policy 偏离参考分布。

RL 阶段的数据通常需要当前 Policy 实际生成：

$$x \xrightarrow{\pi_\theta} y.$$

这种由当前策略在线产生训练样本的过程称为 Rollout。它也是 RLHF 工程成本较高的重要原因之一。

PPO：Policy Ratio、Advantage 与 Clipping 设旧策略为 π_{old}，更新后的策略为 π_θ。

对于动作 token a_t，定义概率比

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

当 ρ_t 偏离 1 很多时，说明一次更新让策略发生了较大变化。

PPO 使用截断目标限制单次策略更新幅度：

$$L_t^{\text{clip}} = \min\left(\rho_t A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)A_t\right).$$

其中 A_t 是 Advantage，用于衡量当前动作相对于基准的好坏。

PPO 通常需要 Value Function 估计状态价值：$V(s_t)$. 然后利用 Reward / Return 与 Value 构造 Advantage。

这意味着 RLHF-PPO 不仅需要 Policy、Reference、Reward Model，还经常需要 Value Head / Critic，因此工程系统复杂度较高。

完整 PPO 训练需要处理：

- 在线 Rollout；
- Reward Model 推理；
- Policy / Old Policy 概率；
- Reference KL；
- Value Estimation；
- Advantage；
- 多轮 PPO Update。

因此后续直接偏好优化方法尝试绕开显式 Reward Model + PPO 这条复杂链路。

DPO：直接偏好优化目标 DPO 直接使用偏好对 (x, y_w, y_l) 训练 Policy，而不需要先单独拟合一个显式 Reward Model，再运行 PPO。

其目标是让 Policy 相对于 Reference Model 更偏向 chosen response，而不是 rejected response。

定义 Policy 对完整回答的 log probability：

$$\log \pi_\theta(y \mid x) = \sum_t \log \pi_\theta(y_t \mid x, y_{<t}).$$

Reference Model 类似：$\log \pi_{\text{ref}}(y \mid x)$.

DPO 关注 chosen 和 rejected 两条回答相对于 Reference 的 log-ratio 差异。

定义

$$\Delta_\theta = \log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x),$$

以及

$$\Delta_{\text{ref}} = \log \pi_{\text{ref}}(y_w \mid x) - \log \pi_{\text{ref}}(y_l \mid x).$$

DPO Loss 可以写成

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta(\Delta_\theta - \Delta_{\text{ref}})\right).$$

展开后为

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right]\right).$$

因此训练会推动 Policy 相对提高 chosen 的概率，并相对降低 rejected 的概率。

β 控制 Preference Optimization 与 Reference 约束之间的尺度关系。它不是简单意义上的 Learning Rate，而是 DPO 目标本身的温度 / 正则化尺度参数。

标准 DPO 训练直接使用静态偏好数据：(x, y_w, y_l). 它不需要：

- 额外训练一个显式 Reward Model；
- 在线 PPO Rollout；
- Value Model；
- PPO Clipped Update。

因此工程实现通常比经典 RLHF-PPO 更简单。

```
def dpo_loss(policy_chosen, policy_rejected, ref_chosen, ref_rejected, beta=0.1):
    policy_logratio = policy_chosen - policy_rejected
    ref_logratio = ref_chosen - ref_rejected
    logits = beta * (policy_logratio - ref_logratio)
    return -F.logsigmoid(logits).mean()
```

这里四个输入都表示整条回答的 log probability，而不是单个 token 的概率。

DPO 与 RLHF 的关系 经典 RLHF 通常先用偏好数据训练 Reward Model，再通过 Online Rollout 与 PPO 优化 Policy；DPO 则直接利用偏好数据优化 Policy，不需要单独训练 Reward Model。

二者使用的偏好数据来源可以类似，但优化路径不同。

比较项	RLHF + PPO	DPO
偏好数据	Pairwise / Reward Data	Chosen / Rejected Pair
显式 Reward Model	通常需要	不需要
在线 Rollout	需要	标准离线 DPO 不需要
Value / Critic	PPO 通常需要	不需要
Reference Model	常用于 KL 约束	直接进入 DPO Loss
工程复杂度	较高	相对较低

IPO、ORPO、KTO 与 SimPO IPO（Identity Preference Optimization）同样属于直接偏好优化路线。它重新设计 Preference Objective，希望减少某些极端偏好概率下 DPO 目标可能出现的过度优化倾向。

学习 IPO 时不必先记住复杂推导，更重要的是理解：

$$\text{Preference Pair} + \text{Reference Policy} + \text{Alternative Direct Objective}.$$

ORPO 将监督学习与偏好排序结合在同一个目标中。它不再把“SFT 阶段”和“偏好阶段”完全拆开，而是在提高 chosen response likelihood 的同时加入 chosen / rejected 的相对偏好约束。

因此可以把其思想抽象为

$$\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \lambda \mathcal{L}_{\text{preference}}.$$

KTO 的一个重要特点是可以利用更弱形式的反馈，例如单条回答被标记为 desirable 或 undesirable，而不一定要求每个样本都严格组成 chosen / rejected pair。

因此它拓展了偏好数据的组织形式。

SimPO 属于 reference-free preference optimization 路线之一，直接从 Policy 的回答概率构造偏好目标，并通过长度归一化等设计减少长度因素的影响。

在不运行完整 PPO 的前提下，重新设计“chosen 应该优于 rejected”的可微目标。

GRPO 与强化学习式 Post-Training：Group Sampling 与 Relative Advantage DPO 通常直接消费离线偏好对：(x, y_w, y_l).

GRPO（Group Relative Policy Optimization）则更接近 PPO 所属的在线强化学习路线。它通常对同一个 Prompt 采样多个回答，根据这些回答的 Reward 构造组内相对 Advantage，再更新 Policy。

因此二者的核心数据流不同。

给定 Prompt x，从当前 Policy 采样 G 个回答：

$$y_1, y_2, \ldots, y_G \sim \pi_{\text{old}}(\cdot \mid x).$$

每个回答获得 Reward：$r_1, r_2, \ldots, r_G$.

计算组内均值

$$\bar{r} = \frac{1}{G}\sum_{i=1}^{G} r_i,$$

以及标准差

$$s_r = \sqrt{\frac{1}{G}\sum_{i}(r_i - \bar{r})^2}.$$

然后构造相对 Advantage：

$$\hat{A}_i = \frac{r_i - \bar{r}}{s_r + \varepsilon}.$$

这意味着 Reward 高于组平均的回答获得正 Advantage，低于组平均的回答获得负 Advantage。

GRPO 的关键思想是：

不依赖单独的 Value / Critic 网络，而是利用同一 Prompt 下多个回答的组内相对 Reward 构造 Advantage。

这可以显著简化 PPO 风格训练中 Critic / Value Model 的部分开销。

GRPO Policy Update 与 Python 实现 定义 token-level policy ratio

$$\rho_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\text{old}}(y_{i,t} \mid x, y_{i,<t})}.$$

GRPO 的简化目标为

$$J_{\text{GRPO}} = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t}\min\left(\rho_{i,t}\hat{A}_i, \text{clip}(\rho_{i,t}, 1-\epsilon, 1+\epsilon)\hat{A}_i\right) - \beta D_{\text{KL}}.$$

实际工程实现还会涉及 Reward 设计、Mask、KL 估计方式以及多次更新等细节。这里重点是理解其“Group Relative Advantage + PPO-style Policy Update”的主线。

```python
def group_relative_advantage(rewards, eps=1e-6):
    mean = rewards.mean(dim=-1, keepdim=True)
    std = rewards.std(dim=-1, keepdim=True, unbiased=False)
    return (rewards - mean) / (std + eps)
```

例如输入 Shape $[B, G]$ 表示 batch 中每个 Prompt 都有 G 个回答 Reward，输出 Shape 保持不变。

9.5 Post-Training 方法的比较

Training Objective：SFT、DPO、PPO 与 GRPO SFT 数据通常是 (x, y). 模型直接模仿目标回答。

DPO 数据为 (x, y_w, y_l). 模型学习 chosen 相对 rejected 的偏好关系。

GRPO 更接近

$$x \longrightarrow \{y_1, \ldots, y_G\} \longrightarrow \{r_1, \ldots, r_G\} \longrightarrow \{\hat{A}_1, \ldots, \hat{A}_G\} \longrightarrow \text{Policy Update.}$$

这些回答通常由当前或旧 Policy 在线采样，而不是预先固定为 chosen / rejected pair。

方法	数据形式	主要训练信号
SFT	Prompt + Target Response	Target token Cross Entropy
DPO	Prompt + Chosen + Rejected	Preference log-ratio
GRPO	Prompt + Sampled Responses + Rewards	Group Relative Advantage + Policy Ratio

方法	输入数据 / 信号	核心目标	所属维度
SFT	Prompt + Target	模仿目标回答	Training Objective
LoRA	任意微调数据	低秩更新 Linear Weight	Parameter Update
QLoRA	任意微调数据	Quantized Base + LoRA	Parameter / Memory
RLHF + PPO	Reward + Online Rollout	最大化奖励并限制策略变化	RL Alignment
DPO	Chosen / Rejected	直接优化偏好 log-ratio	Preference Objective
GRPO	Group Samples + Reward	组内相对 Advantage 的策略优化	RL-based Post-Training

Training Objective 与 Parameter Update 的正交关系 Post-Training 配置包含两个正交维度：训练目标决定优化目标，参数更新方式决定哪些参数参与优化。

$$\begin{aligned}\text{Training Objective:}&\quad \text{SFT / DPO / PPO / GRPO / ...,}\\ \text{Parameter Update:}&\quad \text{Full FT / LoRA / QLoRA / ....}\end{aligned}$$

因此，同一个训练目标可以与不同参数更新方式组合，而同一种 LoRA / QLoRA 也可以承载不同的 Post-Training Objective。

9.6 LLM Post-Training 的完整工程流程

从 Base Model 到 Evaluation 与部署 Post-Training 工程流程通常包括 Base Model 选择、Instruction Dataset 构建、Chat Template / Tokenization、Loss Mask 配置、目标函数与参数更新方式选择、训练、评估和部署。

实际项目不一定包含所有阶段。例如很多领域模型只做 SFT；有些模型会在 SFT 后做 DPO；有些推理模型会采用强化学习式 Post-Training。

需要考虑：

- 参数规模；

- Context Length；
- Tokenizer；
- License；
- Base / Instruct 版本；
- 训练与部署硬件。

如果目标是训练一个新的 Chat Model，通常需要明确选择 Base Model 还是已经完成指令对齐的 Instruct Model。

后训练过程中不能只观察 Training Loss。还需要使用与目标任务对应的 Evaluation，包括：

- Task Accuracy；
- Generation Quality；
- Preference Win Rate；
- Safety / Robustness；
- Domain-specific Metrics；
- 基础能力是否退化。

这也是防止过拟合、灾难性遗忘与 Alignment Tax 的重要环节。

常见实现误区与风险 二者可以共享 Causal LM Loss，但训练数据分布与目标不同：预训练学习大规模自然文本分布，SFT 学习结构化指令响应分布。

LoRA 是低秩参数更新方法；Quantization 是降低权重表示精度的方法。QLoRA 才把二者组合到一起。

QLoRA 中的低比特 Base Weight 通常保持冻结。训练的主要仍然是 LoRA 参数。

训练和推理 Template 不一致会造成输入分布偏移，尤其是 Role Token、Message Separator、EOS 位置不一致时。

不同 SFT 方案可能使用全序列 Loss 或 Assistant-only Loss。二者不是绝对谁正确，而是训练设计选择。实际使用时必须确认数据处理器如何生成 labels。

Target Modules 太少可能限制适应能力；覆盖过多模块会增加训练参数量。不同模型的模块名称也不同，不能机械复制其他模型的配置。

Preference Pair 的质量直接决定优化方向。若 chosen 与 rejected 区别不明确、标签噪声大或偏好标准不一致，DPO 可能学习错误偏好。

在 Reward-based RL 中，Policy 可能利用 Reward Model 漏洞获得高分，而不是实际提升目标质量。奖励设计与独立评估因此非常重要。

模型在某些目标上被强烈对齐后，可能牺牲原有能力。例如输出变得更保守、知识问答能力下降或生成多样性降低。可以将这种权衡概括为 Alignment Tax。

Post-Training 目标与参数更新方式 Post-Training 通常先进行 SFT，再根据需要进行 Preference Alignment。Preference Alignment 又可以分成两条主要思路：

$$\begin{aligned}\text{Direct Preference:}&\quad \text{DPO / IPO / ORPO / KTO / SimPO / ...,}\\ \text{RL-based:}&\quad \text{PPO / GRPO / ....}\end{aligned}$$

与训练目标并行发展的另一条路线是

参数更新方式可以选择 Full Fine-Tuning 或 PEFT；LoRA 和 QLoRA 都属于 PEFT，QLoRA 通过低比特加载基础模型进一步降低显存占用。

这一演进主要解决大模型训练参数量和显存成本问题。

一个真实 Post-Training 项目如何组合 一个 Post-Training 配置由 Base Model、Data Format、Training Objective、Parameter Update Method 与 Evaluation 共同决定：

$$\begin{aligned}&\text{Base Model} + \text{Data Format} + \text{Training Objective}\\ &+\text{Parameter Update Method} + \text{Evaluation.}\end{aligned}$$

例如：

$$\text{Base LLM} + \text{Chat Template} + \text{SFT Loss} + \text{QLoRA}$$

是一套完整配置；而

$$\text{SFT Model} + \text{Chosen / Rejected Data} + \text{DPO Loss} + \text{LoRA}$$

又是另一套配置。

第 10 章

模型量化：从 FP32 到 INT4 高效推理

前面的章节已经讨论了 Transformer 结构、语言模型训练、微调与对齐，以及 MoE 等扩展结构。当模型真正进入部署阶段后，还会遇到一个非常直接的工程问题：**模型参数需要占用多少显存，推理时需要搬运多少数据，以及在有限硬件上能否以足够高的吞吐运行**。

模型量化（Quantization）的核心思想，是使用更低位宽的数值表示模型权重或激活。例如，将 FP16 权重压缩为 INT8 或 INT4，可以显著降低权重存储和内存带宽需求。但量化并不是简单执行一次类型转换，而是要构造一个从浮点空间到有限离散整数空间的近似映射：

$$x \longrightarrow q \longrightarrow \hat{x},$$

其中 x 是原始浮点值，q 是低比特量化值，$\hat{x}$ 是反量化后的近似值。量化的目标是在降低 bit 数的同时，使 $\hat{x} \approx x$.

对于一个包含 N 个参数的模型，如果只考虑权重本身，理论存储量约为

$$\begin{aligned}
\text{FP32} &: 4N \text{ Bytes}, \\
\text{FP16/BF16} &: 2N \text{ Bytes}, \\
\text{INT8} &: N \text{ Bytes}, \\
\text{INT4} &: \frac{N}{2} \text{ Bytes}.
\end{aligned}$$

常见数值格式包括 FP32、FP16 / BF16、INT8、INT4 与 NF4；位宽降低可以减少存储需求，但不保证推理一定更快。

但 bit 数越低，离散表示能力也越有限，量化误差通常越难控制。

量化真正要研究的不是“把模型变成 INT4”这一句话，而是三个问题：量化到多少 bit、多少个数共享一组量化参数，以及如何控制量化误差。

10.1 为什么 LLM 需要量化

参数规模与权重显存 假设模型包含 N 个参数，每个参数使用 b bit 保存，则理想情况下仅权重所需的存储空间约为

$$M_{\text{weight}} = \frac{Nb}{8} \text{ Bytes.}$$

例如，对于约 7×10^9 个参数的模型，不考虑额外 Metadata、Embedding 特殊处理、运行时 Buffer 和 KV Cache，可以得到下表中的理论值。

格式	每参数 bit	7B 权重理论大小	相对 FP16
FP32	32	约 28 GB	2×
FP16 / BF16	16	约 14 GB	1×
INT8	8	约 7 GB	0.5×
INT4	4	约 3.5 GB	0.25×

这里的数字只是按 bit 数计算的理论权重大小。真实部署还需要存储 Scale、Zero Point、Group Metadata，并为 Attention KV Cache、Activation、CUDA Workspace 等预留显存。

量化为什么可能加速推理 LLM 推理并不只受算术运算次数影响。对于很多 Decoder-only 模型，尤其在单 token Decode 阶段，大量时间消耗在把模型权重从显存读取到计算单元。若权重从 FP16 降到 INT4，理论上同一批参数需要搬运的数据量显著下降，因此有机会降低 Memory Bandwidth 压力。

但低 bit 并不自动意味着更快。真正的速度还取决于：

- GPU / CPU 是否有适合该数据类型的高效 Kernel；
- 是否需要频繁 Dequantization；
- Batch Size 与矩阵 Shape 是否适合量化 GEMM；
- Group Size 与 Scale 读取是否造成额外开销；
- Kernel Fusion 与运行时调度是否成熟。

因此，更准确的关系是

量化的核心权衡 量化通常同时影响四个维度：精度、显存、内存带宽和计算效率。

目标	bit 数降低后的潜在收益	主要风险
权重显存	参数存储显著减少	Scale / Metadata 带来额外开销
内存带宽	每次读取的数据更少	Dequantization 可能抵消部分收益
计算吞吐	合适硬件上可使用低比特 Kernel	并非所有硬件都支持高效 INT4
模型质量	理想情况下接近原模型	bit 越低通常越容易产生 Quantization Error

可以把量化问题概括为：

$$\text{Quality} \quad \leftrightarrow \quad \text{Memory / Bandwidth / Throughput.}$$

数值表示：浮点与整数格式 FP32、FP16 与 BF16 都属于 Floating Point Format，通过 Sign、Exponent 和 Fraction / Mantissa 表示数值；INT8 与 INT4 则直接使用有限个整数 Level。

浮点数的重要特点是具有指数部分，因此可以覆盖非常大的数量级；整数格式没有指数动态范围，若直接把浮点 Tensor 转成整数，会丢失大量信息。因此 Integer Quantization 必须引入 Scale，必要时还需要 Zero Point，将浮点区间映射到有限整数区间。

格式	位宽	类型	主要特点	LLM 中常见作用
FP32	32	Floating	动态范围与精度都较高	优化器状态、部分高精度计算
FP16	16	Floating	尾数较精细，但指数范围明显小于 FP32	混合精度训练、推理
BF16	16	Floating	指数位与 FP32 相同，动态范围大	现代 LLM 训练与计算
INT8	8	Integer	256 个离散整数 Level	Weight / Activation Quantization
INT4	4	Integer	16 个离散 Level	Weight-only LLM Quantization

FP16 与 BF16 的区别 FP16 通常包含 5 个 Exponent Bit，而 BF16 包含 8 个 Exponent Bit，与 FP32 的指数宽度一致。因此 BF16 保留了接近 FP32 的动态范围，但 Fraction Bit 更少。对深度学习而言，这意味着 BF16 通常更不容易因为数值范围不足而 Overflow / Underflow，因此现代大模型训练常采用 BF16 作为主要计算格式。

这里需要区分“低精度浮点”与“整数线性量化”。FP16 / BF16 仍然遵循浮点编码规则；INT8 / INT4 Quantization 则通常需要显式维护 Scale / Zero Point。

Storage Dtype 与 Compute Dtype 量化模型中还必须区分存储格式（Storage Dtype）与计算格式（Compute Dtype）。

例如一个模型可以把 Weight 存为 4-bit，但矩阵乘法内部把局部 Weight 反量化到 FP16 / BF16 后计算；也可以使用真正支持低比特权重乘法的 Kernel。于是“W4A16”通常意味着 Weight 使用 4-bit 表示，而 Activation / Compute 仍在较高精度进行。

常见记号可以统一如下。

记号	Weight	Activation	说明
W16A16	FP16/BF16	FP16/BF16	普通半精度推理
W8A16	INT8	FP16/BF16	Weight-only INT8
W4A16	INT4 / 4-bit	FP16/BF16	常见 LLM Weight-only 4-bit
W8A8	INT8	INT8	Weight 与 Activation 都量化
W4A8	4-bit	8-bit	更激进的混合低比特方案

10.2 Quantization 数学：Scale、Zero Point 与误差

Affine Quantization 设原始浮点数为 x，目标整数区间为 $[q_{\min}, q_{\max}]$. Affine Quantization 的典型形式为

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right),$$

其中 $s > 0$ 是 Scale，z 是 Zero Point。

反量化可以写成

$$\hat{x} = s(q - z).$$

由于 Round 与 Clip 都会丢失信息，一般有 $\hat{x} \neq x$. 定义 Quantization Error 为 $e = x - \hat{x}$.

Scale 决定整数相邻 Level 在浮点空间中相隔多远；Zero Point 决定浮点 0 映射到哪个整数值。

Scale 与 Zero Point 的计算 若使用 Asymmetric Quantization，并希望把 $[x_{\min}, x_{\max}]$ 映射到 $[q_{\min}, q_{\max}]$, 则可取

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}.$$

Zero Point 可以由

$$z \approx q_{\min} - \frac{x_{\min}}{s}$$

计算，然后再 Round / Clip 到合法整数范围。

如果

$$x_{\min} = -2, \qquad x_{\max} = 6,$$

使用 UINT8 区间 $[0, 255]$，则 Scale 约为

$$s = \frac{8}{255}.$$

这意味着量化整数每增加 1，对应浮点空间大约增加 $8/255$。

Symmetric 与 Asymmetric Quantization 对于权重，常见做法是使用 Symmetric Quantization。设

$$a = \max(|x_{\min}|, |x_{\max}|),$$

然后将范围对称化为 $[-a, a]$。若使用 Signed INT8，并采用 $q_{\max} = 127$, 则可以取

$$s = \frac{a}{127}, \qquad z = 0.$$

量化公式简化为

$$q = \text{clip}\left(\text{round}(x/s), -127, 127\right).$$

维度	Symmetric	Asymmetric
浮点范围	$[-a, a]$	$[x_{\min}, x_{\max}]$
Zero Point	通常 0	通常非零
实现复杂度	较低	稍高
区间利用率	分布明显偏向一侧时可能浪费 Level	可以完整覆盖非对称区间
常见场景	Weight Quantization	Activation / 非对称 Tensor

Rounding、Clipping 与 Saturation 量化误差主要来自两个地方。第一类是 Rounding Error：即使 x 位于可表示范围内，也必须映射到最近的离散 Level。第二类是 Clipping Error：如果真实值超出预设范围，就会被截断到 $q_{\min}$ 或 $q_{\max}$。

这构成一个重要权衡。扩大浮点覆盖范围可以减少 Clipping，但会增大 Scale，使相邻 Level 更稀疏；缩小范围可以提高中间区域的分辨率，却会让 Outlier 更容易 Saturate。

因此 Calibration 的核心之一就是寻找合适的 Quantization Range，而不只是机械使用绝对最大值。

量化粒度：Per-Tensor、Per-Channel 与 Per-Group Per-Tensor Quantization 为整个 Tensor 使用同一个 Scale，以及必要时的同一个 Zero Point。对于权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$，有

$$\hat{\boldsymbol{W}} = \text{Dequant}\left(Q(\boldsymbol{W}; s, z)\right),$$

其中所有元素共享同一组 (s, z)。

它的优点是 Metadata 少、实现简单，但如果不同 Channel 的数值范围差异很大，一组 Scale 很难同时适配全部权重。

Per-Channel Quantization Per-Channel Quantization 为不同 Channel 使用独立 Scale。若沿输出 Channel 量化，则第 i 行权重 $\boldsymbol{W}_i \in \mathbb{R}^{d_{\text{in}}}$ 拥有自己的 s_i。于是

$$q_{ij} = \text{round}\left(\frac{\boldsymbol{W}_{ij}}{s_i}\right).$$

如果某些 Channel 的值只分布在 $[-0.1, 0.1]$，另一些却分布在 $[-20, 20]$，Per-Tensor 必须用一个 Scale 同时覆盖两者；Per-Channel 则可以分别设置 Scale，从而显著降低小范围 Channel 的离散误差。

Per-Group Quantization Per-Group 进一步把 Channel 内的权重拆成小组。假设 Group Size 为 G，则一行长度为 d_{in} 的 Weight 会被切分为 $\frac{d_{\text{in}}}{G}$ 组，每一组拥有独立 Scale。

例如

$$\boldsymbol{W} \in \mathbb{R}^{4096 \times 4096}, \qquad G = 128,$$

则每一行有 $4096/128 = 32$ 组 Scale。

更小的 Group Size 通常意味着：

但同时 Metadata 与 Kernel 处理成本也会增加。

Granularity	Scale 数量	优点	代价
Per-Tensor	最少	简单、Metadata 少	难以适应不同 Channel 分布
Per-Channel	每个 Channel 一组	精度通常明显更好	Scale 数增加
Per-Group	每个小 Group 一组	很适合 INT4 Weight Quantization	Metadata 与 Kernel 更复杂

Weight、Activation 与量化难点：Weight-only Quantization Weight-only Quantization 只压缩模型权重，Activation 保持 FP16 / BF16。典型形式为

$$\text{W4A16} \quad \text{或} \quad \text{W8A16}.$$

这种方式在 LLM 中很常见，因为权重是模型显存和 Decode 阶段内存带宽的重要来源，而 Activation Quantization 往往更容易受到动态范围和 Outlier 影响。

Activation Quantization 如果 Activation 也量化，例如 W8A8，那么推理 Kernel 有机会进一步减少数据搬运并利用整数矩阵乘法。但 Activation 是输入相关的，不同 token、不同 Batch、不同层的分布都可能变化，因此 Scale 选择更困难。

对于 Activation X，如果存在少量极大 Outlier：

$$|x_{\text{outlier}}| \gg |x_{\text{normal}}|,$$

为了覆盖 Outlier 而增大 Scale，会让大量普通值只能使用很少的离散 Level。

Outlier 为什么破坏低比特量化 考虑两个 Channel：

$$X_1 \in [-0.2, 0.3], \qquad X_2 \in [-50, 60].$$

如果二者共享同一个 INT8 Scale，Scale 必须覆盖约 $[-60, 60]$。此时对 X_1 来说，相邻量化 Level 可能已经接近其全部有效变化范围，导致细节严重丢失。

这就是为什么 LLM Quantization 经常强调：

- Per-Channel / Per-Group Scale；
- Outlier-aware Calibration；
- Activation-aware Weight Scaling；
- 更稳健的 Clipping Range。

Static 与 Dynamic Quantization Static Quantization 在推理前通过 Calibration Data 估计 Activation Range，并固定量化参数；Dynamic Quantization 则在运行时根据当前 Activation 动态计算 Scale。

维度	Static Quantization	Dynamic Quantization
Scale 计算时间	推理前	运行时
Calibration	通常需要	可以减少依赖
Runtime Cost	较低	需要额外统计与量化步骤
适应输入变化	较弱	更灵活

10.3 PTQ 与 QAT

PTQ：Post-Training Quantization PTQ 指模型完成训练后，再进行量化：

PTQ 使用校准样本估计量化参数，再将训练完成的模型转换为量化模型。

PTQ 通常不需要重新执行完整训练，因此成本低，非常适合部署阶段。典型流程是：

1. 读取已经训练好的 FP16 / BF16 模型；
2. 使用少量代表性 Calibration Data 运行部分 Forward；
3. 统计 Weight / Activation 分布；
4. 决定 Scale、Zero Point、Clipping Range 与 Group Size；
5. 生成 INT8 / INT4 权重与量化 Metadata；
6. 使用 Quantized Kernel 进行推理。

QAT：Quantization-Aware Training QAT 在训练过程中主动模拟量化误差。训练参数仍可以保存为高精度，但 Forward 中插入 Fake Quantization：

$$x \longrightarrow Q(x) \longrightarrow \text{Dequant}(Q(x)) \longrightarrow \hat{x}.$$

模型在训练阶段就会看到量化后的近似值，因此可以主动调整参数以适应低比特表示。

量化中的 Round 函数几乎处处不可导。QAT 常用 Straight-Through Estimator（STE）近似处理反向传播。可以概念化为：前向执行 $q = \text{round}(x)$，而反向把局部

梯度近似当作 $\frac{\partial q}{\partial x} \approx 1$. STE 不是 Round 的真实导数，而是一种让优化可以继续进行的梯度近似。

PTQ 与 QAT 对比

维度	PTQ	QAT
量化时机	训练完成后	训练过程中模拟量化
额外训练成本	很低	较高
数据需求	少量 Calibration Data 即可	需要训练 / 微调数据
低 bit 适应能力	依赖算法与 Calibration	模型可主动适应量化误差
工程复杂度	较低	较高

PTQ 与 QAT 是“什么时候量化、是否让模型在训练中适应量化误差”的分类；GPTQ 和 AWQ 则是更具体的 PTQ 算法路线。

10.4 GPTQ 与 AWQ：低比特权重量化

GPTQ 的目标 GPTQ 是面向 Transformer / LLM 的 Post-Training Weight Quantization 方法。它不是简单对每个 Weight 独立 Round，而是关注量化一部分权重后，如何利用剩余权重补偿误差。

设某线性层输出为 $\boldsymbol{Y} = \boldsymbol{W}\boldsymbol{X}$, 量化后为 $\hat{\boldsymbol{Y}} = \hat{\boldsymbol{W}}\boldsymbol{X}$. 一个自然的 Layer Reconstruction 目标是

$$\min_{\hat{\boldsymbol{W}}} \|\boldsymbol{W}\boldsymbol{X} - \hat{\boldsymbol{W}}\boldsymbol{X}\|_F^2.$$

因此量化误差并不是只看 $\|\boldsymbol{W} - \hat{\boldsymbol{W}}\|$, 还要考虑输入 $\boldsymbol{X}$：某些方向的 Weight Error 对实际层输出影响更大。

二阶信息与逐步量化 对线性层重建误差而言，输入统计可以形成近似二阶信息，常见写法与 $\boldsymbol{H} \propto \boldsymbol{X}\boldsymbol{X}^\top$ 相关。GPTQ 使用这类 Hessian / Second-order Information，在逐列或分块量化 Weight 时估计当前 Quantization Error 对剩余参数的影响，并更新尚未量化的权重以补偿误差。

GPTQ 中常见的工程参数 实际 GPTQ 配置经常涉及：

- bit width，例如 4-bit；
- Group Size，例如 32 / 64 / 128；
- Symmetric / Asymmetric Quantization；
- Calibration Dataset；
- 是否对某些 Layer 单独处理。

因此 GPTQ 更准确的理解是：**一种利用输入统计和误差补偿进行低比特 Weight-only PTQ 的方法**，而不是“INT4 文件格式”的同义词。

AWQ：Activation-Aware Weight Quantization AWQ 的核心观察是：并不是所有 Weight Channel 对模型输出都同样敏感。Activation 的幅度可以帮助识别对输出更重要的 Weight 方向，因此量化时不应该只观察 Weight 自身分布。

设线性层为 $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$. 引入可逆的对角缩放矩阵 $\boldsymbol{S}$：

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{S}\boldsymbol{S}^{-1}\boldsymbol{x}.$$

在不量化时，这只是恒等重参数化；但在量化 $\boldsymbol{W}\boldsymbol{S}$ 之后，不同 Channel 的 Scale 会改变 Weight 的量化难度。因此可以寻找合适的 $\boldsymbol{S}$，使重要 Channel 在低 bit Quantization 下得到更好的保护。

Salient Weight 与 Scaling 可以把 AWQ 的逻辑简化为

这里的“Activation-Aware”并不意味着最终一定把 Activation 本身量化成 INT4。它主要是利用 Activation Statistics 帮助选择更好的 Weight Quantization 策略。

GPTQ 与 AWQ 对比

维度	GPTQ	AWQ
分类	PTQ / Weight-only	PTQ / Weight-only
主要依据	输入统计与二阶误差信息	Activation Statistics 与 Channel Sensitivity
核心思想	量化后补偿剩余 Weight	通过 Scaling 保护重要 Weight Channel
常见目标	INT4 / Low-bit Weight	INT4 / Low-bit Weight
是否等于文件格式	否	否

INT4、NF4 与低比特表示：普通 INT4 4-bit 一共只有 $2^4 = 16$ 种离散编码。如果使用 Signed Integer，可以采用类似 $[-8, 7]$ 的整数区间；在对称量化中也常根据具体实现使用对称的有效 Level。

当 Group 内的 Weight 范围较大时，只有 16 个 Level 很容易产生较明显 Rounding Error，因此 INT4 比 INT8 更依赖 Per-Group Scale、Calibration 与量化算法。

NF4：NormalFloat 4-bit NF4（NormalFloat 4-bit）不是普通的均匀 INT4。它使用 4-bit 编码，但离散 Level 的设计针对近似正态分布的权重，使有限的 16 个 Level 更适配 Normal-like Weight Distribution。

因此应区分：

$$\text{INT4} \neq \text{NF4}.$$

二者都是 4-bit 表示，但 Quantization Codebook / Level 分布不同。

维度	Uniform INT4	NF4
bit 数	4	4
Level 分布	近似均匀整数间隔	非均匀、面向 Normal-like Distribution
常见用途	推理 Weight Quantization	QLoRA 中的 4-bit Base Weight
是否等于普通 INT4	是其典型形式	否

Double Quantization 量化 Weight 本身需要保存 Scale。如果模型使用很小的 Group Size，Scale 数量可能很多。Double Quantization 的思路是进一步压缩这些 Quantization Constants，从而减少 Scale Metadata 的平均存储开销。

这再次说明，模型的实际压缩率不仅取决于权重位宽，还取决于 Scale、Zero Point 及其他元数据的存储开销。

BitsAndBytes 与 GGUF BitsAndBytes 更准确的定位是 Low-bit Quantization Library / Backend，而不是一条像 $q = \text{round}(x/s)$ 这样的独立量化数学公式。它在 Hugging Face 生态中常用于 8-bit / 4-bit 模型加载，以及 QLoRA 中的量化 Base Model。

一个常见的 4-bit 配置形式如下。该代码依赖 `transformers`、`bitsandbytes` 与可用硬件环境，因此这里只作为接口示例，不作为本章纯 PyTorch 测试的一部分。

```python
import torch
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=quant_config,
    device_map="auto",
)
```

这里尤其需要区分：

- `load_in_4bit=True` 描述 Weight Storage / Quantization；
- `bnb_4bit_compute_dtype=torch.bfloat16` 描述计算时使用的高精度 Dtype；
- NF4 描述 4-bit Quantization Level 的设计；
- Double Quantization 描述对量化常数的进一步压缩。

GGUF：模型文件格式与部署载体 GGUF 与 GPTQ / AWQ 不在同一分类层级。GGUF 是面向本地推理生态的模型文件格式，用于保存 Tensor、Tokenizer / Metadata，以及多种量化后的权重表示。它常与 `llama.cpp` 一类运行时结合使用。

因此应当把 GPTQ / AWQ 理解为 Quantization Algorithm，把 GGUF 理解为 Model File / Tensor Storage Format，把 BitsAndBytes 理解为 Quantization Library / Runtime Backend。

GGUF 生态中经常出现 Q8、Q6、Q5、Q4、Q3、Q2 以及不同 K-quant Variant。这里的命名描述一套具体 Tensor Quantization / Storage Scheme，而不能简单解释成“模型里的每一个参数都严格使用同一种普通 INT4”。

统一分类

名称	更准确的分类	主要作用
PTQ	Quantization Strategy	训练结束后量化
QAT	Training Strategy	训练中模拟量化误差
GPTQ	PTQ Algorithm	二阶信息 / 误差补偿式 Weight Quantization
AWQ	PTQ Algorithm	Activation-aware Weight Scaling 与量化
BitsAndBytes	Software Library / Backend	8-bit / 4-bit 加载与低比特计算
GGUF	Model File Format	保存模型、Metadata 与多种量化 Tensor

10.5 Quantized Linear 与真实推理 Kernel

Fake Quantization 与 Real Quantization Fake Quantization 先将权重映射为低比特表示，再反量化为 $\hat{\boldsymbol{W}}$；矩阵乘法仍然使用 Float Tensor。这种方式适合验证算法，却不意味着获得了真实 INT4 Kernel 的显存带宽和速度收益。

Real Quantization 需要：

- 把多个 4-bit Value Pack 到真实字节 / Word 中；
- 在 Kernel 中读取 Packed Weight 与 Scale；
- 高效完成 Dequantize-on-the-fly 或 Integer / Mixed GEMM；
- 尽量避免把整个 $\boldsymbol{W}$ 完整恢复为 FP16 后再计算。

Quantized Linear 的两种实现路线 原始 Linear 为 $y = x\boldsymbol{W}^{\mathsf{T}} + b$. 使用反量化权重时，Quantized Linear 的计算为

$$y = x\hat{\boldsymbol{W}}^{\mathsf{T}} + b, \qquad \hat{\boldsymbol{W}} = \mathrm{Dequant}(Q_W).$$

高性能推理则更接近

$$(x, Q_W, s, z) \longrightarrow \text{Quantized GEMM Kernel} \longrightarrow y,$$

其中 Weight 不需要在 Python 层完整 Dequantize。

实现	优点	局限
Fake / Dequant Linear	易理解、易验证量化误差	不代表真实低比特推理性能
Packed Quantized Kernel	可真正降低 Weight Traffic 并获得硬件加速	Kernel 与硬件依赖强、实现复杂

量化显存、速度与 KV Cache：权重显存不是总显存 即使将 7B 模型的理论 Weight 从 FP16 的约 14 GB 压缩到 INT4 的约 3.5 GB，实际 GPU Memory 仍可能明显高于 3.5 GB，因为运行时还包含：

- Quantization Scale / Zero Point / Metadata；
- KV Cache；
- Activation 与临时 Tensor；
- CUDA / ROCm Workspace；
- Sampling、Logits 与 Runtime Buffer。

INT4 Weight Size 不等于 Total Runtime GPU Memory；实际显存还包括前文列出的其他开销。

Weight Quantization 与 KV Cache 是两个维度 模型 Weight Quantization 主要降低 Model Parameter Memory. KV Cache 则来自历史 token 的 Key / Value，主要与序列长度、层数、KV Head 数量和 Cache Dtype 有关。

如果 Attention 使用 MHA，其 KV Cache 量级可概念化为

$$M_{\mathrm{KV}} \propto 2BLh_{\mathrm{kv}}d_{\mathrm{head}} \times \text{layers} \times \text{bytes per element}.$$

因此即使 Weight 已经 INT4，当 L 很大时，KV Cache 仍可能成为显存瓶颈。

前面学习过的 MQA / GQA / MLA 与 Quantization 可以从两个不同方向优化：

$$\begin{gathered}\text{MQA / GQA : 减少 KV Head 数量,}\\ \text{MLA : 压缩 KV Representation,}\\ \text{KV Quantization : 降低每个 Cache 元素 bit 数.}\end{gathered}$$

为什么低 bit 不一定更快 一个 INT4 模型如果运行在缺少高效 INT4 Kernel 的硬件上，可能需要频繁 Unpack / Dequantize，反而无法得到预期速度。另一方面，成熟的 Weight-only INT4 Kernel 可以减少内存带宽并提高 Decode 吞吐。

因此评估量化效果时，应该至少同时记录：

- Model Quality / Perplexity / Task Score；
- Weight Memory 与峰值 GPU Memory；
- Prefill Tokens/s；
- Decode Tokens/s；
- First-token Latency 与 per-token Latency；
- 目标硬件和具体 Kernel Backend。

10.6 Python 实现：基础 INT8 Quantization

Symmetric Per-Tensor Quantization Symmetric INT8 Quantization 将浮点值映射为整数，并记录 Scale；Dequantization 再通过 Scale 恢复近似浮点值。

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

def symmetric_quantize(x, bits=8):
    qmax = 2 ** (bits - 1) - 1
    scale = x.abs().max() / qmax
    scale = scale.clamp_min(torch.finfo(x.dtype).eps)
    q = torch.round(x / scale).clamp(-qmax, qmax)
    return q.to(torch.int8), scale

def symmetric_dequantize(q, scale):
    return q.float() * scale
```

如果 x 的最大绝对值为 a，该实现对应 $s = \frac{a}{2^{b-1}-1}$.

Affine Quantization Asymmetric Affine Quantization 需要同时计算 Scale 与 Zero Point：

```python
def affine_quantize(x, bits=8):
    qmin, qmax = 0, 2 ** bits - 1
    xmin, xmax = x.min(), x.max()
    scale = (xmax - xmin) / (qmax - qmin)
    scale = scale.clamp_min(torch.finfo(x.dtype).eps)
    zero = torch.round(qmin - xmin / scale).clamp(qmin, qmax)
    q = torch.round(x / scale + zero).clamp(qmin, qmax)
    return q.to(torch.uint8), scale, zero

def affine_dequantize(q, scale, zero):
    return scale * (q.float() - zero)
```

量化误差测试

```python
def test_basic_quantization():
    torch.manual_seed(42)
    x = torch.randn(1024)

    q_sym, s_sym = symmetric_quantize(x)
    x_sym = symmetric_dequantize(q_sym, s_sym)

    q_aff, s_aff, z_aff = affine_quantize(x)
    x_aff = affine_dequantize(q_aff, s_aff, z_aff)

    mse_sym = F.mse_loss(x_sym, x).item()
    mse_aff = F.mse_loss(x_aff, x).item()

    print("symmetric MSE:", mse_sym)
    print("affine MSE:   ", mse_aff)
```

这里的 MSE 只衡量 Tensor Reconstruction Error。真实 LLM 量化仍需要使用 Perplexity、Benchmark 或生成质量验证模型级影响。

Python 实现：Per-Channel、Group-wise INT4 与 Quantized Linear 对于二维 Weight Matrix，可以为每个输出 Channel 分别计算 Scale：

```python
def per_channel_quantize(w, bits=8):
    qmax = 2 ** (bits - 1) - 1
    scale = w.abs().amax(dim=1, keepdim=True) / qmax
    scale = scale.clamp_min(torch.finfo(w.dtype).eps)
    q = torch.round(w / scale).clamp(-qmax, qmax)
    return q.to(torch.int8), scale

def per_channel_dequantize(q, scale):
    return q.float() * scale
```

如果 $\boldsymbol{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, 则

$$\texttt{scale.shape} = [d_{\text{out}}, 1].$$

Group-wise INT4 Quantization PyTorch 没有可直接表示所有真实部署格式的通用 Signed INT4 Tensor Dtype。此处用 `int8` Tensor 保存 $[-7, 7]$ 中的离散值，以演示 4-bit Quantization Math；该实现不包含真实 4-bit Packing。

```python
def groupwise_int4_quantize(w, group_size=128):
    out_dim, in_dim = w.shape
    assert in_dim % group_size == 0

    groups = w.view(out_dim, -1, group_size)
    scale = groups.abs().amax(dim=-1, keepdim=True) / 7
    scale = scale.clamp_min(torch.finfo(w.dtype).eps)

    q = torch.round(groups / scale).clamp(-7, 7)
    q = q.to(torch.int8)
    return q, scale

def groupwise_int4_dequantize(q, scale, shape):
    w = q.float() * scale
    return w.view(shape)
```

这里每个 Group 使用一个 Scale。如果 Group Size 从 128 减小到 32，通常会提高局部拟合能力，但 Scale 数量也增加 4 倍。

Weight-only INT8 Quantized Linear 的简化实现 该 Weight-only INT8 Quantized Linear 在初始化时量化 Weight，并在 Forward 时反量化；它用于说明计算路径，不包含高性能量化 Kernel。

```
class QuantizedLinear(nn.Module):
    def __init__(self, linear):
        super().__init__()
        qweight, scale = per_channel_quantize(linear.weight.detach())
        self.register_buffer("qweight", qweight)
        self.register_buffer("scale", scale)

        if linear.bias is None:
            self.bias = None
        else:
            self.register_buffer("bias", linear.bias.detach().clone())

    def forward(self, x):
        weight = per_channel_dequantize(self.qweight, self.scale)
        return F.linear(x, weight, self.bias)
```

真实量化 Runtime 会避免在每次 Forward 中完整创建 Float Weight，而是在底层 Kernel 中直接使用 Packed Weight 和 Scale。

完整测试与误差比较

```
def test_quantization():
    torch.manual_seed(42)

    w = torch.randn(64, 256)

    q_tensor, s_tensor = symmetric_quantize(w)
    w_tensor = symmetric_dequantize(q_tensor, s_tensor)

    q_channel, s_channel = per_channel_quantize(w)
    w_channel = per_channel_dequantize(q_channel, s_channel)

    q4, s4 = groupwise_int4_quantize(w, group_size=64)
    w4 = groupwise_int4_dequantize(q4, s4, w.shape)

    print("INT8 per-tensor MSE :", F.mse_loss(w_tensor, w).item())
```

```
    print("INT8 per-channel MSE:", F.mse_loss(w_channel, w).item())
    print("INT4 group-wise MSE :", F.mse_loss(w4, w).item())

    linear = nn.Linear(256, 64)
    qlinear = QuantizedLinear(linear)
    x = torch.randn(2, 8, 256)

    y_fp = linear(x)
    y_q = qlinear(x)

    print("Linear output shape:", y_q.shape)
    print("Linear output MSE:  ", F.mse_loss(y_q, y_fp).item())

    assert y_q.shape == y_fp.shape
    print("all tests passed")

if __name__ == "__main__":
    test_basic_quantization()
    test_quantization()
```

量化粒度可选 Per-Tensor、Per-Channel 或 Per-Group，随后由匹配的 Quantized Linear Kernel 执行计算。

10.7 量化方法的比较与常见误区

主要方法统一比较

方法 / 名称	分类	核心思想	典型用途
PTQ	Strategy	训练完成后确定量化参数	快速部署
QAT	Training Strategy	训练中插入 Fake Quantization	更强的低 bit 适应
GPTQ	PTQ Algorithm	二阶信息 + Error Compensation	Low-bit Weight-only
AWQ	PTQ Algorithm	Activation-aware Channel Scaling	Low-bit Weight-only
NF4	4-bit Number / Codebook	Normal-aware Non-uniform Levels	QLoRA Base Weight
BitsAndBytes	Library / Backend	提供 8-bit / 4-bit 加载与计算	HF / QLoRA 生态
GGUF	File Format	保存模型 Metadata 与量化 Tensor	本地 CPU / GPU 推理生态

量化与其他优化方法 量化只是在一个维度上降低数值表示成本。它可以与前面的 Attention、MoE 和参数高效微调技术组合。

技术	主要减少 / 改变什么	与 Quantization 的关系
Quantization	Weight / Activation bit 数	直接降低表示位宽
MQA / GQA	KV Head 数量	可与 Weight / KV Quantization 同时使用
MLA	KV Representation	可进一步配合 Cache Quantization
MoE	单 token Active Parameters	Expert Weight 仍可再量化
LoRA	Trainable Parameters	可形成 QLoRA：量化 Base + Low-rank Adapter
Pruning	参数或结构本身	与低 bit 表示是不同压缩维度

常见误区

常见误区	更准确的理解
“FP16 就是 INT16”	FP16 是浮点格式，INT16 是整数格式，编码与动态范围完全不同
“INT4 模型所有运算都是 INT4”	Weight 可以 4-bit 存储，而 Activation / Accumulation 仍可能是 FP16/BF16
“INT4 显存一定是 FP16 的严格四分之一”	还需要 Scale、Metadata、KV Cache 和 Runtime Buffer
“GGUF 是一种量化算法”	GGUF 是文件 / Tensor 存储格式，可包含多种量化表示
“BitsAndBytes 是一种公式”	它是低比特加载和计算的软件库 / Backend
“GPTQ / AWQ 是文件格式”	二者是 Weight Quantization Algorithm
“所有量化都必须有非零 Zero Point”	Symmetric Quantization 常使用 $z = 0$
“bit 越低一定越快”	速度还取决于硬件支持、Kernel、Batch、Memory Bandwidth 与 Dequantization Cost
“Tensor MSE 越低，LLM 一定越好”	最终仍需要 Perplexity、任务指标和 Generation Quality 验证

分析一个量化模型时的五个问题 看到一个“4-bit LLM”时，不应只问“它是不是 INT4”，而应该进一步确认：

1. 哪些 Tensor 被量化：Weight、Activation 还是 KV Cache？
2. 具体 bit / Number Format 是什么：INT8、INT4、NF4 还是其他格式？
3. Granularity 是 Per-Tensor、Per-Channel 还是 Per-Group，Group Size 是多少？
4. 使用什么算法或 Calibration：普通 PTQ、GPTQ、AWQ、QAT 等？
5. Runtime 是否有真正匹配该格式的高效 Kernel？

低比特量化的工程权衡 量化方法的核心不是单纯追求更低 bit，而是在降低位宽的同时控制 Quantization Error，并让硬件通过更低存储和带宽开销兑现性能收益。

其中算法发展的核心不是不断追求更低 bit，而是在更低 bit 下继续控制 Quantization Error，并让硬件真正把低 bit 转化为

$$\text{Lower Memory} + \text{Lower Bandwidth} + \text{Higher Throughput}.$$

第 11 章

FlashAttention：从标准 Attention 到 IO-Aware Attention

前面的 QKV 注意力机制章节已经建立了 MHA、MQA、GQA 与 MLA 的结构视角：它们主要改变 Query、Key、Value 的组织方式，或者改变 KV Cache 的表示方式。但即使注意力公式本身完全不变，实际运行速度仍然会受到另一个问题限制：**同一个数学公式，在 GPU 上采用不同的计算顺序与内存访问方式，性能可能相差很大**。

标准缩放点积注意力写为

$$\text{Attention}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\top}}{\sqrt{d_{\text{head}}}}\right)\boldsymbol{V}.$$

若

$$\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V} \in \mathbb{R}^{B\times H\times L\times d_{\text{head}}},$$

则 Attention Score 的 Shape 为 $[B,H,L,L]$. 当序列长度 L 增大时，完整 Score Matrix 与 Softmax Matrix 都会以 L^2 的规模增长。传统实现往往需要把这些中间结果写入 GPU 的高带宽显存，再从显存读取回来继续执行 Softmax 与矩阵乘法。

FlashAttention 的核心并不是近似 Attention，也不是改变

$$\text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\top}}{\sqrt{d_{\text{head}}}}\right)\boldsymbol{V}$$

这一数学目标，而是重新设计计算顺序：将 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 切成较小的 Tile，把局部数据加载到片上快速存储中，通过 Online Softmax 分块累计结果，从而避免把完整 $L\times L$ Attention Matrix 反复写入和读出 HBM。

因此，FlashAttention 的计算路径为

FlashAttention 将 Attention 分块计算，并使用 Online Softmax 累积结果，避免把完整的 Attention Matrix 反复写入和读出 HBM。

理解 FlashAttention 最重要的一点是：它优化的是 Attention 的实现与 IO，而不是把标准 Attention 换成一个近似公式。

11.1 标准 Attention 的计算与显存问题

从 Q、K、V 到 Attention Matrix 设

$$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{B\times H\times L\times D},$$

其中 $D = d_{\text{head}}$。标准 Attention 通常按下面的顺序计算：

$$\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}} \longrightarrow \text{Scaling} \longrightarrow \text{Mask} \longrightarrow \text{softmax} \longrightarrow \times \boldsymbol{V}.$$

首先得到 Score：

$$\boldsymbol{S} = \frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{D}},$$

其 Shape 为 $\boldsymbol{S} \in \mathbb{R}^{B\times H\times L\times L}$. 然后计算 $\boldsymbol{P} = \text{softmax}(\boldsymbol{S})$, 最后得到 $\boldsymbol{O} = \boldsymbol{P}\boldsymbol{V}$.

如果使用 Causal Attention，还需要在 Softmax 之前对未来位置执行 Mask：

$$S_{ij} = -\infty, \qquad j > i.$$

L^2 中间 Tensor Attention 的算术复杂度通常写成 $O(L^2D)$, 但从实现角度看，还必须关注中间 Tensor：$\boldsymbol{S}, \boldsymbol{P} \in \mathbb{R}^{B\times H\times L\times L}$.

忽略其他 Buffer，仅考虑一个 $L \times L$ 矩阵，每个元素使用 b Byte，则单个 Head 的理论存储量为 $M = L^2 b$. 当序列长度从 L 增长到 $2L$ 时，中间矩阵规模变成 $(2L)^2 = 4L^2$.

因此，长上下文 Attention 的问题并不仅仅是计算量增加，还包括大量中间结果的读写。

对象	Shape	主要作用
$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$	$[B, H, L, D]$	输入表示
Score $\boldsymbol{S}$	$[B, H, L, L]$	Query 与 Key 的两两打分
Probability $\boldsymbol{P}$	$[B, H, L, L]$	Softmax 后的注意力权重
Output $\boldsymbol{O}$	$[B, H, L, D]$	Value 的加权结果

标准实现中的数据移动 一个直观的普通实现可以理解为

$$\boldsymbol{Q}, \boldsymbol{K} \longrightarrow \boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}} \longrightarrow \boldsymbol{S}$$

$$\boldsymbol{S} \longrightarrow \text{HBM} \longrightarrow \text{Read } \boldsymbol{S} \longrightarrow \text{softmax} \longrightarrow \boldsymbol{P}$$

$$\boldsymbol{P} \longrightarrow \text{HBM} \longrightarrow \text{Read } \boldsymbol{P}, \boldsymbol{V} \longrightarrow \boldsymbol{P}\boldsymbol{V}.$$

虽然不同框架会进行 Kernel Fusion，并不一定严格按照这个最朴素的顺序执行，但这个模型能够帮助理解 FlashAttention 的动机：**若完整 S 和 P 被频繁物化到大容量显存中，Attention 会产生大量 Memory Traffic**。

GPU Memory Hierarchy 与 IO 瓶颈：HBM、片上存储与 Register GPU 中的数据并不是存储在一个统一速度的内存中。为了理解 FlashAttention，可以使用一个简化层级：

存储层级	容量	相对访问速度	Attention 中的角色
HBM / Global Memory	大	较慢	保存模型、$\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 与大规模 Tensor
Shared Memory / SRAM	小	快	保存当前 Tile 与局部中间值
Register	最小	很快	每个线程的局部累加与标量状态

这里的重点不是记住某一张 GPU 的具体容量或带宽，而是理解一个稳定的硬件事实：

$$\text{Large Capacity} \quad \Longleftrightarrow \quad \text{Higher Data-Movement Cost.}$$

Compute-Bound 与 Memory-Bound 如果一个 Kernel 的主要时间消耗在执行矩阵乘法等算术操作，可以称为偏 Compute-Bound；如果大量时间消耗在从内存读取数据、写回中间结果，则可能偏 Memory-Bound。

因此，一个算法的运行时间不只由 FLOPs 决定。可以粗略写成

$$T \approx \max\left(T_{\text{compute}}, T_{\text{memory}}\right),$$

这不是严格的性能模型，但足以说明：即使 GPU 有很强的矩阵乘法能力，如果每一步都等待大规模数据从 HBM 搬运，计算单元仍可能无法被充分利用。

FlashAttention 的“IO-Aware”意味着算法设计时主动考虑 GPU 存储层级，而不只看数学运算次数。

11.2 IO-Aware Attention 与 Online Softmax

从完整物化到分块计算 FlashAttention 不直接构造并长期保存完整 $L \times L$ Attention Matrix，而是把矩阵拆成较小的 Tile。设 Query 被分成 $Q_1, Q_2, \ldots, Q_m$, Key 和 Value 被分成 $(K_1, V_1), (K_2, V_2), \ldots, (K_n, V_n)$.

对于一个 Query Tile Q_i，算法依次处理各个 KV Tile：

$$Q_i K_1^\mathsf{T}, \quad Q_i K_2^\mathsf{T}, \quad \ldots, \quad Q_i K_n^\mathsf{T}.$$

每次只需要将当前 Tile 加载到片上存储，完成局部 Score、Softmax 统计量和 Value 累加，然后继续处理下一个 Tile。

为什么不能直接对每个 Tile 单独 Softmax 如果把 Score 拆为两个块 $s = [s^{(1)}, s^{(2)}]$, 不能简单计算 softmax($s^{(1)}$) 和 softmax($s^{(2)}$) 后直接拼接，因为 Softmax 分母必须包含所有 Key：

$$\sum_j \mathrm{e}^{s_j}.$$

因此，Tiling 本身并不足以得到正确 Attention。真正关键的是：**如何在不一次保存全部 Score 的情况下，仍然计算全局 Softmax**。这正是 Online Softmax 的作用。

Online Softmax：数值稳定 Softmax 普通 Softmax 为

$$p_i = \frac{\mathrm{e}^{x_i}}{\sum_j \mathrm{e}^{x_j}}.$$

为了避免指数溢出，通常减去最大值 $m = \max_j x_j$, 得到

$$p_i = \frac{\mathrm{e}^{x_i - m}}{\sum_j \mathrm{e}^{x_j - m}}.$$

定义归一化因子

$$\ell = \sum_j \mathrm{e}^{x_j - m}.$$

那么一行 Softmax 可以通过二元状态 (m, ℓ) 来描述其全局统计量。

分块更新最大值与归一化因子 假设已经处理完前面的 Score Block，并维护

$$m_{\text{old}}, \qquad \ell_{\text{old}}.$$

对于新的 Score Block s，先计算

$$m_{\text{tile}} = \max(s),$$

然后更新

$$m_{\text{new}} = \max\left(m_{\text{old}}, m_{\text{tile}}\right).$$

旧的归一化因子需要重新缩放：

$$\ell_{\text{new}} = \mathrm{e}^{m_{\text{old}} - m_{\text{new}}} \ell_{\text{old}} + \sum_j \mathrm{e}^{s_j - m_{\text{new}}}.$$

这一步非常重要，因为当新的 Tile 出现更大的 Score 时，之前所有指数项的参考最大值都发生了变化。

同时累计 Attention Output 只计算 Softmax 分母还不够。Attention 最终需要

$$O = \text{softmax}(S)V.$$

对于每一个 Query Row，可以维护未归一化累加量 A_{old}. 处理新 Tile 时：

$$A_{\text{new}} = \mathrm{e}^{m_{\text{old}} - m_{\text{new}}} A_{\text{old}} + \mathrm{e}^{S_{\text{tile}} - m_{\text{new}}} V_{\text{tile}}.$$

所有 Tile 处理完成后：

$$O = \frac{A}{\ell}.$$

因此算法只需要不断维护 $m, \qquad \ell, \qquad A$, 而不需要在 HBM 中保留完整 Softmax Matrix。

Online Softmax 的本质是维护可组合的全局统计量，使分块处理后的结果仍与完整 Softmax 数学等价。

FlashAttention 的完整数据流：一个 Query Tile 的处理过程 对于 Query Tile Q_i，完整过程可以表示为

$$
\begin{aligned}
Q_i &\longrightarrow \text{Load On-Chip} \\
(K_j, V_j) &\longrightarrow \text{Load On-Chip} \\
S_{ij} &= \frac{Q_i K_j^{\mathsf{T}}}{\sqrt{D}} \\
&\longrightarrow \text{Update } m_i, \ell_i, A_i \\
&\longrightarrow \text{Next KV Tile.}
\end{aligned}
$$

遍历完全部 KV Tile 后：$O_i = \frac{A_i}{\ell_i}$.

Causal Mask 与 Tile Causal Attention 要求 $j \le i$。在 Tile 级别，可以跳过完全位于未来区域的 KV Tile；对于与因果边界相交的 Tile，则在 Tile 内部继续使用逐元素 Mask。

因此，Causal Mask 并不会改变 FlashAttention 的基本思想，只是决定哪些 Score 元素可以参与 Online Softmax。

Backward 为什么也能节省中间显存 训练阶段还需要反向传播。普通自动微分如果保存完整 Probability Matrix，会继续占用大量显存。FlashAttention 的另一个重要思想是：反向时可以利用保存的必要统计量与输入重新计算部分局部结果，而不是把所有 $L \times L$ 中间矩阵都保存在前向缓存中。

反向传播时只保存必要状态，并重新计算开销较低的局部结果，避免缓存全部中间矩阵。

FlashAttention 为什么是 Exact Attention：没有改变目标公式 FlashAttention 的目标仍然是

$$
O = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{D}}\right) V.
$$

Tiling、Online Softmax 和 Kernel Fusion 只改变：

- 矩阵被分成多大 Block；
- Block 按什么顺序被计算；
- 哪些中间数据放在片上存储；
- 哪些量被重算而不是长期保存；
- HBM 读写次数如何减少。

它们没有把标准 Attention Matrix 替换成一个低秩或稀疏近似。

与近似 Attention 的区别

方法	是否保持标准 Attention 目标	核心变化
Standard Attention	是	直接物化中间矩阵
FlashAttention	是	IO-Aware Tiling + Online Softmax
Sparse Attention	通常改变可连接位置集合	减少 Attention Pair
Linear Attention	改写 Attention 计算形式	使用 Kernel / Associativity 等降低复杂度

FlashAttention 的“快”来自更好的数据移动与 Kernel 调度，而不是通过减少 Attention Pair 来换取近似结果。

复杂度分析：FLOPs、显存与 IO 标准 Dense Attention 的主要矩阵乘法仍然包含 $\boldsymbol{QK}^{\mathsf{T}}$ 与 $\boldsymbol{PV}$，因此对序列长度的算术复杂度仍大致是 $O(L^2D)$。

FlashAttention 并没有一般性地把它变成 $O(LD)$。

真正改变的是：不再需要把完整 $L \times L$ Score / Probability Matrix 作为长期中间 Tensor 保存到 HBM。

三种复杂度要分开 学习推理优化时，建议同时区分：

维度	问题	FlashAttention 的影响
Arithmetic Complexity	做多少数学运算	Dense Attention 主体仍是二次复杂度
Intermediate Memory	保存多少中间 Tensor	避免保存完整 Score / Probability Matrix
Memory IO	HBM 与片上存储之间搬多少数据	通过 Tiling / Fusion 显著减少无效读写

11.3 FlashAttention-2：并行策略与性能改进

从 IO-Aware 到更好的 GPU 并行 FlashAttention 第一代已经建立了 IO-Aware 的基本算法框架，并保持 Exact Attention。

FlashAttention-2 不重新定义 Attention，而是在相同核心思路上进一步改进 GPU 的 Work Partition。

可以将改进方向概括为：

- 减少 Softmax、Mask、Rescaling 等非矩阵乘法操作的相对开销；
- 改善不同 Thread Block / Warp 之间的工作分配；
- 提升长序列或不同 Head Dimension 下的并行利用率；
- 让更多时间落在 GPU 擅长的矩阵乘法路径上。

因此，可以把两代算法的关系理解为

FlashAttention 通过 IO-aware 计算实现精确 Attention；FlashAttention-2 进一步改进 Work Partition 与并行利用率。

进一步到

算法与 Kernel 的区别 在阅读 FlashAttention-2 时，需要区分两个层次：

Mathematical Attention Algorithm 与 GPU Kernel Scheduling.

很多性能提升并不是因为 Attention 公式发生了新的数学变化，而是因为同样的计算被更合理地映射到 GPU Thread Block、Warp、Shared Memory 与矩阵乘法单元。

Prefill 与 Decode 中的 Attention 瓶颈：Prefill 对于长度为 L 的 Prompt，Prefill 可以并行处理多个 Query：$\boldsymbol{Q} \in \mathbb{R}^{B\times H\times L\times D}$. 此时 Query Length 与 KV Length 都较大，Attention 包含大量二维 Score 计算，因此高效 Dense Attention Kernel 非常重要。

Decode 自回归 Decode 每一步通常只有一个或少量新 Query，即 $L_q \approx 1$，但需要读取历史 KV Cache：

$$K, V \in \mathbb{R}^{B\times H_{\text{kv}}\times L_{\text{kv}}\times D}.$$

此时计算形态更接近 $[1, D] \times [L_{\text{kv}}, D]^{\mathsf{T}}$，性能瓶颈会更强地受到 KV Cache 读取、Memory Bandwidth、Batching 与调度影响。

维度	Prefill	Decode
Query Length	通常较长	通常约为 1
主要计算	大规模并行 Attention / GEMM	单步增量 Attention
典型瓶颈	Compute + Attention IO	KV Cache Read + Memory + Scheduling
优化重点	FlashAttention、Chunked Prefill 等	Paged KV Cache、Batching、Decode Kernel 等

这个差异将直接引出下一章的 vLLM、PagedAttention 与 Continuous Batching。

FlashAttention 与 MQA、GQA、MLA 的关系 MHA、MQA、GQA、MLA 与 FlashAttention 不处于同一个技术维度。

技术	主要改变什么	主要目标
MQA	KV Head 数量	降低 KV Cache
GQA	Query 与 KV Head 分组关系	表达能力与 KV Cache 折中
MLA	KV Representation	压缩 KV Cache 表示
FlashAttention	Attention Kernel 的计算顺序与 IO	减少中间 Tensor 与 HBM Traffic

因此，一个模型可以同时使用 GQA 与 FlashAttention，也可以使用其他 KV 组织结构配合 IO-Aware Kernel。

模型结构决定“算什么”；FlashAttention 主要决定“同样的 Attention 应该怎样更高效地算”。

11.4 Python：标准 Attention 与 Tiled Online Attention

分块 Attention 的正确性验证 FlashAttention 的性能来自 CUDA / Triton 等低层 Kernel 与硬件映射，普通 Python for-loop 无法达到同样速度。PyTorch 分块 Online Softmax 示例用于验证两点：

1. 不物化完整 Probability Matrix，也可以分块得到 Attention Output；
2. 在浮点误差允许范围内，分块 Online Attention 与标准 Attention 结果一致。

下面代码使用 $[B, H, L, D]$ 作为统一输入 Shape，并支持 Causal Mask。

完整实现示例

```python
import math
import torch

def standard_attention(q, k, v, causal=False):
    scores = q @ k.transpose(-2, -1)
    scores = scores / math.sqrt(q.size(-1))

    if causal:
        Lq, Lk = q.size(-2), k.size(-2)
        mask = torch.tril(
            torch.ones(Lq, Lk, dtype=torch.bool, device=q.device)
        )
        scores = scores.masked_fill(~mask, float("-inf"))

    probs = torch.softmax(scores, dim=-1)
    return probs @ v

def tiled_online_attention(q, k, v, block_size=4, causal=False):
    B, H, Lq, D = q.shape
    Lk = k.size(-2)
    output = torch.empty_like(q)

    for q0 in range(0, Lq, block_size):
        q1 = min(q0 + block_size, Lq)
        q_tile = q[:, :, q0:q1].float()

        rows = q1 - q0
        m = torch.full(
            (B, H, rows, 1),
            float("-inf"),
            device=q.device,
        )
        ell = torch.zeros(B, H, rows, 1, device=q.device)
        acc = torch.zeros(B, H, rows, D, device=q.device)
```

```
35
36          for k0 in range(0, Lk, block_size):
37              k1 = min(k0 + block_size, Lk)
38
39              if causal and k0 >= q1:
40                  break
41
42              k_tile = k[:, :, k0:k1].float()
43              v_tile = v[:, :, k0:k1].float()
44
45              scores = q_tile @ k_tile.transpose(-2, -1)
46              scores = scores / math.sqrt(D)
47
48              if causal:
49                  qi = torch.arange(q0, q1, device=q.device)[:, None]
50                  kj = torch.arange(k0, k1, device=q.device)[None, :]
51                  allowed = kj <= qi
52                  scores = scores.masked_fill(
53                      ~allowed[None, None],
54                      float("-inf"),
55                  )
56
57              tile_max = scores.max(dim=-1, keepdim=True).values
58              m_new = torch.maximum(m, tile_max)
59
60              old_scale = torch.exp(m - m_new)
61              p = torch.exp(scores - m_new)
62
63              ell = old_scale * ell + p.sum(dim=-1, keepdim=True)
64              acc = old_scale * acc + p @ v_tile
65              m = m_new
66
67          output[:, :, q0:q1] = (acc / ell).to(q.dtype)
68
69      return output
70
71  def test_tiled_attention():
72      torch.manual_seed(42)
73
74      B, H, L, D = 2, 4, 16, 32
```

```
    q = torch.randn(B, H, L, D)
    k = torch.randn(B, H, L, D)
    v = torch.randn(B, H, L, D)

    ref = standard_attention(q, k, v, causal=True)
    tiled = tiled_online_attention(
        q, k, v, block_size=4, causal=True
    )

    max_error = (ref - tiled).abs().max().item()

    print("output:", tiled.shape)
    print("max error:", max_error)

    assert tiled.shape == ref.shape
    assert torch.allclose(ref, tiled, atol=1e-5, rtol=1e-5)

    print("all tests passed")

if __name__ == "__main__":
    test_tiled_attention()
```

这段实现的重点是状态 m, ℓ, acc. 它们分别对应 Running Maximum、Running Normalizer 与未归一化 Output Accumulator。

这里的 Python 循环用于展示数学过程。高性能实现需要专用 Kernel，将 Tiling、共享内存、Warp 级并行、矩阵乘法和数据加载协同优化。

11.5 PyTorch SDPA 与工程实现

现代深度学习框架通常提供统一的 Scaled Dot-Product Attention 接口。工程代码可以表达为：

```python
import torch.nn.functional as F

out = F.scaled_dot_product_attention(
    q,
    k,
    v,
    attn_mask=None,
    dropout_p=0.0,
    is_causal=True,
)
```

Attention API 是调用接口，Selected Backend / Kernel 才是实际执行实现。框架会根据设备、数据类型、Shape 和运行环境选择 Backend；调用统一 API 并不意味着需要手写 Online Softmax。

常见误区

误区	正确理解
FlashAttention 把 Attention 变成 $O(L)$	Dense Attention 的主要算术复杂度仍近似 $O(L^2D)$
FlashAttention 是 Attention 近似算法	它通过重排计算与 Online Softmax 保持标准 Attention 目标
FlashAttention 与 GQA 是竞争关系	GQA 是结构优化，FlashAttention 是 Kernel / IO 优化，可组合
FlashAttention 就是 KV Cache	FlashAttention 主要优化 Attention 计算；KV Cache 是增量推理状态
只要减少 FLOPs 就一定更快	GPU 性能还受到 HBM IO、Kernel Launch、并行度等因素限制
Python Tiling 就等于 FlashAttention	普通 Python for-loop 只展示计算思路，不具备实际 FlashAttention Kernel 的性能

11.6 FlashAttention 的适用边界与总结

可以把整章压缩为五个层次。

第一层：Attention Mathematics

$$O = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{D}}\right)V.$$

第二层：Memory Problem

$$\boldsymbol{QK}^{\mathsf{T}} \longrightarrow [B, H, L, L]$$

带来大规模 Intermediate Tensor 与 HBM Traffic。

第三层：Tiling

$$Q_i \times K_j^{\mathsf{T}}$$

逐块计算，不长期保存完整 Score Matrix。

第四层：Online Softmax 维护 m, ℓ, A，使每一个 Tile 都能正确更新全局 Softmax 与 Output。

第五层：GPU Mapping 利用片上存储、Kernel Fusion、并行 Work Partition，把更多时间用于高吞吐矩阵乘法，并减少昂贵 HBM 往返。

最终路线为

第 12 章

vLLM 与高性能 LLM 推理

前一章从单次 Attention Kernel 的角度讨论了 FlashAttention：在不改变标准 Attention 数学目标的前提下，通过 Tiling、Online Softmax 与更好的 GPU IO 降低中间 Tensor 与 HBM Traffic。但当一个 LLM 真正作为在线服务运行时，系统同时面对的是很多长度不同、到达时间不同、生成长度未知的请求。

此时需要解决的问题不再只是“一次 Attention 怎么算得更快”，而是“大量请求如何共享 GPU、KV Cache 与计算资源”。

对于 Decoder-only LLM，一个请求通常包含两个阶段：

对于 Decoder-only LLM，请求先执行 Prefill，再进入 Decode。

Prefill 读取整个 Prompt，并为历史 token 建立 KV Cache；Decode 则逐 token 生成，每一步读取已有 KV Cache、追加新 KV，并继续采样下一个 token。

如果 Serving Runtime 只是把请求按固定 Batch 处理，就容易产生显存碎片、GPU 空槽、长 Prompt 阻塞 Decode、相同 Prefix 被重复计算等问题。vLLM 的代表性设计之一 PagedAttention，借鉴虚拟内存分页思想，把 KV Cache 划分为 Block，并使用 Block Table 将每个请求的逻辑 KV Block 映射到物理 GPU Block。进一步配合 Continuous Batching、Prefix Caching、Chunked Prefill、Speculative Decoding 与多 GPU Parallelism，就构成现代 LLM Serving 的核心优化栈。

推理服务的优化路径为

理解 vLLM 时不要只记 PagedAttention。高性能 Serving 是 Memory Management、Scheduling、Attention Kernel、Caching 与 Distributed Execution 的组合。

12.1 LLM 推理的两个阶段：Prefill 与 Decode

Prefill 设 Prompt 长度为 L_p。Prefill 一次输入整个 Prompt：$x_1, x_2, \ldots, x_{L_p}$. 在 Causal Mask 下，多个位置的 Hidden State 可以并行计算。对于每一层 Attention，还会得到需要保留到后续生成阶段的 Key 与 Value：

$$K_{\text{cache}}, \qquad V_{\text{cache}}.$$

因此 Prefill 完成两个任务：

$$\text{Prompt Representation} + \text{Build KV Cache}.$$

如果 Prompt 很长，Prefill 包含较大的矩阵乘法与 Attention，通常具有较高的并行度。

Decode 完成 Prefill 后，模型开始逐 token 生成。第 t 步只输入一个新 token 的表示，并读取历史 KV：

$$\boldsymbol{K}_{1:t}, \qquad \boldsymbol{V}_{1:t}.$$

新的 Query 可以概念化为 $\boldsymbol{Q}_t \in \mathbb{R}^{B\times H\times 1\times D}$, 而历史 KV 长度不断增长：$L_{\text{kv}} = L_p + t$.

每一步完成：

每个 Decode Step 使用当前 Query 读取历史 KV，计算 Attention，经 LM Head 得到 logits，再选择下一个 token。

为什么两个阶段的瓶颈不同

维度	Prefill	Decode
输入 token 数	一次处理较多 Prompt Token	每步通常新增一个 Token
矩阵 Shape	较大、并行度高	小 Query + 长 KV
KV Cache	创建	读取并追加
典型瓶颈	Compute、Attention IO	Memory Bandwidth、KV Read、Scheduling
常见优化	FlashAttention、Chunked Prefill	PagedAttention、Continuous Batching、Decode Kernel

Serving 性能指标：Latency 与 Throughput 在线 LLM Serving 通常同时关心低延迟和高吞吐。Latency 描述单个请求等待多久，Throughput 描述单位时间内系统完成多少工作。

对于流式生成，需要进一步区分：

$$\text{TTFT} = \text{Time To First Token},$$

以及

$$\text{TPOT} = \text{Time Per Output Token}.$$

TTFT 强烈受到 Queueing 与 Prefill 影响；TPOT 则更多反映 Decode 阶段的迭代速度。

Tokens per Second 与 Requests per Second 吞吐可以用不同尺度衡量：

$$\text{Tokens/s}, \qquad \text{Requests/s}.$$

若请求长度差异很大，只看 Requests/s 可能无法反映真实计算负载；只看 Tokens/s 也可能掩盖某些请求等待时间过长的问题。

因此工程上通常需要同时关注：

- TTFT；
- TPOT / Inter-Token Latency；
- Token Throughput；
- Request Throughput；
- GPU Utilization；
- Peak / Average KV Cache Utilization。

Latency 与 Throughput 的权衡 更大的 Batch 往往有利于提高 GPU 利用率：

但等待更多请求凑 Batch，也可能导致 Queueing Delay ↑.

因此 Serving Scheduler 的核心任务不是单纯最大化 Batch Size，而是动态平衡：

$$\text{Latency} \quad \Longleftrightarrow \quad \text{Throughput}.$$

12.2 KV Cache：推理显存的核心状态

KV Cache 为什么存在 如果每次生成一个新 token 都重新计算整个历史序列的 Key / Value，会重复大量工作。KV Cache 保存历史层的 $\boldsymbol{K}/\boldsymbol{V}$，使下一步只需要计算新 token 对应的 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$，并复用已有历史 $\boldsymbol{K}/\boldsymbol{V}$。

若一层使用 H_{kv} 个 KV Head，每个 Head Dimension 为 D，序列长度为 L，每个元素占 b Byte，则该层每个请求的 KV Cache 理论量级约为

$$M_{\text{KV,layer}} \approx 2LH_{\text{kv}}Db.$$

其中系数 2 对应 Key 和 Value。

如果有 N_{layer} 层，则

$$M_{\text{KV}} \approx 2N_{\text{layer}}LH_{\text{kv}}Db.$$

KV Cache 与模型权重是两块显存 模型量化可以把 Weight 从 FP16 压缩到 INT8 / INT4：$M_{\text{weight}} \downarrow$. 但这并不自动减少 KV Cache。长上下文、大 Batch 或大量并发请求下，即使模型权重已经很小，也可能出现 M_{KV} 成为主要显存占用。

Weight Quantization 与 KV Cache Management 解决不同问题：前者压缩模型权重，后者管理自回归推理期间的历史 K/V。

传统连续 KV 分配的问题：生成长度事先未知 请求开始时通常不知道最终会生成多少 token。如果为每个请求提前按照最大长度 L_{max} 预留一块连续显存，而实际只使用 $L_{\text{actual}} \ll L_{\text{max}}$，就会产生大量未使用空间。

请求长度不一致 假设同时有三个请求：

A 的 Prompt / 输出长度分别为 128 / 300 tokens，B 为 1024 / 1100，C 为 64 / 800。

它们的 Prompt 长度和生成长度都不同。如果每个请求使用一块独立连续 KV 区域，随着请求进入、结束和扩展，Memory Allocator 很容易面对碎片化与难以复用的问题。

固定 Batch 的资源浪费 传统固定 Batch 中，只要某些序列提前完成，剩余 Batch Slot 可能暂时空闲；而还在等待队列中的新请求不能立即进入当前 Decode Batch。

这说明高性能 Serving 需要同时优化两种资源：Memory Space 与 Compute Slots.

12.3 PagedAttention：分页式 KV Cache 管理

从 Virtual Memory 到 KV Block PagedAttention 的关键思想是：**一个请求的逻辑 KV Cache 不需要在物理显存中连续**。

把 KV Cache 划分为固定大小的 Block。假设每个 Block 保存 T_b 个 token 的 KV，则 Request A 的逻辑序列可以表示为

$$\text{Logical Blocks} = [0, 1, 2, \ldots].$$

这些 Logical Block 可以映射到任意可用 Physical Block，例如

$$0 \to 7, \qquad 1 \to 2, \qquad 2 \to 11.$$

Block Table 保存 Logical Block ID 到 Physical Block ID 的映射。

为什么像操作系统分页 这个思想与 Virtual Memory 类似：

概念	操作系统	Paged KV Cache
逻辑地址	Virtual Address	Request Logical Token / Block
映射表	Page Table	Block Table
物理位置	Physical Page	GPU KV Physical Block
按需扩展	Allocate Page	Allocate New KV Block

注意这只是帮助理解的类比。LLM KV Cache Block 的数据布局、Kernel 和生命周期与通用 CPU Virtual Memory 并不完全相同。

按需分配 随着 Request A 生成更多 token，Block Manager 只在当前 Block 用满后分配新的 Physical Block，因此无需在请求开始时一次性预留最大生成长度。

PagedAttention 如何执行 Attention：从 Block Table 找到历史 KV 对于当前 Query $\boldsymbol{q}_t$，Attention 仍然需要历史：

$$\boldsymbol{K}_{1:t}, \qquad \boldsymbol{V}_{1:t}.$$

区别只在于这些 $\boldsymbol{K}/\boldsymbol{V}$ 可能分散在不同 Physical Block 中。

Attention Kernel 根据 Request 的 Block Table：$[7, 2, 11, \ldots]$ 依次读取对应 Physical Block 中的 $\boldsymbol{K}/\boldsymbol{V}$，并完成

$$\text{softmax}\left(\frac{\boldsymbol{q}_t\boldsymbol{K}^\top}{\sqrt{D}}\right)\boldsymbol{V}.$$

PagedAttention 不是新的 Attention 数学公式。它主要改变 KV Cache 的物理存储与访问方式。

Block Size 的权衡 Block 越小：$T_b \downarrow$ 通常意味着最后一个 Block 内部浪费更少，但 Block Table 更长、管理粒度更细。

Block 越大，即 $T_b \uparrow$，管理 Metadata 较少，但请求结束时最后一个 Block 可能留下更多空位。

因此 Block Size 也属于

$$\text{Memory Efficiency} \quad \Longleftrightarrow \quad \text{Management / Kernel Efficiency}$$

的权衡。

12.4 请求调度与生成优化

Static Batching 最简单的方法是收集一组请求 A, B, C，组成固定 Batch，并让它们一起运行到结束。若 C 先完成，而 A/B 仍然继续生成，C 对应 Slot 可能无法立刻被新请求复用。

Dynamic Batching Dynamic Batching 通常表示系统根据到达时间与当前负载动态组合请求，而不是预先固定所有 Batch。

但在 LLM Decode 中，更关键的是粒度进一步细化到每一个迭代 Step。

Continuous Batching Continuous Batching 可以在每一轮 Decode 前重新组织 Running Set。例如：

$$\begin{aligned}
\text{Step 1}: &\quad A, B, C,\\
\text{Step 2}: &\quad A, B, C,\\
&\text{C finished},\\
\text{Step 3}: &\quad A, B, D.
\end{aligned}$$

因此新请求 D 不需要等 A 和 B 全部结束。

方式	Batch 何时变化	主要问题 / 优势
Static Batching	整个请求组结束后	简单，但容易出现空闲 Slot
Dynamic Batching	请求进入前动态组合	可以适应到达流量
Continuous Batching	Decode Iteration 之间持续调整	完成请求可立即退出，新请求可以持续进入

Continuous Batching 的调度逻辑：Waiting、Running 与 Finished 一个简化 Scheduler 可以维护三个集合：

$$\text{Waiting}, \qquad \text{Running}, \qquad \text{Finished}.$$

每轮迭代：

1. 清理已经完成的请求；
2. 释放其 KV Physical Block；
3. 根据显存与 Batch Capacity 从 Waiting 中 Admit 新请求；
4. 对 Running Requests 执行一次 Prefill Chunk 或 Decode Step；
5. 更新每个请求的 Length、状态与 Sampling Result。

Scheduler 不只是“拼 Batch” 真正的 Scheduler 还要同时考虑：

- 当前可用 KV Block；
- 每个请求是 Prefill 还是 Decode；
- Prompt Length 与剩余 Prefill Token；
- Decode Request 的延迟；
- 最大 Batched Token 数；
- 优先级与抢占策略；
- Distributed Worker 的同步。

因此，LLM Serving Scheduler 本质上是在做多资源约束下的动态调度。

Prefix Caching：为什么 Prefix 会重复 多个请求可能共享相同的 System Prompt、模板或文档前缀。例如

$$\text{Request A} = P + Q_A,$$

$$\text{Request B} = P + Q_B,$$

其中 P 是相同 Prefix。

如果两个请求都从头 Prefill，P 会被重复计算，并重复产生相同的历史 KV。

复用 Prefix KV Prefix Caching 的思路是

后续请求命中相同 Prefix 时直接复用相应 KV Block。

于是 Request B 可以从 Cached KV(P) 继续计算 Q_B，减少重复 Prefill。

Prefix Caching 与普通 KV Cache 的区别 普通 KV Cache 主要服务于一个正在生成的请求：Current Request History. Prefix Cache 则强调：Cross-Request Reuse.

因此二者都缓存 $\boldsymbol{K}/\boldsymbol{V}$，但生命周期和复用范围不同。

Chunked Prefill：长 Prompt 为什么会阻塞 Decode 假设系统已经有多个请求正在 Decode，此时来了一个超长 Prompt。如果一次把整个 Prompt 作为一个 Prefill Batch 执行，可能持续占用大量计算资源，使已有 Decode Request 的下一个 token 等待更久，导致 TPOT 上升。

把 Prefill 拆成 Chunk Chunked Prefill 将长度 L_p 的 Prompt 拆成 $C_1, C_2, \ldots, C_n$.

Scheduler 可以在不同迭代中交错安排：

$$\text{Decode Tokens} + \text{Prefill Chunk}.$$

这样不必让一个长 Prompt 独占整个调度周期。

TTFT 与 TPOT 的权衡 如果 Prefill Chunk 太小，长 Prompt 需要更多调度轮次，可能增加其 TTFT；如果 Chunk 太大，又可能影响正在 Decode 的请求。

因此 Chunked Prefill 体现的是：

$$\text{Long-Prompt TTFT} \quad \Longleftrightarrow \quad \text{Decode TPOT}.$$

Speculative Decoding：为什么 Decode 是串行的 普通自回归生成：

$$x_t \longrightarrow x_{t+1} \longrightarrow x_{t+2}$$

存在严格依赖。大型 Target Model 每一步通常只能确认一个新的 token。

Draft + Verify Speculative Decoding 引入一个更便宜的 Draft Model。Draft Model 先提出多个候选：

$$\tilde{x}_{t+1}, \tilde{x}_{t+2}, \ldots, \tilde{x}_{t+k}.$$

Target Model 一次前向验证这一段 Candidate。在满足接受规则的前提下，一次可以确认多个 token，从而减少 Target Model 串行迭代次数。

不是“大模型直接照抄小模型” Speculative Decoding 不是直接采用小模型输出。正确的 Speculative Sampling 需要定义 Proposal Distribution 与 Target Distribution 之间的接受 / 拒绝规则，在保持 Target Model 分布的同时减少串行 Decode Step。

12.5 LLM 推理的分布式并行

切分单层 Tensor 如果一个 Linear Weight $\boldsymbol{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 太大，可以沿某一维切分：

$$\boldsymbol{W} = \begin{bmatrix} \boldsymbol{W}_1 \\ \boldsymbol{W}_2 \end{bmatrix}$$

或 $\boldsymbol{W} = [\boldsymbol{W}_1 \ \boldsymbol{W}_2]$.

不同 GPU 保存不同 Shard，并协同完成同一层的计算。这就是 Tensor Parallel 的基本思想。

通信不可忽略 由于同一层跨多个 GPU，局部结果之间通常需要 Collective Communication，例如 All-Reduce、All-Gather 或 Reduce-Scatter 等操作。

因此，Tensor Parallel 是否带来收益，取决于分片计算节省的开销能否抵消跨设备通信成本。

Pipeline Parallelism：按 Layer 切分 Pipeline Parallel 把不同层放到不同设备：

$$\begin{aligned} &\text{GPU 0}: \quad \text{Layer } 1 \sim 10, \\ &\text{GPU 1}: \quad \text{Layer } 11 \sim 20, \\ &\text{GPU 2}: \quad \text{Layer } 21 \sim 30. \end{aligned}$$

Activation 沿 Pipeline Stage 传递：

在 Pipeline Parallel 中，Activation 按 Stage 顺序从 GPU 0 传到 GPU 1，再传到 GPU 2。

Pipeline Bubble 与 Stage Balance 如果不同 Stage 计算时间差异很大，较快 Stage 会等待较慢 Stage。若只有单个请求顺序经过多个 Stage，还会出现 Pipeline Bubble。

Serving 场景中需要通过 Microbatch、并发请求与合理 Layer Partition 改善利用率。

Data Parallelism Data Parallel 在多个设备上保留完整或逻辑完整的模型副本，把不同请求分配给不同 Replica：

$$\begin{aligned}\text{Replica } 0: &\quad A, B, C,\\ \text{Replica } 1: &\quad D, E, F.\end{aligned}$$

它最适合解决：

与 Tensor / Pipeline Parallel 不同，Data Parallel 并不主要用于让单个模型跨设备才能放得下，而是通过模型副本服务更多独立请求。

TP、PP 与 DP 的统一比较

并行方式	切分对象	主要用途	主要代价
Tensor Parallel	单层 Tensor / Matrix	单层跨多个 GPU 计算	高频 Collective Communication
Pipeline Parallel	Transformer Layers	超深 / 超大模型跨 Stage 放置	Stage Communication 与 Pipeline Bubble
Data Parallel	Requests / Replicas	横向扩展 Serving Throughput	多份模型权重与负载均衡

三者可以组合：

$$\text{TP} + \text{PP} + \text{DP},$$

形成多维 Distributed Inference Topology。

对于 MoE 模型还可能出现 Expert Parallel，但它属于 Expert Placement 与 All-to-All Routing 的另一维度，已经在 MoE 章节单独讨论。

12.6 一次请求的完整 Serving 生命周期

请求通常依次经过 Tokenization、排队、Prefix Cache 检查、Prefill、KV Block 分配、Decode 与 Token Streaming；结束后释放 KV Blocks。

使用 Speculative Decoding 时，Decode 阶段还会增加 Draft Model 生成候选与 Target Model 验证步骤。如果使用多 GPU，请求还需要经过对应的 TP / PP / DP Worker 与通信路径。

vLLM 与 FlashAttention、Quantization 的关系 这些优化技术位于不同层次。

技术	主要优化对象	所属层次
Quantization	Weight / Activation bit 数	Numerical Representation / Kernel
FlashAttention	Attention 中间 Tensor 与 IO	Attention Kernel
GQA / MLA	KV Head / KV Representation	Model Architecture
PagedAttention	KV Cache Physical Layout	Memory Management
Continuous Batching	Running Request Set	Scheduler
Prefix Caching	重复 Prefix Prefill	Cross-Request Cache
Speculative Decoding	Target Model 串行 Decode Step	Decoding Algorithm
TP / PP / DP	多设备计算与请求分布	Distributed Runtime

因此它们不是互相替代关系。一个实际推理栈完全可能同时使用：

$$\text{Quantized Weights} + \text{GQA} + \text{FlashAttention}$$
$$+\text{Paged KV Cache} + \text{Continuous Batching} + \text{Tensor Parallel}.$$

12.7 Python：简化 Paged KV Cache

Block Manager 下面使用纯 Python 模拟 Logical Block 到 Physical Block 的映射。它不存储真实 $\boldsymbol{K}/\boldsymbol{V}$ Tensor，只关注 Paged Memory Management。

```python
from collections import deque

class BlockManager:
    def __init__(self, num_blocks, block_size):
        self.block_size = block_size
        self.free_blocks = deque(range(num_blocks))
        self.tables = {}
        self.lengths = {}

    def add_request(self, request_id):
        assert request_id not in self.tables
        self.tables[request_id] = []
        self.lengths[request_id] = 0

    def _allocate_block(self):
        if not self.free_blocks:
            raise RuntimeError("no free KV blocks")
        return self.free_blocks.popleft()

    def append_token(self, request_id):
        length = self.lengths[request_id]

        if length % self.block_size == 0:
            physical = self._allocate_block()
            self.tables[request_id].append(physical)

        self.lengths[request_id] += 1

    def append_tokens(self, request_id, n):
        for _ in range(n):
            self.append_token(request_id)

    def locate(self, request_id, token_index):
        logical_block = token_index // self.block_size
        offset = token_index % self.block_size
        physical_block = self.tables[request_id][logical_block]
        return physical_block, offset

    def free_request(self, request_id):
```

```
        for block in self.tables.pop(request_id):
            self.free_blocks.append(block)
        self.lengths.pop(request_id)

    def block_table(self, request_id):
        return list(self.tables[request_id])

def test_block_manager():
    manager = BlockManager(num_blocks=8, block_size=4)

    manager.add_request("A")
    manager.append_tokens("A", 10)

    print("A table:", manager.block_table("A"))
    print("token 7:", manager.locate("A", 7))

    assert len(manager.block_table("A")) == 3

    before = len(manager.free_blocks)
    manager.free_request("A")
    after = len(manager.free_blocks)

    assert after > before
    print("block manager passed")

if __name__ == "__main__":
    test_block_manager()
```

假设 Block Size 为 4，Request A 有 10 个 token，则需要 $\lceil 10/4 \rceil = 3$ 个 Physical Block。最后一个 Block 只使用两个位置，但系统无需提前为一个很大的 $L_{\max}$ 分配连续空间。

Python 实现：Continuous Batching 调度模拟 下面进一步加入一个简化 Scheduler。每个请求只记录：

$$\text{prompt length}, \quad \text{generated tokens}, \quad \text{max new tokens}.$$

为了突出 Continuous Batching，本例假设 Prompt 已经完成 Prefill，每轮为所有 Running Request 各生成一个 token。

```python
from collections import deque
from dataclasses import dataclass

@dataclass
class Request:
    request_id: str
    prompt_len: int
    max_new_tokens: int
    generated: int = 0

    @property
    def finished(self):
        return self.generated >= self.max_new_tokens

class ContinuousBatchScheduler:
    def __init__(self, max_running):
        self.max_running = max_running
        self.waiting = deque()
        self.running = []
        self.finished = []

    def submit(self, request):
        self.waiting.append(request)

    def _admit(self):
        while (
            len(self.running) < self.max_running
            and self.waiting
        ):
            self.running.append(self.waiting.popleft())

    def step(self):
        self._admit()

        active_ids = [r.request_id for r in self.running]

        for request in self.running:
            request.generated += 1

```

```
        still_running = []

        for request in self.running:
            if request.finished:
                self.finished.append(request)
            else:
                still_running.append(request)

        self.running = still_running

        return active_ids

def test_scheduler():
    scheduler = ContinuousBatchScheduler(max_running=2)

    scheduler.submit(Request("A", 10, 3))
    scheduler.submit(Request("B", 20, 1))
    scheduler.submit(Request("C", 30, 2))

    step = 0

    while scheduler.waiting or scheduler.running:
        step += 1
        active = scheduler.step()
        print(f"step {step}: {active}")

    done = [r.request_id for r in scheduler.finished]

    assert set(done) == {"A", "B", "C"}
    print("scheduler passed")

if __name__ == "__main__":
    test_scheduler()
```

运行逻辑可以出现类似：

$$\begin{aligned}\text{Step 1}: &\quad A, B,\\ \text{Step 2}: &\quad A, C,\\ \text{Step 3}: &\quad A, C.\end{aligned}$$

B 在第一步完成后，C 可以立即进入 Running Batch，而不需要等待 A 完成。

12.8 推理优化的统一性能框架

现代 LLM 推理可以从四种瓶颈统一理解。

Compute 典型问题：

$$\text{Large GEMM}, \quad \text{Attention Compute}.$$

对应技术包括：

$$\text{FlashAttention}, \quad \text{Tensor Parallel}, \quad \text{Speculative Decoding}.$$

Memory 典型问题：

$$\text{Weight Memory}, \quad \text{KV Cache}, \quad \text{Memory Bandwidth}.$$

对应：

$$\text{Quantization}, \quad \text{PagedAttention}, \quad \text{GQA / MLA}, \quad \text{KV Cache Quantization}.$$

Communication 多 GPU 下需要考虑：

$$\text{All-Reduce}, \quad \text{All-Gather}, \quad \text{Pipeline Transfer}, \quad \text{All-to-All for MoE}.$$

对应：TP / PP / Expert Parallel.

Scheduling 在线服务还需要处理：

$$\text{Arrival Time}, \quad \text{Request Length}, \quad \text{Prefill / Decode Mix}.$$

对应：

$$\text{Continuous Batching}, \quad \text{Chunked Prefill}, \quad \text{Prefix Cache Scheduling}.$$

最终可以写成：

$$\text{Inference Performance} = f(\text{Compute}, \text{Memory}, \text{Communication}, \text{Scheduling}).$$

常见误区

误区	正确理解
PagedAttention 是一种新的 Attention Formula	它主要改变 KV Cache 的物理 Block Layout 与访问方式
Continuous Batching 就是固定 Batch Size	Running Set 可以在 Decode Iteration 之间持续变化
Prefix Caching 等于当前请求自己的 KV Cache	Prefix Caching 强调跨请求复用共享前缀
Chunked Prefill 只是把 Tensor 随便切小	它服务于 Prefill 与 Decode 的调度公平和性能权衡
Speculative Decoding 是直接采用小模型输出	Candidate 需要由 Target Model 验证并执行接受规则
Tensor Parallel 与 Data Parallel 相同	TP 切单个模型 Tensor，DP 主要复制模型并分发请求
INT4 模型就不需要 Paged KV Cache	Weight Memory 与 KV Cache 是不同显存来源
FlashAttention 与 PagedAttention 是同一个东西	前者主要优化 Attention IO，后者主要优化 KV Cache Memory Management

高性能 Serving 的主要优化问题 各项技术分别解决以下问题：

问题	代表技术
权重太大怎么办?	Quantization
Attention 中间 IO 太大怎么办?	FlashAttention
KV Cache 太大怎么办?	MQA、GQA、MLA、KV Quantization
KV Cache 容易碎片化怎么办?	PagedAttention
请求长度不一样怎么办?	Continuous Batching
重复 Prefix 怎么办?	Prefix Caching
长 Prefill 阻塞 Decode 怎么办?	Chunked Prefill
Decode 串行 Step 太多怎么办?	Speculative Decoding
单 GPU 放不下 / 吞吐不够怎么办?	TP、PP、DP